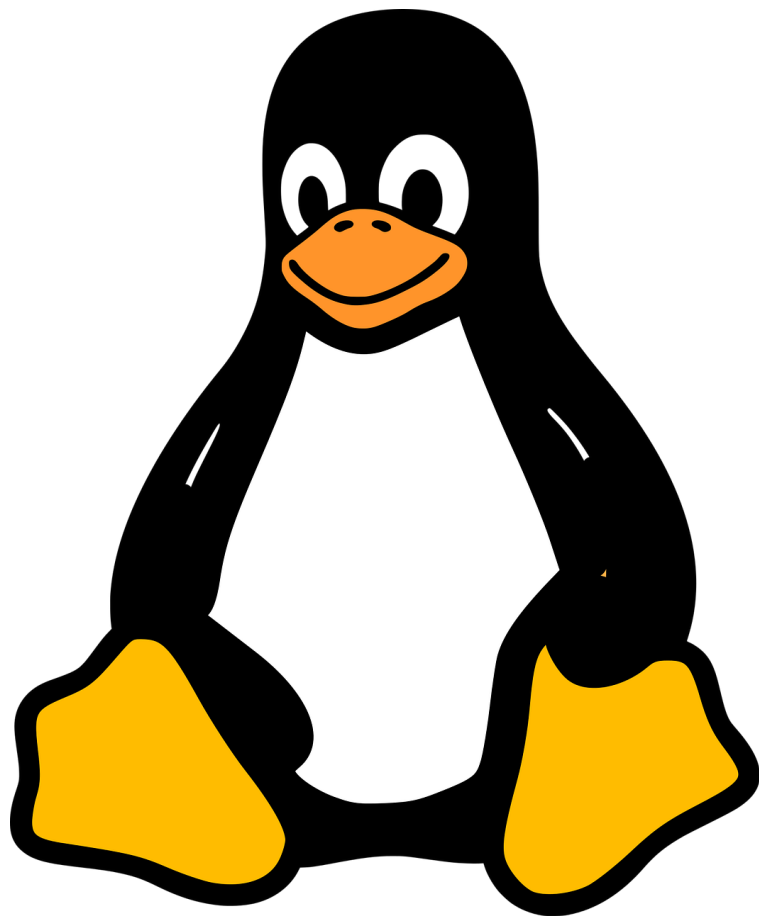




표준 리눅스 엔지니어 200
(엔터프라이즈 리눅스의 재편과 엔지니어
의 선택)

소개

과정

목차

담당자 및 강사

소개

01

엔터프라이즈 리눅스 운영
경험을 보유한 시스템·인프라
엔지니어

02

벤더 종속 없는 표준 리눅스
환경설계·운영·전환하고자
하는 엔지니어

목차

강의 일정은 아래와 같이 진행이 될 예정입니다.

1일차 | 리눅스 생태계와 표준의 붕괴

리눅스 배포판의 역사와 철학

RHEL GPL 정책 변화와 SRPM 이슈

ABI vs API vs Kernel Compatibility

Rocky / Alma / SUSE의 전략적 차이

랩: 배포판별 패키지/커널/라이브러리 구조 비교

목차

2일차 | 표준 리눅스 시스템 구조 이해

부팅 구조 비교 (GRUB2, systemd, initramfs)

systemd 심화 구조

파일 시스템 전략 (XFS, Btrfs, Snapper)

패키지 관리자 비교 (dnf/zypper)

랩: systemd unit 작성 & 장애 복구

목차

3일차 | 마이그레이션과 운영 전략

RHEL → Rocky/Alma 전환 전략

RHEL → SUSE 전환 시 고려사항

glibc / kernel / SELinux 차이

설정 파일 및 서비스 마이그레이션

랩: 실제 마이그레이션 시나리오 실습

목차

4/5일차 | 엔지니어 관점의 리눅스 운영

보안 정책과 하드닝 차이

패치 전략과 장기 운영(LTS)

자동화 도구와 연계 (Ansible 기준)

장애 대응 및 트러블슈팅 전략

랩: 장애 상황 시나리오 기반 대응

강사

강사

강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

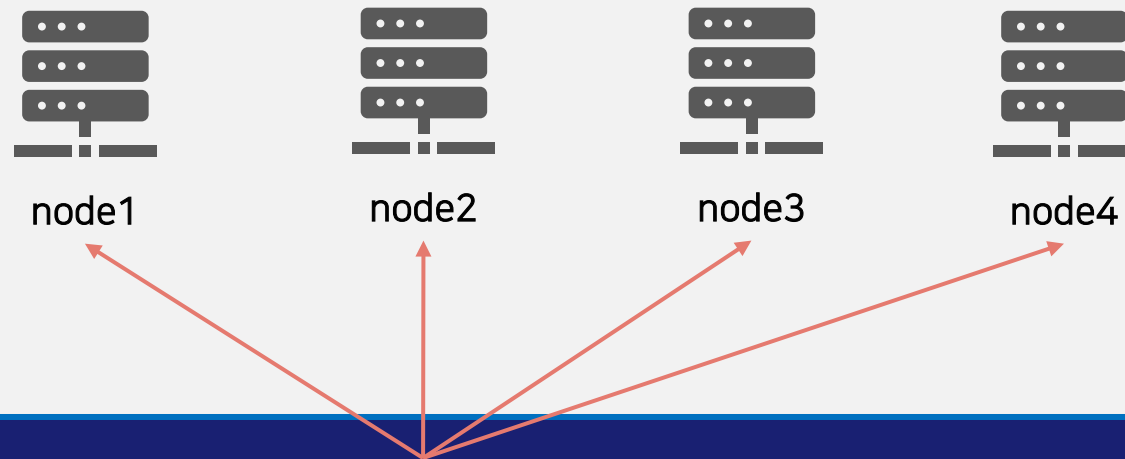
언제든지 질문 및 요청 환영입니다.



LAB

랩 및 교육대상

랩 다이어그램



오픈스택 랩 정보

- 웹 주소: <https://vlab.dustbox.kr>
- 사용자 계정: user
- 비밀번호: vlabhorizon
- 도메인: lab user

위 정보는 강의 진행 시 접근이 가능하며, 이후에는 접근이 불가능 합니다.

오픈소스 소개

리눅스와 오픈소스 관계

약간의 동향 이야기(라이선스)

오픈소스

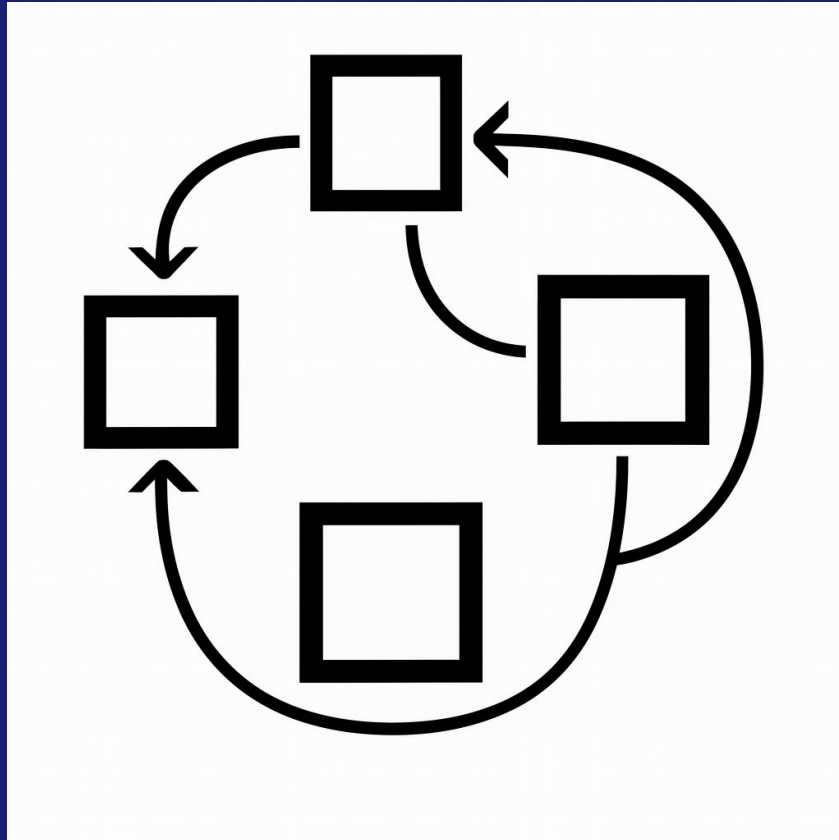
오픈소스는 리처드 스톨만이 GNU도구 및 라이선스를 선언하였음. 리눅스 토발즈는 GNU 도구 중 리눅스 커널(Linux Kernel)를 만들었다. 리처드 스톨만은 Hurd라는 커널을 만들기도 하였음.

선진적인 디자인의 마이크로 커널 디자인(Micro Kernel)를 사용하여서, 당시 성능으로는 구현 및 사용하기가 어려운 부분이 많았음.

- GNU/Linux Kernel
- GNU/Hurd Kernel

현재 허드 커널을 사용하는 배포판은 데비안 리눅스 배포판 밖에 없다.

하드커널



리눅스 커널



GNU/LINUX 라이선스

리눅스의 대다수 소프트웨어는 GNU 기반의 소프트웨어를 사용하고 있다.

GNU와 Unix 소프트웨어와 큰 차이점은 없지만, 라이선스 및 커뮤니티 기반의 생태계를 좀 더 강하게 가지고 있다. 오픈소스에서 많이 비교되는 대상인 BSD 소프트웨어와 제일 큰 차이점은 BSD는 유닉스 코드 기반으로 구성되어 있다.

리눅스는 리눅스 토발즈가 만든 리눅스 커널을 지칭을 하며, 리눅스 커널은 GNU 소프트웨어와 같이 사용하기 때문에 보통 GNU/Linux라고 부른다.

리눅스 커널 뒷 이야기

리눅스 커널은 현재 안드로이드 및 IoT 그리고 엔터프라이즈 시스템에 많이 사용하고 있다.

한때, 리눅스 커널은 애플사의 대표인 스티브 잡스를 통해서, Mac OS의 커널로 사용이 될 뻔 하였으나, 애플에서 비공개라는 조건으로 토발즈에게 요구(?) 하였고, 이에 대해서 리누즈 토발즈는 거절 하였다. (Just Fun에 해당 내용이 나와 있음)

OS X에서 사용하는 커널은 4.4BSD Lite-2기반으로 구성이 되어 있다. 이 프로젝트에서 파생된 BSD, MIT라이선스 기반으로 작성된 프로그램 다윈(Darwin)으로 공개가 되고 있다. 현재 BSD에서 사용하는 커널은 XNU라고 부르고 있으며, 마이크로 커널 기반의 하이브리드 커널 구조를 가지고 있다.

루머링크(?): [퀵링크](#)

오픈소스 라이선스

현재 리눅스는 다음과 아래와 같이 소프트웨어 구성이 되어 있다. 배포판을 만드는 회사는 혹은 단체마다 허용하는 범위가 많이 다르지만, 대다수 상용 리눅스 배포판은 라이선스에 매우 민감하다. 특히, 레드햇 및 수세 리눅스는 매우 민감하게 라이선스 범주를 다룬다.

BSD Software

버클리 소프트웨어 라이선스. 대다수 유닉스 소프트웨어는 BSD 라이선스 기반으로 작성이 되어 있으며, GNU 소프트웨어의 대다수는 라이선스 및 이상으로 인하여 다시 GNU 라이선스 기반으로 재작성이 되었다.

오픈소스 라이선스

GNU Software (Version 2 and 3)

GNU 소프트웨어는 상용 소프트웨어 및 기존 유닉스 라이선스의 단점, 예를 들어 기여코드도 상용 라이선스에 내포가 되는 불합리 및 불공정한 부분을 없애기 위해서 만든 라이선스. 현재 리눅스의 대다수 라이선스 GNU 기반으로 구성되어 있다.

MIT Software

MIT 라이선스는 MIT 대학교에서 만든 라이선스이다. MIT에서 배포되는 소프트웨어 및 기술은 MIT 라이선스로 구성되어 있으며, GNU 라이선스와 호환은 된다. MIT 라이선스는 BSD 라이선스와 매우 흡사한 구조를 가지고 있으며 둘 다 공통점은 소스코드 오픈에 대한 강제성이 없다.

오픈소스 라이선스

LGPL Software

최대한 압축해서 이야기 하면 "GPL 라이선스 라이브러리에 상용 애플리케이션 링크 허용" 정도이다. 본래 GPL 라이선스는 매우 엄격하기 때문에 라이선스에 따르지 않는 바이너리에 대해서 링크 및 허용하지 않는다. 하지만 LGPL 기반의 라이선스 제품에는 상용 제품에 대해서도 링크를 허용한다. 이전에는 GPL3에서는 LGPL에 대해서 허용하지 않았는데, 현재는 GPL3에서도 허용한다.

최근 레드햇 라이선스 이슈

현재 레드햇은 다음과 같은 라이선스 정책을 오픈소스 커뮤니티에 알렸음.

- SRPM에 대한 재배포는 레드햇 서비스 라이선스에 위반
- 개인 사용 및 소스코드에 대해서는 문제가 없으나(GPL 2/3), 레드햇에서 제공한 패키지 기반으로 리-빌드 하는 경우, 앞으로 레드햇 서비스 사용 금지
- Rocky, Alma, Oracle 리눅스 같은 레드햇 클론 버전은 직접적으로 SRPM 복제 불가능

Check: Unauthorized Use of Subscription Services section

https://www.redhat.com/licenses/Appendix_1_Global_English_20230309.pdf

Partner will not use Red Hat Products or Services to create an offering competitive with Red Hat, directly or indirectly,

https://www.redhat.com/licenses/Partner_Agreement_Webversion_North_America_English_20220308.pdf

앞으로 기업용 리눅스

지금 벌어진 일의 본질, 그리고 레드햇 라이선스 변경 이후, 모든 기업용 배포판은 같은 방법을 걸어가고 있다.

Red Hat

→ 소스는 공개하되, *엔터프라이즈* *신뢰*는 계약으로 묶음

CIQ

→ Rocky Linux는 무료, *운영·보증·보안·SLA*는 유료

왜 CIQ의 서브스크립션은 '배신'이 아닌가?

"Rocky는 무료라더니, 이제 돈 받네?" 하지만 정확히 보면,

✓ 무료로 남아있는 것

1. OS 이미지
2. 패키지
3. 커뮤니티 사용

왜 CIQ의 서브스크립션은 '배신'이 아닌가?

✓ 돈을 받는 것

1. 보안 패치 SLA
2. CVE 대응 시간 보장
3. 장기 유지보수(LTS)
4. 인증 하드웨어/소프트웨어
5. 책임 주체

코드는 개인에게 여전히 무료이지만, 상업적인 용도에는 유료이다. 레드햇, 로키 및 오라클은 이와 같은 형태를 가져가고 있고, 아직 수세 리눅스 SRPM에 대해서는 이러한 제한이 아직 없다.

최근 레드햇 라이선스 이슈 결론?

1. 앞으로 100% RHEL 클론 버전은 어렵다.
2. 대다수 레드햇 계열은 ABI/kABI 호환성 위주로 개발 진행이 될 듯 하다.
3. 수세/데비안/알마/우분투 리눅스 배포판으로 다양하게 사용할 예정.
4. "호환성"보다 "책임"을 어느정도 영역까지 지원한가? 이 부분은 기업용 리눅스의 주요 논쟁.
5. ABI는 더 이상 레드햇 리눅스와 100% 호환성 지원하지 않음.
6. 서브스크립션은 이제 보험과 같은 역할.

Popularity on DistroWatch →

Symbol

Stable Release Year →

Linux Mint ←

Name

2.006

Legend:

- Debian/Ubuntu
- Arch
- Red Hat/Fedora/Mandriva
- Slackware
- Gentoo
- Other

Element	Symbol	Stable Release Year	Category
Argon	Ar	2.002	Debian/Ubuntu
Manganese	Ma	2.013	Debian/Ubuntu
Antimony	Ab	2.010	Debian/Ubuntu
Refrigerant	Re	2.017	Debian/Ubuntu
Chlorine	Ch	2.010	Debian/Ubuntu
Paraffin	Pa	2.011	Debian/Ubuntu
Uranium	Ue	2.008	Debian/Ubuntu
Bismuth	Bi	2.008	Debian/Ubuntu
Lithium	Li	2.014	Debian/Ubuntu
Uranium	Us	2.007	Debian/Ubuntu
Uranium	Um	2.015	Debian/Ubuntu
Tantalum	Ta	2.010	Debian/Ubuntu
Pepper	Pe	2.011	Debian/Ubuntu
Du	Du	2.017	Debian/Ubuntu
Paraffin	Pa	2.016	Debian/Ubuntu
Ethanol	Ec	2.008	Debian/Ubuntu
Uranium	Ue	2.016	Debian/Ubuntu
Krypton	Kn	2.017	Debian/Ubuntu
Xenon	Xu	2.006	Debian/Ubuntu
Gallium	Gm	2.010	Debian/Ubuntu
Ruthenium	Ru	2.015	Debian/Ubuntu
Osmium	Os	2.009	Debian/Ubuntu
Zinc	Ze	2.005	Debian/Ubuntu
Antimony	An	2.013	Debian/Ubuntu
Gallium	Gp	2.006	Debian/Ubuntu
Dysprosium	Dp	2.004	Debian/Ubuntu
Uranium	Uk	2.013	Debian/Ubuntu
Uranium	Ub	2.004	Debian/Ubuntu
Neon	Ne	2.010	Debian/Ubuntu
Bismuth	Bi	2.012	Debian/Ubuntu
Krypton	Ku	2.005	Debian/Ubuntu
Osmium	Og	2.017	Debian/Ubuntu
Cadmium	Cz	2.008	Debian/Ubuntu
Sh	Sh	2.017	Debian/Ubuntu
Q	Qo	2.015	Debian/Ubuntu
Uranium	Ug	2.013	Debian/Ubuntu
Sp	Sp	2.012	Debian/Ubuntu
Kn	Kn	2.003	Debian/Ubuntu
Ma	Ma	2.011	Debian/Ubuntu
Sc	Sc	2.007	Debian/Ubuntu
Sa	Sa	2.009	Debian/Ubuntu
Bismuth	Bs	2.016	Debian/Ubuntu
El	El	2.011	Debian/Ubuntu
Mx	Mx	2.016	Debian/Ubuntu
St	St	2.013	Debian/Ubuntu
El	El	2.005	Debian/Ubuntu
Vo	Vo	2.012	Debian/Ubuntu
Em	Em	2.014	Debian/Ubuntu
Bo	Bo	2.012	Debian/Ubuntu
So	So	2.013	Debian/Ubuntu
Ax	Ax	2.010	Debian/Ubuntu
Ro	Ro	2.014	Debian/Ubuntu
Tr	Tr	2.007	Debian/Ubuntu
De	De	1.996	Debian/Ubuntu
Fo	Fo	2.016	Debian/Ubuntu
Pc	Pc	2.005	Debian/Ubuntu
Ro	Ro	2.011	Debian/Ubuntu
Ve	Ve	2.003	Debian/Ubuntu
Ba	Ba	2.014	Debian/Ubuntu
Tp	Tp	2.016	Debian/Ubuntu
Lk	Lk	2.017	Debian/Ubuntu
Ne	Ne	2.017	Debian/Ubuntu
Wa	Wa	2.010	Debian/Ubuntu
Sk	Sk	2.004	Debian/Ubuntu
Us	Us	2.016	Debian/Ubuntu
Gr	Gr	2.004	Debian/Ubuntu
Ip	Ip	2.011	Debian/Ubuntu
Un	Un	2.008	Debian/Ubuntu
Lu	Lu	2.010	Debian/Ubuntu
Mx	Mx	2.017	Debian/Ubuntu
Zo	Zo	2.009	Debian/Ubuntu
Bl	Bl	2.016	Debian/Ubuntu
Co	Co	2.001	Debian/Ubuntu
Re	Re	1.995	Debian/Ubuntu
Po	Po	2.011	Debian/Ubuntu
Ka	Ka	2.014	Debian/Ubuntu
Bp	Bp	2.003	Debian/Ubuntu
Qu	Qu	2.010	Debian/Ubuntu
Ra	Ra	2.017	Debian/Ubuntu
Ss	Ss	2.004	Debian/Ubuntu
Go	Go	2.002	Debian/Ubuntu
Ni	Ni	2.014	Debian/Ubuntu
Nu	Nu	2.010	Debian/Ubuntu
Ap	Ap	2.010	Debian/Ubuntu
An	An	2.009	Debian/Ubuntu
Ge	Ge	2.016	Debian/Ubuntu
So	So	2.016	Debian/Ubuntu
Ge	Ge	2.002	Debian/Ubuntu
Sa	Sa	2.005	Debian/Ubuntu
Pk	Pk	2.014	Debian/Ubuntu
Pu	Pu	2.006	Debian/Ubuntu
Fm	Fm	2.014	Debian/Ubuntu
Di	Di	2.002	Debian/Ubuntu
Ay	Ay	2.015	Debian/Ubuntu
Su	Su	1.998	Debian/Ubuntu
Le	Le	2.017	Debian/Ubuntu
Oe	Oe	2.012	Debian/Ubuntu
St	St	2.008	Debian/Ubuntu
Vd	Vd	2.015	Debian/Ubuntu
La	La	2.016	Debian/Ubuntu
Ov	Ov	2.014	Debian/Ubuntu
Tc	Tc	2.009	Debian/Ubuntu
Rc	Rc	2.017	Debian/Ubuntu
Sc	Sc	2.004	Debian/Ubuntu
Ca	Ca	2.009	Debian/Ubuntu

재설치 없는 배포판(커뮤니티 → 상용)

- 레드햇 센트 스트림 → 레드햇 엔터프라이즈(제한적으로)
- 우분투 → 우분투 프로페셔널
- 오픈수세 → 수세 엔터프라이즈
- 알마 리눅스 → 현재, 오픈 커뮤니티 배포판으로 많은 사용자가 이동 중
- 데비안 리눅스

정리

1. RHEL 경우에는 SRPM를 개인 혹은 내부적으로 사용하는 용도 이외에는, 배포 용도로 사용이 불가능하다.
2. 레드햇은 라이선스 정책이 다른 배포판보다 보수적인 편이다.
3. 데비안 계열(우분투 포함)은 다양한 저장소를 제공하기 때문에, 애플리케이션 확장이 편하지만, 라이선스 및 패키지 품질에 레드햇보다 느슨하다.

시스템 관리 명령어

시스템 및 모니터링

소개

여기서 다루는 명령어는 **중/고급 명령어**에 대해서 다룬다. 이러한 명령어 학습을 통해서 리눅스 기반으로 동작하는 서버의 성능 향상 혹은 장애 분석을 좀 더 쉽게 하도록 한다.

이 과정에서는 기본 명령어를 다루지 않기 때문에, 명령어에 대해서 잘 모르는 경우 **표준 리눅스 어드민 과정**을 참고 하도록 한다.

시스템 블록 명령어

앞으로 모든 리눅스는, **freedesktop**와 같은 프로젝트가 시작이 되면서 리눅스 구조는 이전과 많이 달라졌다. 하지만, 여전히 기본적인 명령어는 똑같이 사용하지만, 관리용으로 사용하는 명령어가 많이 변경이 되었다.

변경된 기본 명령어는 다음과 같다. 이 부분은 아래 시스템 블록에서 좀 더 다루도록 한다.

```
systemctl  
hostnamectl  
journalctl  
networkctl
```


상태 및 모니터링 명령어

시스템 운영하면서 보안 혹은 운영적인 측면에 알아두면 프로세스 행동 추적이 가능한 몇가지 명령어를 같이 소개한다. 여기에서 나열한 명령어는 일부분이다.

`lsof`

`sar`

`mpstat`

`nstat`

`ltrace/strace`

SAR 모니터링

CLI

sar

시스템 모니터링 시, **ps**, **top**와 같은 명령어를 많이 사용하지만, 자세하게 시스템의 내용을 모니터링을 원하는 경우 **sar**기반으로 확인을 권장한다.

sar는 **sysstat**에 포함이 되어 있으며, 이 패키지에는 다음과 같은 명령어들이 함께 포함이 되어 있다.

프로그램	설명
sar	시스템의 모든 부분에 대해서 모니터링 및 조회가 가능합니다.
sadc	"system activity data collector"의 약자. 이를 통해서 sar에서 사용한 데이터가 추출이 된다.
sa1/2	"sar1/2"는 이전에는 "crond"를 통해서 수집이 되었지만, 지금은 systemd의 "timer", "sleep", "service"를 통해서 수집 및 동작한다.
iostat	CPU 및 I/O 기록에 대해서 수집 및 기록한다.

sar

프로그램	설명
mpstat	CPU상태에 대해서 출력한다.
pidstat	특정 PID에 대해서 상태 정보를 출력한다.
nfsiostat	NFS 3/4에 대해서 성능 기록을 출력한다.
cifsioostat	Samba에 대해서 성능 기록을 출력한다.

sar

CPU 사용 상태를 확인 시, 다음과 같은 명령어로 수행합니다.

```
# sar -u 2 5
```

```
Linux 5.14.0-362.8.1.el9_3.x86_64 (node1.example.com) 03/31/24 _x86_64_  
(2 CPU)
```

22:41:07	CPU	%user	%nice	%system	%iowait	%steal	%idle
22:41:09	all	0.00	0.00	0.00	0.00	0.00	100.00
22:41:11	all	0.00	0.00	0.25	0.00	0.00	99.75
22:41:13	all	0.00	0.00	0.50	0.00	0.00	99.50
22:41:15	all	0.00	0.00	0.00	0.00	0.00	100.00
22:41:17	all	0.00	0.00	0.00	0.00	0.00	100.00
Average:	all	0.00	0.00	0.15	0.00	0.00	99.85

sar

메모리 사용 상태를 확인 시, 다음과 같은 명령어로 확인.

```
# sar -r 2 5
```

22:42:51	66428	117924	121756	28.23	12	171432
313796	22:42:53	66428	117924	121756	28.23	12
171432	313796					

sar

파일 시스템 사용 상태를 확인 시, 다음과 같은 명령어로 수행.

```
# sar -F 2 5
```

22:44:25	69546	2070	2.89	2.89	36658628	41532	0.11
22:44:25	592	7	1.17	1.17	0	0	0.00
22:44:25	53945	411	0.76	0.76	27863037	3	0.00

mpstat

CPU코어 혹은 소켓에서 사용 상태를 출력한다. 별도로 명령어를 명시하지 않고 실행하면, 실시간으로 사용 정보를 출력한다. `sysstat`명령어에 포함된 명령어.

```
# mpstat 1 5  
# mpstat -P 1 5
```


nstat

TCP스택 상태에 대해서 확인한다. 일반적으로 TCP 모니터링 시 사용한다.

```
# nstat -z | head -10
```

IpInReceives	43	3.2
IpInAddrErrors	0	0.0
IpInDelivers	38	2.9
IpOutRequests	27	2.0
IpOutNoRoutes	0	0.0
IcmpOutMsgs	0	0.0
IcmpOutRateLimitHost	0	0.0
IcmpOutDestUnreachs	0	0.0
IcmpMsgOutType3	0	0.0

ltrace

바이너리에서 사용하는 라이브러리를 추적한다. 올바르게 동작하지 않는 프로그램 디버깅 시 사용한다.

```
# ltrace hostname
rindex("hostname", '/') = nil
strcmp("hostname", "dnsdomainname") = 4
strcmp("hostname", "domainname") = 4
strcmp("hostname", "ypdomainname") = -17
strcmp("hostname", "nisdomainname") = -6
getopt_long(1, 0x7ffd7d610528, "aAdfbF:h?iIsVy", 0x55c5c04d6aa0, nil) = -1
```

strace

프로그램에서 사용하는 시스템 콜을 확인한다.

```
# strace hostname
execve("/usr/bin/hostname", ["hostname"], 0x7ffe7a5c63f0 /* 32 vars */) = 0
brk(NULL)                               = 0x55da657ba000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffda3a91f70) = -1 EINVAL (Invalid
argument)
access("/etc/ld.so.preload", R_OK)       = -1 ENOENT (No such file or
directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
uname({sysname="Linux", nodename="node1.example.com", ... }) = 0
newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ... },
AT_EMPTY_PATH) = 0
```

PCP/SYSTEM TAP 모니터링

CLI

도구 소개

전통 유틸리티 시대 (1990년대 ~ 2000년대 초반)

- top, vmstat, iostat, sar 같은 CLI 기반 유틸리티들이 주류였다.
- CPU, 메모리, 디스크, 네트워크 자원 사용률을 각각 별도로 모니터링하며, 운영자가 직접 조합해서 사용해야 했다.
- 단기적인 관찰에는 유용했지만, 장기간 데이터 수집이나 심층적인 분석에는 한계가 있었다.

SystemTap 시대 (2005년 ~)

- Red Hat 주도로 개발된 SystemTap은 리눅스 커널 내부와 사용자 공간 애플리케이션 사이에서 발생하는 이벤트를 추적할 수 있도록 했다.
- SystemTap 스크립트를 작성해 특정 시스템 콜, 커널 함수, I/O 동작을 실시간으로 탐지하고, 성능 병목이나 오류 원인을 깊게 분석이 가능.
- 문법이 C언어 기반으로 되어 있기 때문에 유지 보수 및 사용이 어려운 단점.

도구 소개

PCP(Performance Co-Pilot) 시대 (2010년대 ~)

- SystemTap처럼 커널 깊숙이 파고드는 대신, 운영자가 쉽게 활용할 수 있는 성능 수집/분석 프레임워크
- pmcd, pmlogger, pmchart 등을 통해 실시간/장기간 모니터링, 원격 수집, 다양한 메트릭 통합 관리
- 상대적으로 사용하기 쉽기 때문에 많은 사용자가 사용. 하지만, 여전히 관리 기능은 부족.

BPF/BCC/bpftrace 시대 (2017년 ~ 현재)

- eBPF(Extended Berkeley Packet Filter)의 발전으로, 리눅스 커널 내부 동작을 안정적이고 성능 저하 없이 추적 가능
- bcc-tools, bpftrace 같은 도구들은 SystemTap보다 가볍고 안전하며, 클라우드/컨테이너 환경에서 사용 가능
- 표준으로 각광받고 있지만, 보안 부분에 대해서 취약점이 발생 할 수 있음

SYSTEM TAP 소개

SystemTap은 리눅스 시스템에서 발생하는 커널 수준 이벤트와 사용자 공간 동작을 동적으로 추적하기 위해 개발된 강력한 분석 도구이다.

이 도구는 DTrace(Solaris의 동적 추적 프레임워크)에서 영감을 받아 만들어졌으며, 주로 Red Hat과 커뮤니티에서 유지해 왔다.

SystemTap은 관리자가 작성한 스텝 스크립트(.stp)를 바탕으로 동작한다. 이 스크립트는 커널 내부 함수, 시스템 콜, I/O 이벤트 같은 특정 지점을 프로브(probe)로 설정하고, 이벤트 발생 시 필요한 정보를 출력하도록 구성된다.

예를 들어, open() 시스템 콜이 호출될 때마다 파일 이름을 출력하거나, 특정 프로세스의 CPU 사용 패턴을 추적할 수 있다. 대다수 배포판에서 SYSTEM TAP를 지원 및 제공하고 있지만, 사용 방법 및 운영이 어렵기 때문에 점점 사용하지 않는 추세이다.

PCP 소개

PCP(Performance Co-Pilot)는 리눅스와 유닉스 계열 시스템의 성능을 모니터링하고 분석하기 위한 프레임워크이다. 이 도구는 시스템 성능 데이터를 수집하고, 다양한 방식으로 저장 및 시각화 및 원격에서도 접근 가능하다. 특히 대규모 서버 환경이나 클러스터 환경에서 성능 문제를 추적하는데 유용하다.

PCP

PCP는 크게 세 가지 요소로 구성된다.

pmcd (Performance Metrics Collector Daemon)

- 시스템에서 성능 지표를 수집하는 데몬
- 다양한 "PMDA(Performance Metrics Domain Agent)"를 통해 CPU, 메모리, 디스크, 네트워크 등 지표를 제공

pmda (Performance Metrics Domain Agents)

- 특정 서브시스템(CPU, 메모리, PostgreSQL, Apache 등)의 성능 지표를 수집하는 모듈
- 필요에 따라 추가로 설치 가능

클라이언트 도구

- pmstat, pminfo, pmchart, pmlogger 등을 사용하여 데이터 조회 및 분석

PCP 설치

PCP설치는 다음과 같다.

```
# dnf install pcp pcp-system-tools pcp-gui -y
# zypper install pcp pcp-system-tools pcp-gui
# apt update
# apt install pcp pcp-system-tools pcp-gui -y

# systemctl enable --now pmcd
# systemctl enable --now pmlogger
# systemctl enable --now pmie
```

PCP 명령어

시스템 전체 상태 요약.

```
# pmstat 1
```

PCP 명령어

사용 가능한 메트릭(유닛) 확인.

```
# pminfo
```

PCP 명령어

특정 메트릭 조회 (예: CPU idle 비율)

```
# pminfo kernel.all.cpu.idle  
# pmval -t 1 kernel.all.cpu.idle
```

PCP 명령어

로그 기록 (성능 데이터 수집).

```
# pmlogger -r -T 1h mylog
```

-T 1h : 1시간 후 자동 종료

-r : 이미 같은 이름의 아카이브가 있으면 이어서(resume) 기록 (없으면 새로 생성)

이 방식은 시스템 기본 pmlogger 구성(디폴트 템플릿)에 따라 많이 사용하는 유닛 위주로 수집. 혹은 아래와 같은 방식으로 정보 수집이 가능.

```
# mkdir -p ~/pcp-logs && cd ~/pcp-logs
```

```
# pmlogconf -c ./mylog.conf
```

```
# pmlogger -c ./mylog.conf -t 5s -T 2h ./mylog
```

SYSTEM TAP 설치

설치는 다음과 같이 한다.

```
# dnf install systemtap systemtap-runtime kernel-devel-$(uname -r) -y
```

SYSTEM TAP 명령어

CPU 스케줄러 이벤트 추적.

```
# stap -ve 'probe scheduler.cpu_on { println("CPU on:", cpu()) }'
```


SYSTEM TAP 명령어

open() 시스템콜 모니터링.

```
# stap -e 'probe syscall.open { printf("%s(%d) open(%s)\n", execname(),  
pid(), argstr) }'
```

SYSTEM TAP 명령어

디스크 I/O 모니터링.

```
# stap -e 'probe vfs.read { printf("%s read %d bytes\n", execname(),  
count) }'
```

비교

구분	SystemTap	PCP
목적	커널/시스템 콜 수준 추적 (Debug/Trace 중심)	시스템 성능 통합 모니터링
난이도	스크립트 작성 필요, 진입장벽 높음	CLI·GUI 툴 제공, 쉬운 접근
확장성	특정 분석 시나리오에 강함	장기간 수집·분석·원격 모니터링에 강함
활용	성능 병목 원인 분석, 커널 이벤트 추적	운영 환경 성능 모니터링, 대규모 서버 관리

lsof

특정 프로그램이 어떤 파일을 사용 중인지 확인한다. 보통 의심이 되는 프로그램에 대해서 해당 명령어로 확인한다.
또한, 링크가 깨진 바이너리에 대해서 검사 시 사용한다.

```
# lsof -u root | head -5
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
systemd	1	root	cwd	DIR	253,0	235	128	/
systemd	1	root	rtd	DIR	253,0	235	128	/

```
# lsof -i TCP:22 | head -5
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
sshd	746	root	3u	IPv4	19170	0t0	TCP	*:ssh (LISTEN)
sshd	746	root	4u	IPv6	19172	0t0	TCP	*:ssh (LISTEN)

lsof

이어서...

```
# lsof -nP -iTCP -sTCP:LISTEN
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
httpd	873	root	4u	IPv6	20773	0t0	TCP	*:80 (LISTEN)
sshd	874	root	3u	IPv4	20728	0t0	TCP	*:22 (LISTEN)
sshd	874	root	4u	IPv6	20737	0t0	TCP	*:22 (LISTEN)

```
# lsof -nP -i:80
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
httpd	873	root	4u	IPv6	20773	0t0	TCP	*:80 (LISTEN)
httpd	913	apache	4u	IPv6	20773	0t0	TCP	*:80 (LISTEN)
httpd	914	apache	4u	IPv6	20773	0t0	TCP	*:80 (LISTEN)
httpd	996	apache	4u	IPv6	20773	0t0	TCP	*:80 (LISTEN)

리눅스 커널/셸

새로운 커널 기능

커널

커널 아키텍처 및 기능

커널 아키텍트

리눅스 커널은 초기에 x86만 지원하였다. 현재 리눅스 커널은 대다수 CPU 아키텍처를 지원하고 있다. 모든 리눅스 배포판은 아래와 같은 아키텍처를 지원 및 제공하고 있다.

- Intel
- AMD
- IBM PowerPC
- Sparc
- IBM s390

리눅스 커널 기능 비교

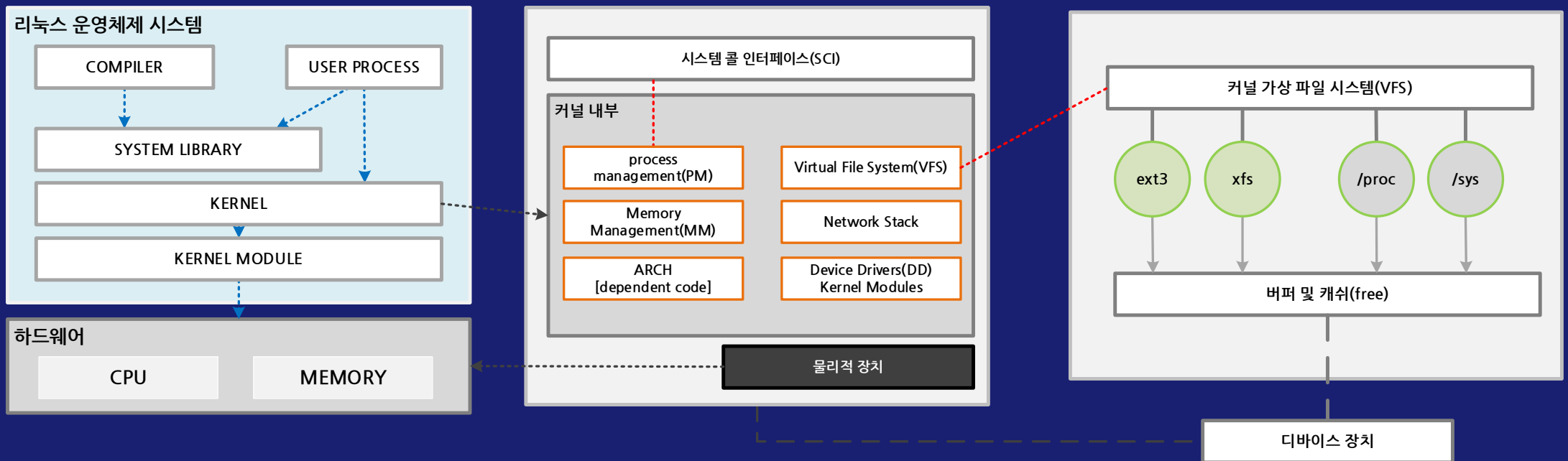
항목	RHEL	Ubuntu	SUSE	바닐라(업스트림)
전략	보수적 안정 + 장기지원. 한 메이저 수명 내에 커널 기본 버전 고정, 대규모 백포트	LTS(안정) + HWE(신규 HW 추격).	엔터프라이즈 시스템 성능 및 안정	최신 기능/드라이버를 빠르게 릴리스
kABI/ABI	강력한 kABI 안정 정책(심볼 stablelist 유지)로 DKMS/벤더 모듈 호환성 보장 (Red Hat Customer Portal)	ABI 관리를 명시(커널 변형/버전별 ABI 관리, 새 버전으로의 HWE) (Ubuntu)	kABI 안정 문서화 (드라이버 호환 초점) (drivers.suse.com)	공식 "stable ABI" 는 제한적 문서 수준 (주로 user-space 인터페이스) (Linux Kernel Documentation)

리눅스 커널 기능 비교

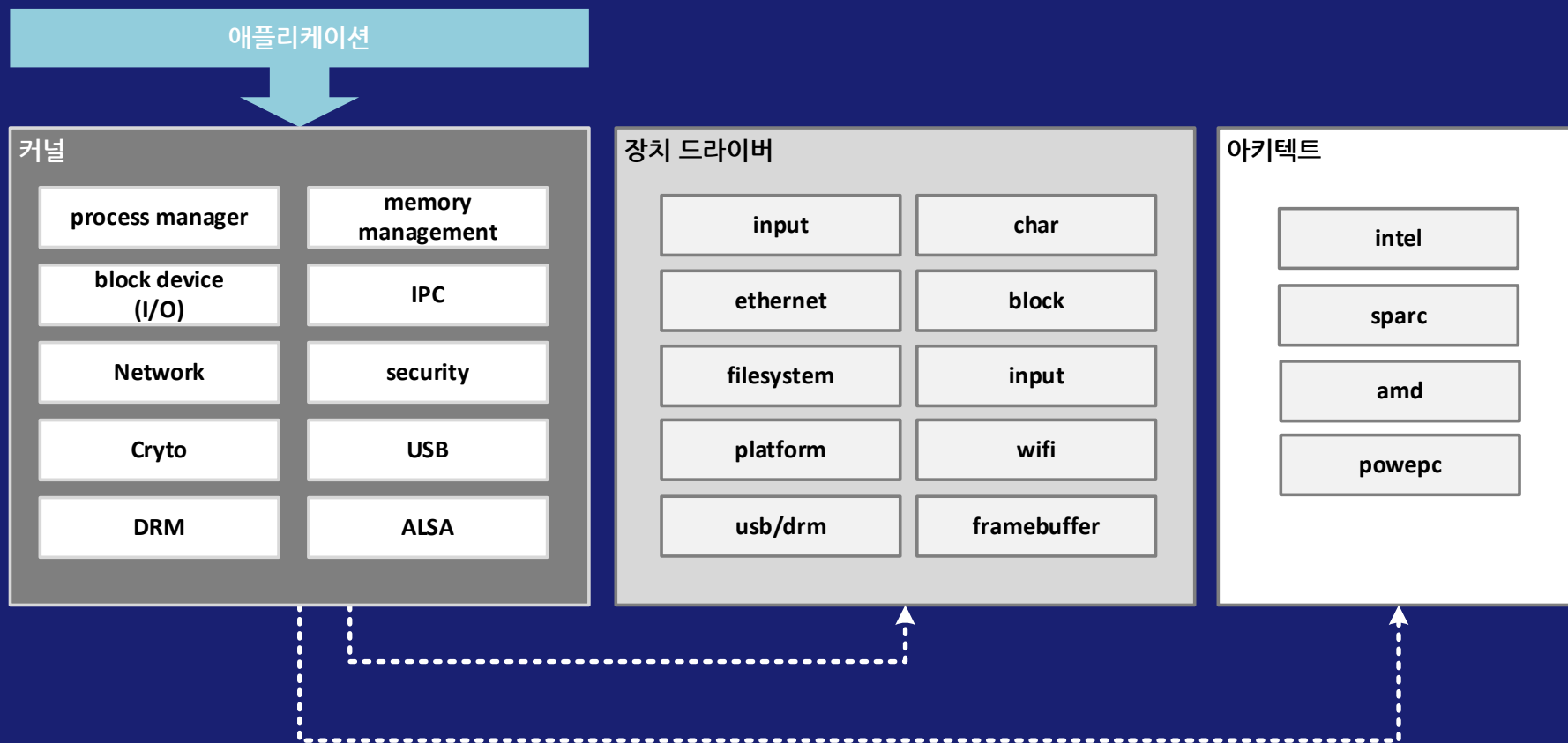
항목	RHEL	Ubuntu	SUSE	바닐라(업스트림)
라이브 패치	kpatch(라이브 커널 패치) 문서/서비스 제공 (레드햇)	Canonical Livepatch(우분투 서비스 측) - 배포/정책은 OS 서비스 측 문서 참고	SUSE Kernel Live Patching(KLP) 공식 문서/제품화 (SUSE Documentation)	기능 자체는 외부 프로젝트/배포판 통합에 의존
업데이트 정책	장기 수명 내 백포트 중심. 메이저 사이 이동은 느리지만 안정	보안 공지(USN)와 ABI 변경 시 모듈 재빌드 요구를 명확히 공지 (Ubuntu)	SLES 릴리스 노트 및 커널 버전 기록 공개, 다중 커널 병행 설치 가이드 (suse.com)	업스트림 릴리스 노트/체인지로그 중심

커널 아키텍트

리눅스 커널은 처음에는 모놀릭(Monolithic Kernel) 커널 구조 기반으로 작성되었다. 현재는 모듈기능 지원 및 커널 핫 패치(Hot-Patch)를 지원하고 있기 때문에, 모놀릭 커널의 단점을 많이 제거하였다.



커널 레이어



커널 버전 방식

리눅스 커널은 두 가지 형태로 현재 나누어져 있다. 커널 버전은 <https://www.kernel.org>에서 확인이 가능하다.

LT Release

LTS 버전은 Long Term Support 버전이다. 해당 버전의 소스코드는 5년에서 최장 10년 이상 지원하기도 하다.

이 커널은 오래된 하드웨어 모듈도 지속적으로 지원하며, 커널 버그 또한, 최소화 되어있기 때문에, 안정적인 시스템 운영에는 적절한 커널이다. 다만, 성능이나 새로운 기능 사용은 어려운 부분이 있다.

커널 버전 방식

Normal Release(stable)

일반 버전은 새로운 기능 추가 혹은 특정 기능이 패치가 되면, 보통 일반 커널 버전으로 릴리즈가 된다. 해당 버전은 x.y.z형태로 계속 y/z패치가 유지가 된다. LT와 다르게 새로운 기능도 추가가 되는 버전이다. 일반적으로 유지 기간은 대략 3년 정도 된다.

커널 버전 방식

Mainline Release

자주 업데이트 되는 커널 릴리즈 버전.

새로운 기능이 많이 추가가 되며, ML(Main Line)버전 기준으로 지속적으로 커널 기능 및 성능 개선이 된다. 보통 3개월 주기로 갱신된다.

커널 버전 방식

1. **w** = Kernel Version = 4
2. **xx** = Major Revision = 4
3. **y** = Minor Revision = 0
4. **zzz** = Patch number = 45

mainline:	6.4-rc7	2023-06-18	[tarball]	[patch]	[inc. patch]	[view diff]	[browse]
stable:	6.3.9	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	6.1.35	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	5.15.118	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	5.10.185	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	5.4.248	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	4.19.287	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
longterm:	4.14.319	2023-06-21	[tarball]	[pgp]	[patch]	[inc. patch]	[view diff] [browse] [changelog]
linux-next:	next-20230623	2023-06-23					[browse]

커널 버전 방식

대다수 리눅스 배포판은 LTS 버전을 채용 혹은 선택하는데, 이것도 각각 기업마다 입장이 조금씩 다르다.

레드햇 경우에 LTS 커널을 사용하지 않으며 자체 LTS 버전을 구성 및 운영한다. 수세 리눅스 경우에는 자체 LTS 및 바닐라 버전 커널을 같이 제공하고 있다.

리누즈 토발즈는 리눅스 커널 릴리즈 방식을 변경 하였는데, 이유는 다음과 같다.

"내 마음에 안 들어서"

커널 버전 방식

좀 더 공식적으로 풀어서 이야기 하면, 다음과 같이 발표 하였다.

"since we no longer do version numbers based on features, but based on time, just saying 'we're about to start the third decade'"

아래 링크에서 넘버링 변경에 대한 이유를 확인 할 수 있다.

[왜 리눅스 토발즈는 넘버링을 변경했는가?](#)

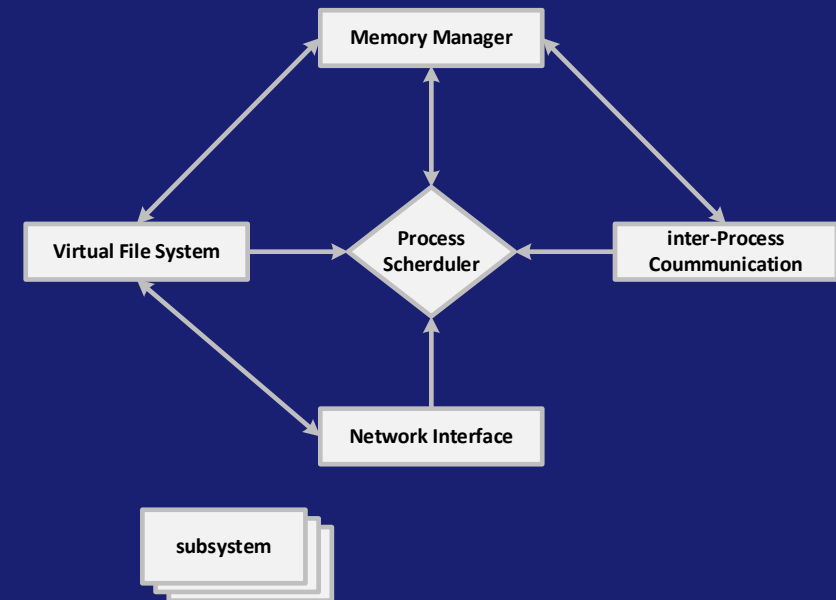
커널 버전 방식

- Linux pre-1.0 versions ran from 1991 to 1994: Three years.
- Linux 1.x ran from 1994 through mid-1996: Two and half years.
- Linux 2.0.x was the mainline stable kernel from mid-1996 through early 1999: Less than three years.
- Linux 2.2.x was the mainline stable kernel from early 1999 through early 2001: About two years.
- Linux 2.4.x was the mainline stable kernel from early 2001 through late 2003: About three years.
- Linux 2.6.x is the mainline stable kernel since mid-December 2003: More than seven years.

커널 아키텍트

리눅스(커널)는 모놀리식(Monolithic Kernel) 커널 구조이다. 마이크로 커널 혹은 하이브리드 커널과 달리, 서브시스템 기반으로 각 자원을 관리 및 사용한다. 아래 그림은 모놀리식 커널의 구조이며, 핵심 커널 서비스 이외 나머지 영역은 하위 영역으로 별도로 구성이 된다.

<https://docs.kernel.org/subsystem-apis.html>



커널 아키텍트

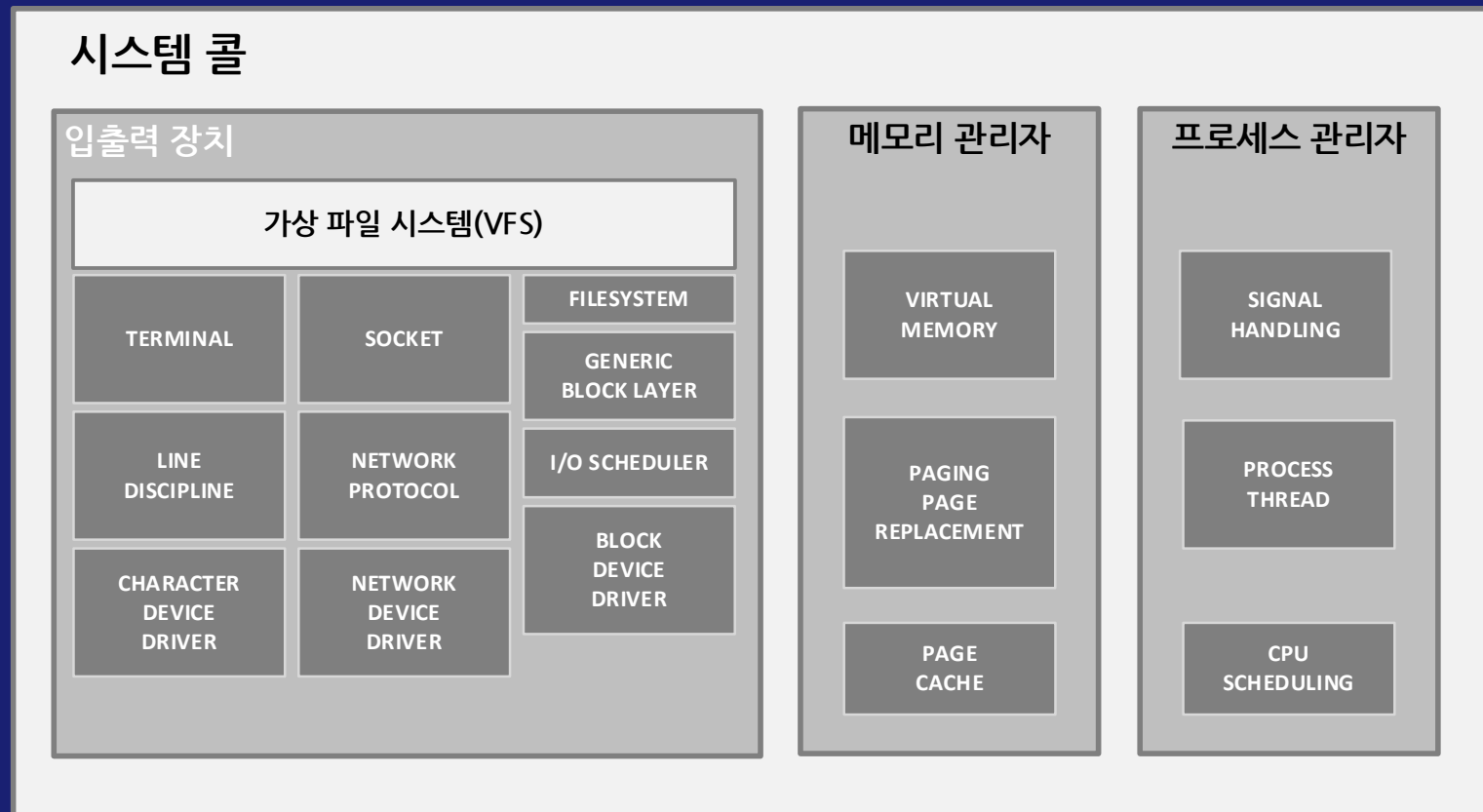
리눅스 시스템 콜은 다음과 같다.

- I/O
- Memory
- Process

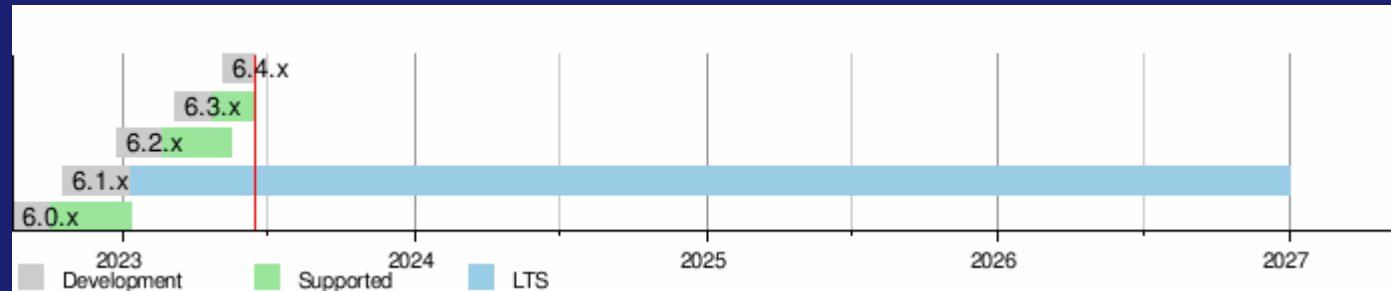
각 서브 시스템은 스케줄러가 있으며, 커널은 스케줄러를 통해서 작업을 할당 및 처리한다. API 관련 문서는 아래 링크에서 확인이 가능하다.

<https://linux-kernel-labs.github.io/refs/heads/master/lectures/syscalls.html>

커널 아키텍트



커널 버전 6.x



커널 버전 5.x



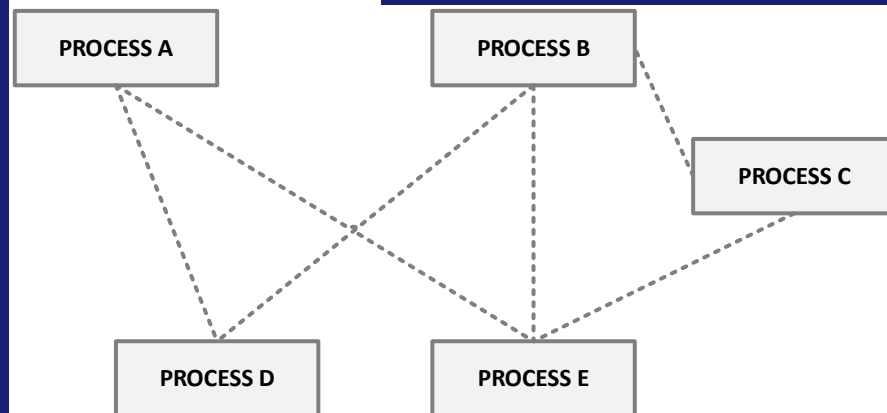
dbus

윈도우 시스템 계열은 시스템 컴포넌트는 **com/dcom** 기반으로 구성되어 있다.

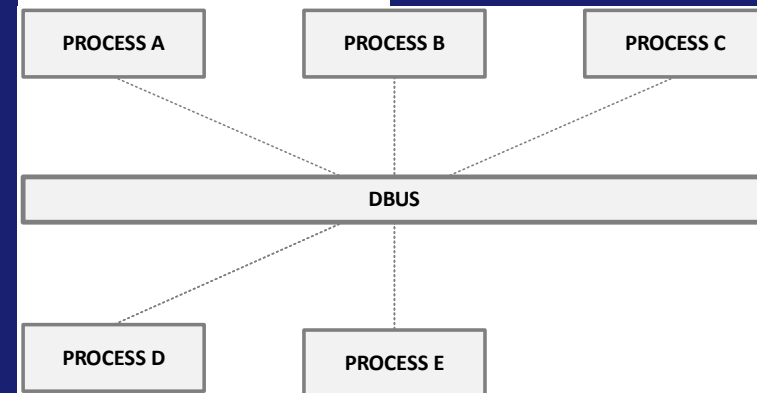
현재 리눅스도 현대적으로 구조가 변경이 되면서 **dbus** 서비스 중심으로 컴포넌트를 통합하고 있다. 시스템 블록에서 특정 정보가 변경이 되면, 다른 컴포넌트들도 해당 정보를 참조하여 시스템 운영에 반영한다.

대표적으로 윈도우 서버는 네트워크 정보를 **Windows Service Bus, (WSB)**를 통해서 다른 서비스와 공유한다.

이전 리눅스 방식



최근 리눅스 방식



dbus

dbus 서비스는 다음과 같은 명령어로 조회가 가능하다.

실제로 시스템 운영시에는 많이 사용하지 않는 명령어이다. 다만, systemd 기반으로 통합이 되고 있기 때문에, 앞으로 dbus 에서 문제가 발생하는 경우, 서비스가 올바르게 동작이 되지 않을 수 있다.

```
# systemctl status dbus
```

```
# gdbus introspect --system --dest org.freedesktop.systemd1 --object-path /org
```

컨테이너 및 가상화

커널 2.4 이후 컨테이너 및 가상화 기능을 지원하고 있다.

본래 리눅스의 가상화 및 컨테이너 프로젝트는 Linux vServer에서 시작하였다.

컨테이너 기능은 구글에서 제공한 cgroup, 그리고 레드햇에서 제공한 namespace를 통해서 최종적으로 구현이 되었으며, 이를 기반으로 루트리스 컨테이너(rootless container) 런타임인 docker가 탄생하게 되었다.

현재 표준 컨테이너 런타임은 Docker에서 Podman으로 전환이 되고 있다.

리눅스 모듈

초기 리눅스는 소스코드에 빌트인 형태로 드라이버, 즉, 커널 모듈을 배포하였다. 이 방법에 문제는 새로 추가하거나 혹은 업데이트 시, 매번 커널을 다시 컴파일을 해야 하는 문제가 있었다.

초창기 커널은 이러한 기능이 없었고, **리눅스 커널 버전 2.0** 이후로는 모든 커널에서 모듈 기능을 제공한다. 초창기 리눅스는 x86만 고려하여 제작이 되었기 때문에, 모듈 및 CPU를 확장 지원을 할 계획이 없었다. 이전 1.0 커널은 모듈 기능이 거의 없었다.

리눅스 커널에 포함이 되어 있는 모듈은 대다수가 **GNU GPLv2/3기반** 강제하고 있다.

BSD 및 **MIT 라이선스 모듈**도 종종 있지만, 해당 모듈은 **mainline module**로 포함이 되지 않는다.

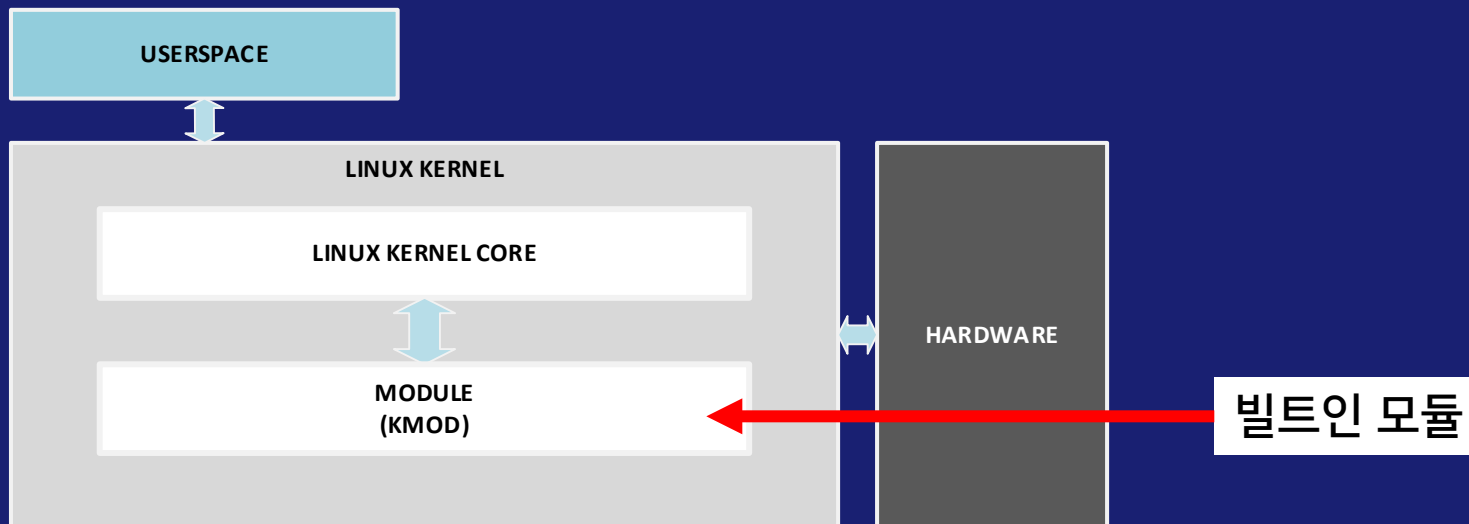
대표적인 모듈이 바로 **Nvidia** 모듈이다. 우분투 배포판을 제외하고, 대다수 GNU 배포판은 엔비디아 드라이버를 저장소에서 제공하지 않는다. 심지어 **라이브러리 링크(ldconfig)**도 제공하지 않는다. 유일하게 우분투 배포판만 바이너리 드라이버 및 라이브러리를 제공하고 있다.

리눅스 모듈(kmod)

일반적인 리눅스 커널 모듈은 전부 **kmod** 모듈 기반으로 구성이 되어 있다. **kmod**는 일반적인 리눅스 커널 모듈이다. `modprobe`, `insmod`같은 명령어를 통해서 메모리에 커널을 상주하여 하드웨어를 초기화한다.

장점: 커널과 호환성 및 안정성이 매우 높다. 빌트인 혹은 모듈 형태로 활성화 된 모듈은 바로 사용이 가능하다.

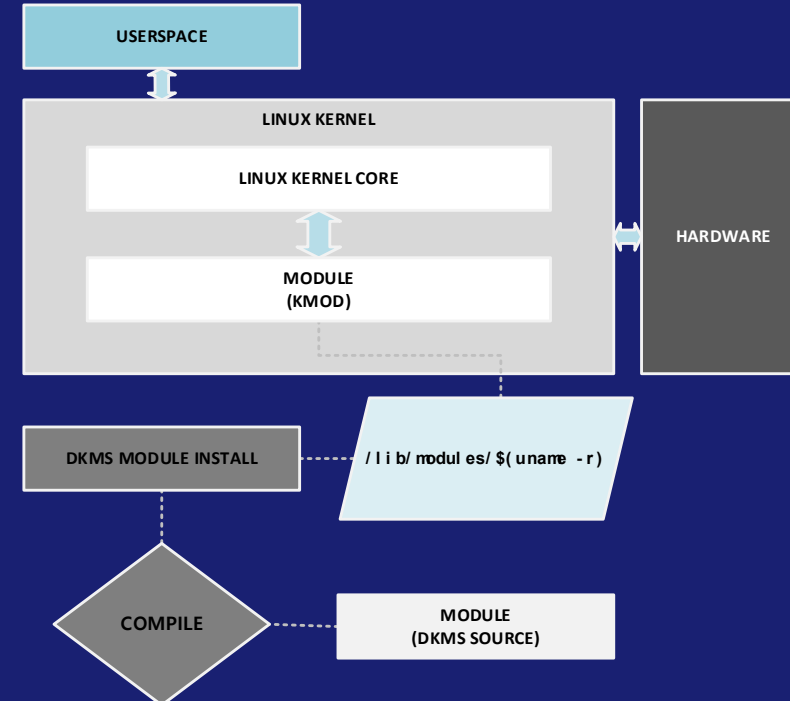
단점: 다른 커널 버전에서 동작이 불가능하다.



리눅스 모듈(dkms)

커널 컴파일을 별도로 요구하지 않는 부분이 장점이며, 모듈만 컴파일 혹은 바이너리 설치하여 바로 모듈로 사용이 가능하다. 보통 **dkms**, **akmod**라는 이름으로 배포판에서 제공한다. 다만, 사용하기 위해서 라이브러리 링크 컴파일이 종종 필요하기 때문에 컴파일러 도구가 필요하다.

장점	kmod의 보다 사용하기 편하다
단점	호환성 문제가 발생할 수 있다.



커널모듈 정리

커널 모듈 기능을 정리하면 다음과 같다.

이름	설명
kmod	빌트인 커널. 다른 커널 버전에서 사용이 불가능하다.
akmod	빌트인 커널과 비슷하지만, 이 모듈은 커널 업데이트를 신경 쓰지 않고 사용이 가능하다.
dkms	외부에서 별도로 제작해서 제공하는 커널. 보통 Nvidia 커널이 dkms 형태로 제공이 되며, 이 형태로 되어 있는 모듈은 커널과 링크 형태로 제공이 된다.

AI 시대의 커널, 그리고 모듈

리눅스 커널은 다음과 같은 이유로 Nvidia 커널 모듈을 기본값으로 반영하지 않는다.

- 커널은 범용성·안정성·검증을 최우선으로 설계됨
- ABI 고정 불가/내부 구조 변경 가능
- 특정 하드웨어·워크로드에 종속되지 않는 것이 핵심 가치
- AI/GPU/가속기는 커널의 목적이 아님
- 커널은 "플랫폼" 입장으로 동작, AI는 "워크로드" 중 하나

AI 시대의 커널, 그리고 모듈

1. 커널: LTS / 안정 버전 고정
2. GPU 드라이버: 외부 모듈 (DKMS)
3. AI 스택:
 - CUDA/ROCm/oneAPI
 - 컨테이너 & 사용자 공간 중심

엔비디아 리눅스 모듈과 리눅스 커널

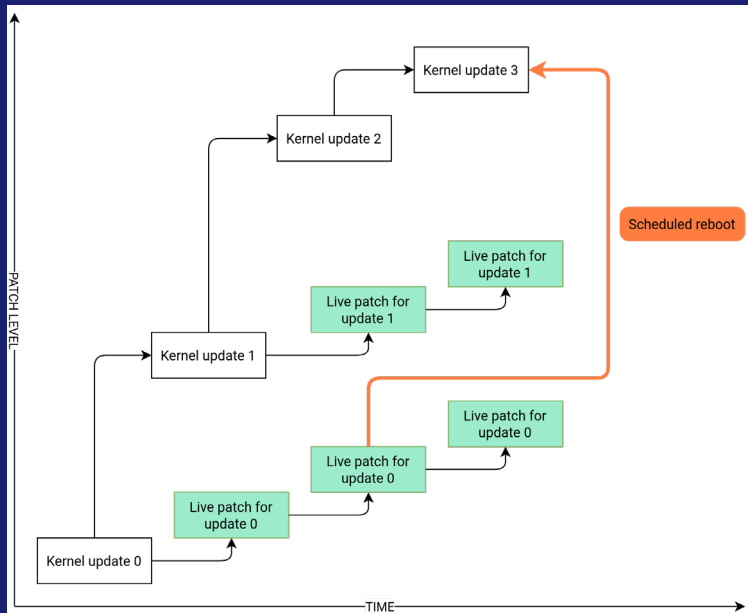


2026-01-02

커널 라이브 패치

이전 리눅스 커널은 유닉스 커널처럼 라이브 패치 기능이 없었다.

현재, 라이브 패치 기능이 추가가 되었으며, 공식적인 이름은 **kpatch**이다. 현재 최신 리눅스 커널은 이 기능 사용이 가능하다. 사용하기 위해서는 다음과 같은 기능이 활성화가 되어 있어야 한다.



커널 라이브 패치

각 밴더별로 다음과 같은 이름으로 제품 및 기능을 제공하고 있다.

구분	kpatch	kGraft	Livepatch
진영	RHEL 계열	SUSE 계열	Ubuntu
제공 방식	서브스크립션 기반	서브스크립션 기반	서브스크립션 기반
커뮤니티 사용	직접 빌드 후 사용 가능	불가능	불가능
패치 생성	Red Hat (유료) 사용자 (커뮤니티)	SUSE	Canonical
적용 방식	커널 모듈 (라이브 로딩)	커널 라이브 패치	커널 라이브 패치
현실적 난이도	직접 생성 시 높음	낮음	낮음
운영 적합성	엔터프라이즈		

커널 라이브 패치 예제(kpatch)

커널 라이브 패치를 하기 위해서 다음처럼 간단하게 구성이 가능하다. 다만, 버전에 따라서 동작이 안될 수 있으니, 이점 주의한다. 아래는 간단하게 레드햇 계열에서 사용하는 kpatch를 구현한다.

```
# dnf update kernel
# reboot

# dnf install kexec-tools
# kexec -l /boot/vmlinuz-3.10.0-862.3.2.el7.x86_64 \
--initrd=/boot/initramfs-3.10.0-862.3.2.el7.x86_64.img --reuse-cmdline
# sync; sudo umount -a; sudo kexec -e
```

kpatch 구현

아래와 같은 단계로 간단하게 kpatch 구현 및 테스트 한다.

```
# uname -r
# sudo dnf install -y git make gcc bc elfutils-libelf-devel \
kernel-devel-$(uname -r) kernel-debuginfo-$(uname -r) \
kernel-debuginfo-common-$(uname -r)
# git clone https://github.com/dynup/kpatch.git
# cd kpatch
# make
# make install
# mkdir ~/kpatch-democd ~/kpatch-demo
```

kpatch 구현

아래와가상으로 커널 패치를 구성 및 생성한다.

```
# cat > demo.patch << 'EOF'
diff --git a/kernel/sys.c b/kernel/sys.c
index 4d9f8b7..deadbee 100644
--- a/kernel/sys.c
+++ b/kernel/sys.c
@@ -2503,6 +2503,7 @@ SYSCALL_DEFINE1(sethostname, char __user *, name)
         if (copy_from_user(tmp, name, len))
             return -EFAULT;
+        pr_info("kpatch demo: sethostname called!\n");
+        strncpy(hostname, tmp, len);
+        return 0;
     }
EOF
```

kpatch 구현

아래와가상으로 커널 패치를 구성 및 생성한다.

```
# kpatch-build demo.patch
# ls
> kpatch-demo.ko
# dmesg -w
# hostname test-before
# insmod kpatch-demo.ko
# lsmod | grep kpatch
# hostname test-after
# dmesg -w
```


Nvidia와 kpatch의 관계

Nvidia에서 모든 드라이버 영역을 공개하지 않았기 때문에, Nvidia 모듈 및 커널 관련 패치는 대다수 커널 콜이 비공개로 되어 있음. 이러한 이유로 kpatch를 통한 Nvidia 관련된 드라이버 혹은 커널 업데이트는 수동으로만 적용이 가능하다.

최근 리눅스 구조

API

ABI

POSIX

현대화된 리눅스 구조

최근에 나온 리눅스 아래와 같은 도구로 통일 및 통합이 되고 있다. 여기서 다루는 부분은 커널의 API/ABI/POSIX 영역이다.

1. nftables/firewalld
2. NetworkManager/NetPlan/systemd-networkd
3. dbus/systemd
4. API/ABI/KABI
5. Linux POSIX

API/ABI

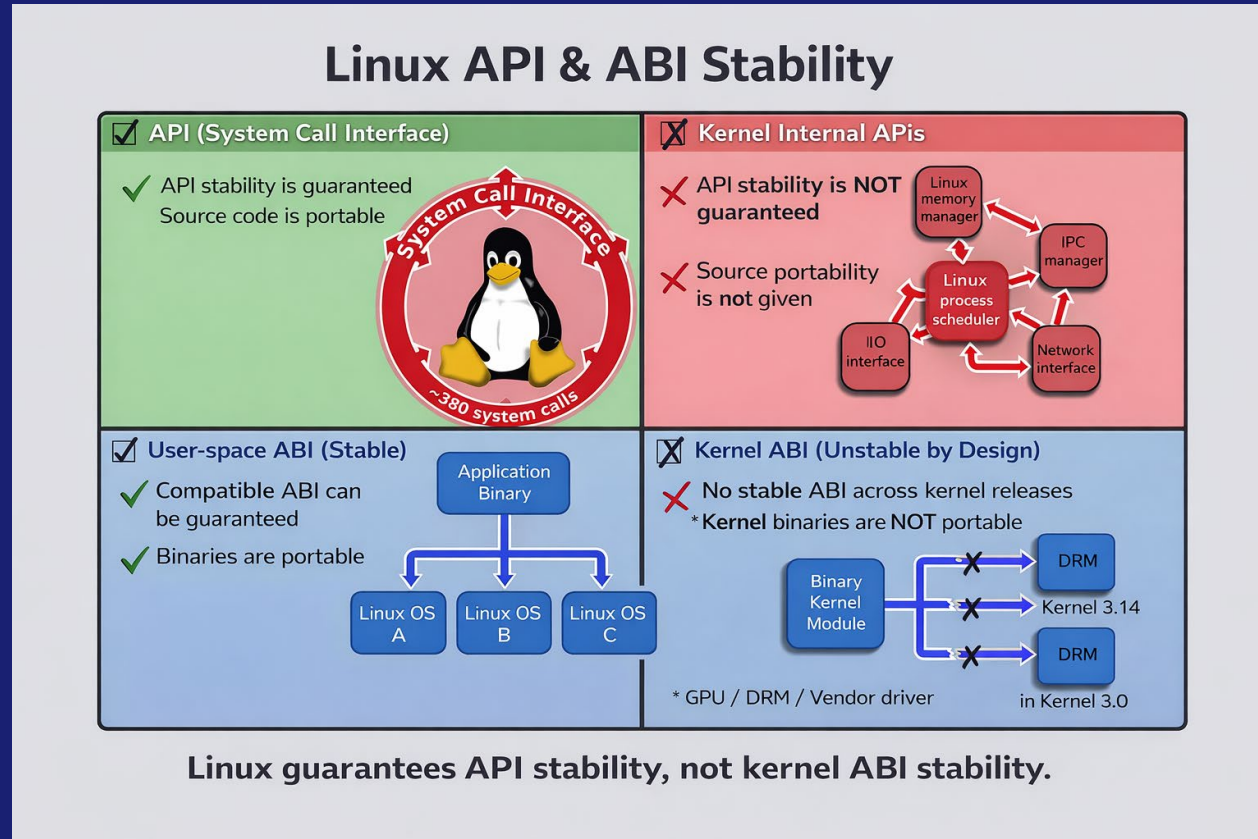
Application Programming Interface(API)

리눅스 커널 혹은 애플리케이션에서 사용이 가능하다. 하지만, 리눅스 커널 기반의 API는 서로간 호환성을 보장하지 않는다. 여기서 말하는 호환성은 커널간의 호환성을 이야기 한다.

Application Binary Interface (ABI)

일반 사용자들이나 혹은 최종 개발자들은 ABI에 관심이 더 많다. 애플리케이션은 같은 LSB 버전을 사용하는 경우 호환이 된다. 하지만, 커널은 같은 버전이 아닌 경우, 호환성이 거의 보장되지 않는다.

API/ABI 안정성 및 호환성



KERNEL API

1. 사용자 공간(User-space)에서의 커널 API = 시스템 콜(System call)

- `open()`, `read()`, `write()`, `fork()`, `execve()`, `uname()` 같은 호출
- 유저 프로그램이 커널 기능을 쓰는 공식 출입문
- 리눅스는 이 시스템 콜 인터페이스(= 커널의 외부 API)를 비교적 안정적으로 유지하려고 함 (그래서 소스코드 이식성이 높아짐)

2. 2) 커널 내부(Kernel internal) API = 커널 코드끼리 쓰는 함수/구조체

- 예: VFS, 메모리, 스케줄러, 네트워크, DRM 같은 서브시스템 내부 함수들
- 이건 "외부 고객"을 위한 API가 아니라서 수시로 바뀔
- 그래서 커널 모듈(특히 바이너리-only 모듈)은 커널 버전 바뀌면 깨지기 쉬움 (이게 "커널 ABI 안정성 없음" 얘기의 핵심)

KERNEL API 예제

강사의 가이드에 따라서 소스코드 다운로드 후 아래와 같이 진행한다. 코드는 C, GO 언어 기반으로 제공한다. 커널의 devel 및 header를 사용하여 커널 함수를 호출한다.

```
# curl -o kernel_api_test.c http://10.0.0.250/materials/kernel_api_test.c
# gcc -O2 -Wall -o kernel_api_test kernel_api_test.c
# ./kernel_api_test

# curl -o kernel_api_test.c http://10.0.0.250/materials/kernel_api_test.go
# go run kernel_api_test.go#
# go build -o kernel_api_test_go kernel_api_test.go
# ./kernel_api_test_go
```

KERNEL API 모듈 예제

마지막으로 커널 모듈 기반으로 API를 호출 및 접근한다. 커널 모듈은 GO 언어로 생성 및 접근은 가능하나, 아직 공식적인 모듈로 인증이 안된다.

```
# curl -o Makefile http://10.0.0.250/materials/hello_kmod_Makefile
# curl -o hello_kmod.c http://10.0.0.250/materials/hello_kmod.c
# make
# insmod hello_kmod.kodmesg | tail -n 20
# rmmod hello_kmoddmesg | tail -n 20
```


커널 ABI 설명

ABI(Application Binary Interface)는 "바이너리(컴파일된 프로그램)가 다른 구성요소와 맞물리는 규칙"이야. API가 소스 코드 레벨의 약속이라면, ABI는 *바이너리 레벨의 약속*.

ABI에 포함되는 것들.

1. 함수 호출 규약(Calling convention): 인자/리턴값을 레지스터/스택에 어떻게 놓는가
2. 데이터 타입/구조체 레이아웃: struct의 필드 순서, 정렬(alignment), 패딩
3. 심볼/링킹 규칙: 공유 라이브러리 심볼 이름, 버전
4. 시스템 콜 인터페이스(커널 ABI 성격)
5. 동적 링커 규칙: ELF, ld-linux, rpath 등

ABI가 깨지면 어떻게 되냐?

컴파일은 됐는데 실행시 죽음

- undefined symbol: ...
- version 'GLIBC_2.xx' not found
- 더 무서운 케이스: 조용히 틀린 값 (struct layout 변화)

ABI 랩

ABI를 구현하기 위해서 다음과 같이 구성한다. 버전 1/2를 만들어서 glibc를 통해서 ABI 생성 및 연결한다.

```
# mkdir -p abi-lab/{v1,v2,app}  
# cd abi-lab
```

ABI 랩

ABI를 구현하기 위해서 다음과 같이 구성한다. 버전 1/2를 만들어서 glibc를 통해서 ABI 생성 및 연결한다. v1은 정상적으로 동작하는 버전이다.

```
# vi v1/abi.h
#pragma once
#include <stdint.h>

typedef struct {
    uint32_t a;
    uint32_t b;
} Packet;

uint32_t sum(Packet p);
```

ABI 랩

ABI 헤더를 통해서 공유 라이브러리 구성을 한다.

```
# vi v1/libabi.c
#include "abi.h"
```

```
uint32_t sum(Packet p) {
    return p.a + p.b;
}
```

```
# gcc -fPIC -shared -o v1/libabi.so v1/libabi.c
```

ABI 랩

ABI 헤더를 통해서 공유 라이브러리 구성을 한다.

```
# vi app/main.c
#include <stdio.h>
#include "../v1/abi.h"

int main() {
    Packet p = { .a = 10, .b = 20 };
    printf("a=%u b=%u sum=%u\n", p.a, p.b, sum(p));
    return 0;
}

# gcc -o app/app app/main.c -L../v1 -labi -Wl,-rpath,'$ORIGIN/../v1'
# ./app/app
> a=10 b=20 sum=30
```

ABI 깨트리기 랩

이번에는 장애를 강제로 발생 시킨다.

```
# vi v2/abi.h
#pragma once
#include <stdint.h>

typedef struct {
    uint32_t a;
    uint32_t x;    // ← 중간에 새 필드 추가 (ABI 파괴)
    uint32_t b;
} Packet;

uint32_t sum(Packet p);
```

ABI 깨트리기 랩

ABI 헤더를 통해서 공유 라이브러리 구성을 한다.

```
# vi v2/libabi.c
#include "abi.h"

uint32_t sum(Packet p) {
    return p.a + p.b;
}

# gcc -fPIC -shared -o v2/libabi.so v2/libabi.c
# LD_LIBRARY_PATH=./v2 ./app/app
```


메모리 크기 검사

최종적으로 메모리 검사를 하여, 커널이 호환이 되는지 확인한다.

```
# cat > app/sizeof.c << 'EOF'
#include <stdio.h>
#include "../v1/abi.h"
int main(){ printf("sizeof(Packet)=%zu\n", sizeof(Packet)); }
EOF
# gcc -o app/sizeof app/sizeof.c
# ./app/sizeof
```

메모리 크기 검사

최종적으로 메모리 검사를 하여, 커널이 호환이 되는지 확인한다.

```
# cat > app/sizeof2.c << 'EOF'
#include <stdio.h>
#include "../v2/abi.h"
int main(){ printf("sizeof(Packet)=%zu\n", sizeof(Packet)); }
EOF
# gcc -o app/sizeof2 app/sizeof2.c
# ./app/sizeof2
```

정리

현재 레드햇 호환 리눅스는 ABI/kABI, API 위주로 호환성을 맞추고 있다. 이러한 이유로 리눅스 시스템 엔지니어는 다른 배포판 혹은 RHEL에서 호환 배포판으로 마이그레이션 시, 적절하게 glibc와 호환성을 고려해야 한다.

정리

아래와 같은 이유로 라이브러리 링크 및 버전 문제를 확인 후 적절하게 수정하여 사용이 가능하다.

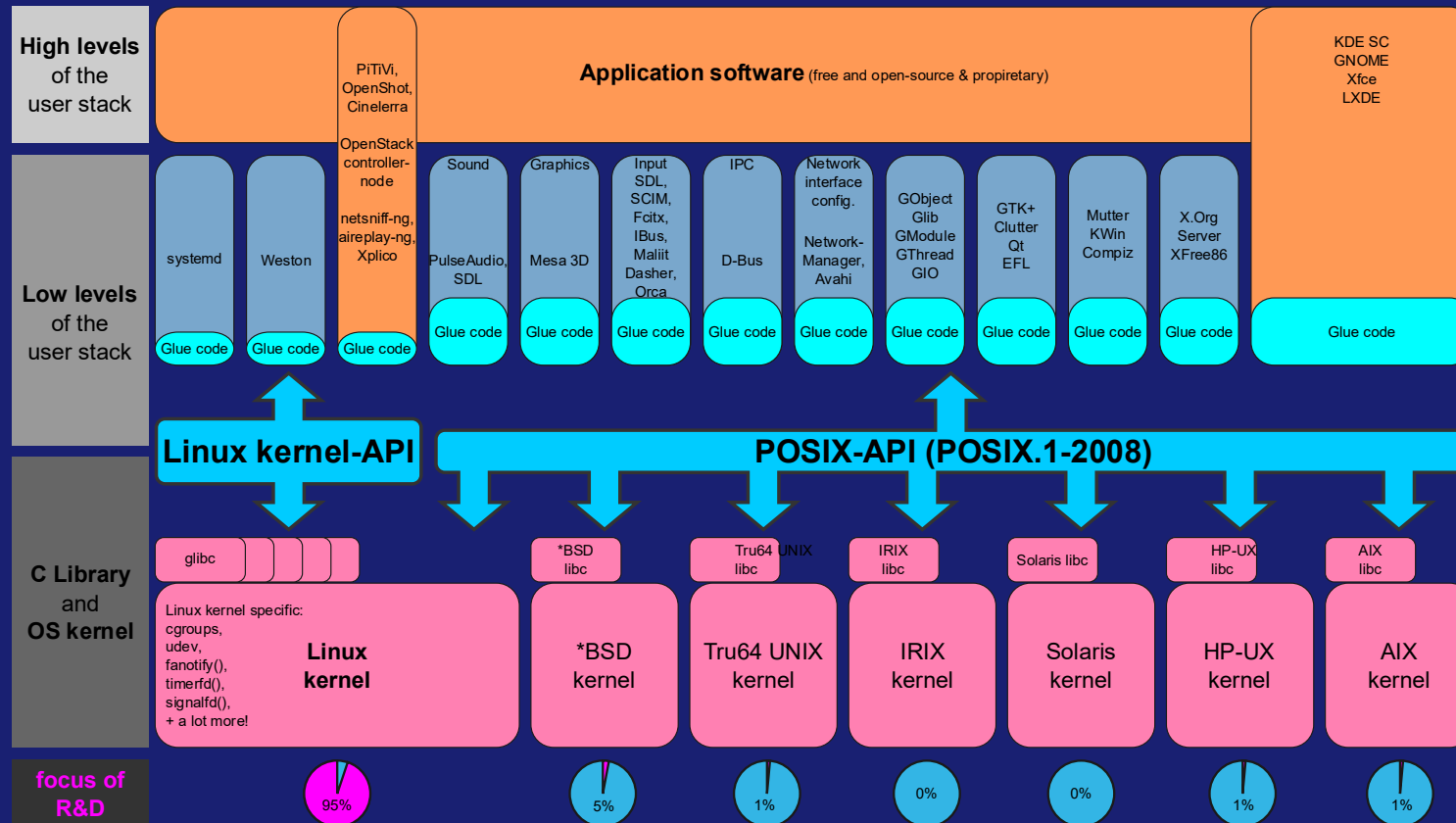
```
# ldd --version | head -n1  
# objdump -T /bin/ls | grep GLIBC_ | tail -n 5
```

정리

POSIX는 각기 다른 유닉스 커널의 시스템 콜의 호환성을 위해 만들어졌다.

리눅스도 POSIX를 지원을 하며, 각기 유닉스에서 사용하던 POSIX를 거의 지원한다. **Portable Operating System Interface**의 약자이다.

정리



systemd

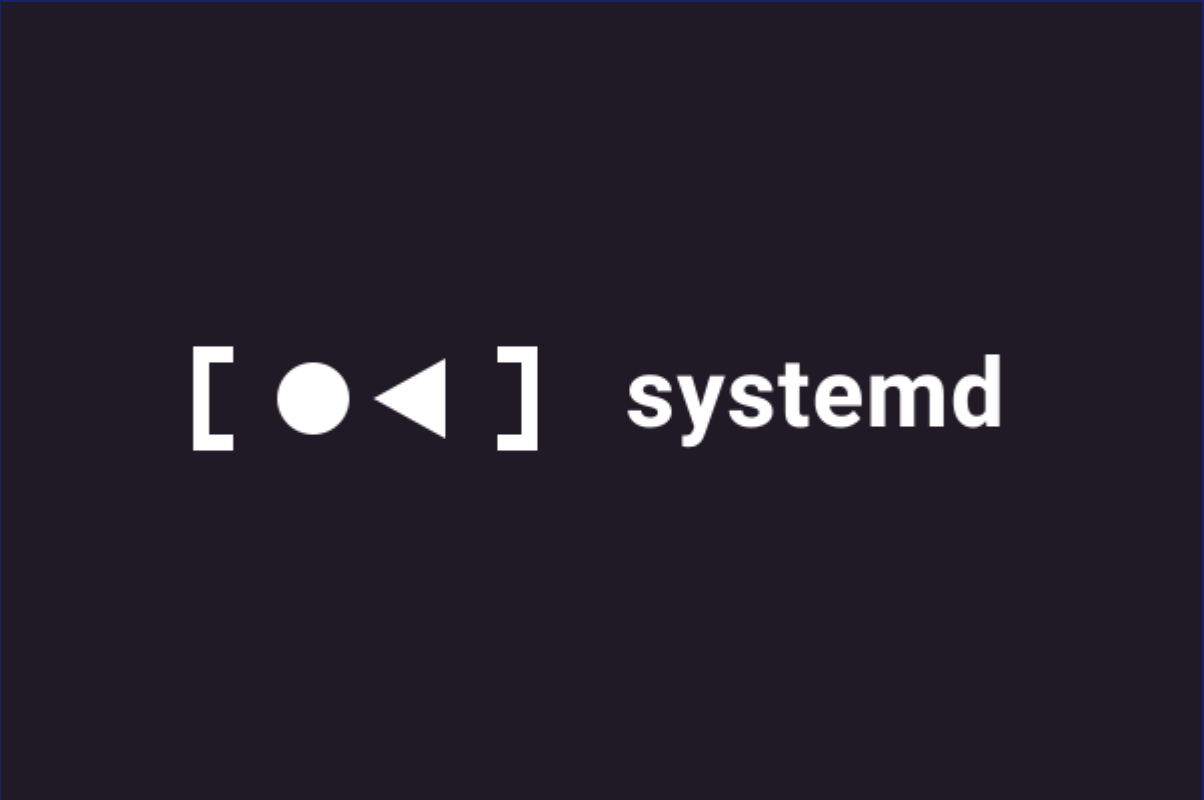
시스템 설명 및 통합 및 변경된 부분

간단한 서비스 및 명령어 소개

소개 및 설명

systemd

systemd



[● ◀] systemd

systemd

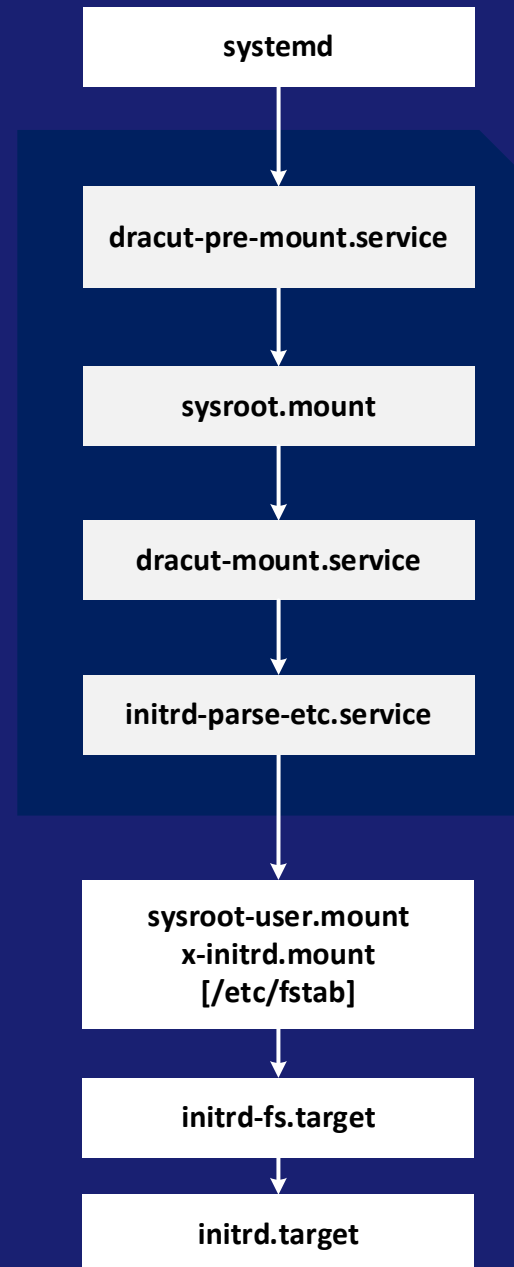
systemd는 다음과 같은 구조로 부팅이 된다. target은 일종의 서비스 묶음, 그리고 이전의 런 레벨과 비슷한 역할을 한다. 옆에 그림은 간단하게 systemd에서 부팅 단계를 다이어그램으로 표시하였다.

systemd는 레드햇의 업 스트리밍 버전인 **페도라 20버전**에서 실험적으로 도입이 되었으며, 안정화 및 기능 정립이 완료가 된 후, 공식적으로 RHEL버전에 도입 되었다.

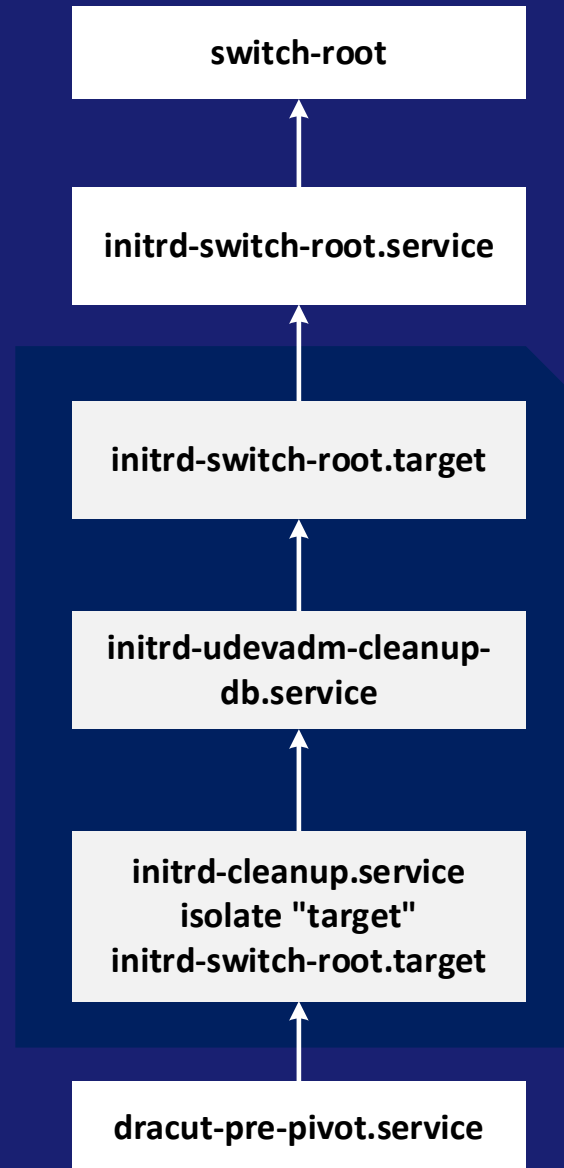
기존 System V와 많이 다른 부분은 cgroup, D-Bus기능이 시스템 systemd의 시스템 블록으로 통합 되었다. 운영체제 업데이트 부분은 **systemd**의 **initrd(ramdisk)**에서 실행한다.

현재 LSB사양을 따르는 대다수 배포판은 systemd기반으로 init의 PID 1번으로 사용하고 있다.

systemd-1

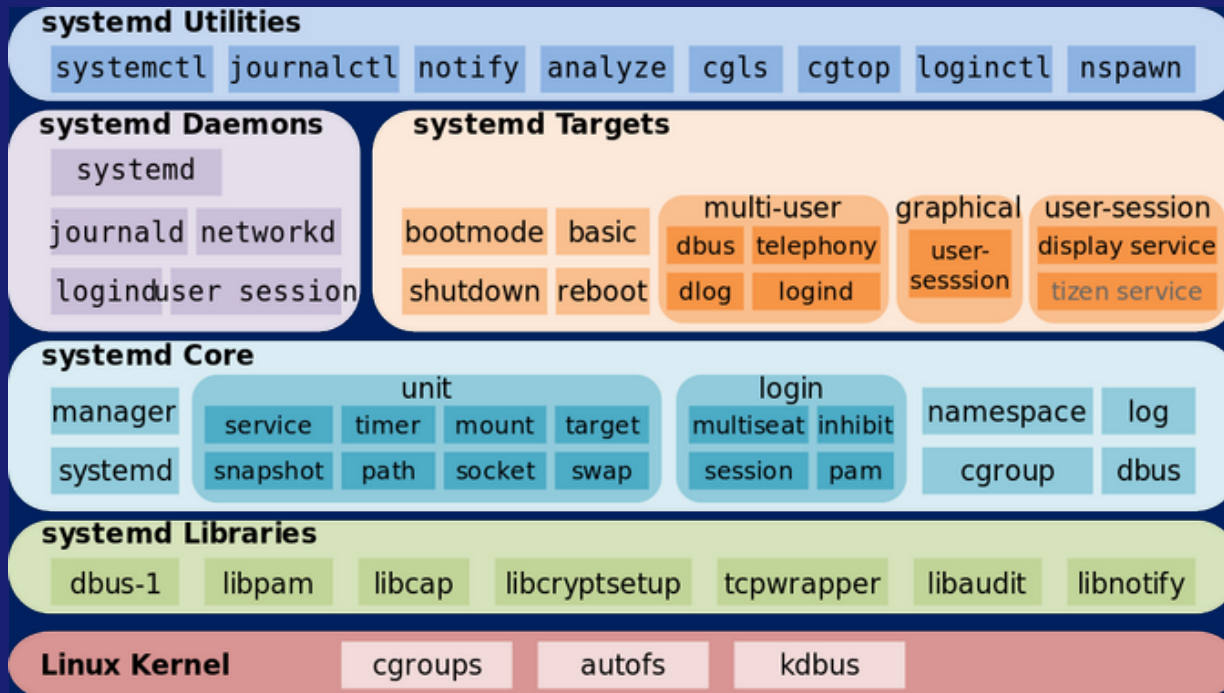


systemd-2



systemd

systemd는 기존에 사용하던 system-v, up-start를 대신한다. 현재 사용중인 LSB(Linux Standard Base)에서는 systemd기반으로 구성이 되어있다. 레드햇 계열의 리눅스는 systemd를 채용하고 있다.



systemd 통합 중...

다음과 같이 현재 systemd는 kABI를 통해서 통합을 하고 있다.

/sys	커널의 장치, 드라이버 및 기타 커널 정보를 사용자 공간(userspace)에 노출하기 위한 인터페이스
/proc	커널 설정, 프로세스 및 기타 커널 정보를 사용자 공간에 노출하기 위한 인터페이스
/dev	커널 디바이스 노드를 사용자 공간에 노출하기 위한 디렉터리
/run	사용자 공간 소켓 및 런타임 파일을 위한 위치
/tmp	휘발성(volatile) 사용자 공간 임시 파일 시스템 객체를 위한 위치 (X)
/sys/fs/cgroup	커널 컨트롤 그룹(cgroup) 계층 구조를 사용자 공간에 노출하기 위한 파일 시스템
/sys/kernel/security	특수 목적의 커널 객체를 사용자 공간에 노출하기 위한 인터페이스 **/sys/kernel/debug (X), /sys/kernel/config (X) 포함

systemd 통합 중...

앞에 내용 계속 이어서...

/sys/fs/selinux	SELinux 보안 데이터를 사용자 공간에 노출하기 위한 인터페이스
/dev/shm	사용자 공간 공유 메모리(shared memory) 객체를 위한 위치
/dev/pts	커널의 가상(pseudo) TTY 디바이스 노드를 사용자 공간에 노출하기 위한 인터페이스
/proc/sys/fs/binfmt_misc	커널에 추가적인 바이너리 실행 형식(binary format)을 등록하기 위한 인터페이스 (X)
/dev/mqueue	메시지 큐(mqueue) IPC 객체를 사용자 공간에 노출하기 위한 인터페이스 (X)
/dev/hugepages	"Huge Page" 메모리 페이지를 할당하기 위한 사용자 공간 API (X)
/sys/fs/fuse/connections	커널의 FUSE 연결 정보를 사용자 공간에 노출하기 위한 인터페이스 (X)
/sys/firmware/efi/efivars	펌웨어(EFI) 변수들을 사용자 공간에 노출하기 위한 인터페이스

systemd

systemd는 PID 1(/bin/init) 대체함.

기존에 사용하던 System-V Init/Up-Start(Init) 혹은 BSD Init를 더 이상 사용하지 않는다. 기존에 사용하던 init 은(는) 시작만 바이너리이며, 대다수 자원은 쉘 스크립트 기반으로 구성 및 관리하였다.

예를 들어서, 기존에 사용하던 쉘 스크립트는 어떤 서비스가 어떤 파일이나 혹은 디렉터리는 **생성/삭제/제거**하는지 알 수 없다.

하지만, systemd는 어떠한 부분에 대해서 자원 변경 및 수정이 발생하였는지 확인이 가능하다.

또한, systemd는 이전에 외부 프로그램으로 사용하였던 부분도, 시스템 블록 영역으로 통합하고 있다. 대표적인 서비스는 "chronyd", "crond"와 같은 서비스가 있다.

기본 명령어

systemd

systemctl 명령어 자동화

진행 전, `bash-completion`이나 혹은 `fish`, `zsh` 셸 기반으로 작업을 권장. `bash-completion`를 사용하기 위해서는 다음과 같이 명령어 실행.

```
# dnf install bash-completion
# rpm -qa bash-completion
# complete -r -p
```

다시 로그인 혹은 `bash` 명령어 실행. `source` 명령어는 종종 오류가 발생함.

```
# bash
# source /etc/profile.d/bash_completion.sh
```

관리 명령어(ctl1)

systemd로 통합이 되면서, 장치 영역 및 커널 모듈과 같은 부분은 관리 명령어 제어. 관리 명령어로 대다수 하드웨어 및 커널에 대해서 조정이 가능하다.

너무 명령어가 많기 때문에 아래 명령어로 간단히 ctl(조정 명령어)를 확인한다.

```
# find /usr/bin -name "*ctl" -type f -print -exec grep -IL . "{}" \; | uniq
/usr/bin/keyctl
/usr/bin/teamdctl
/usr/bin/powerprofilesctl
/usr/bin/wdctl
/usr/bin/busctl
/usr/bin/hostnamectl
/usr/bin/journalctl
```

관리 명령어(ctl2)

/usr/bin/locaectl

/usr/bin/loginctl

/usr/bin/systemctl

/usr/bin/timedatectl

/usr/bin/bluetoothctl

/usr/bin/boltctl

/usr/bin/switcherooctl

/usr/bin/bootctl

/usr/bin/coredumpctl

관리 명령어(ctl3)

/usr/bin/evmctl

/usr/bin/kdumpctl

/usr/bin/udisksctl

/usr/bin/pactl

/usr/bin/wpctl

/usr/bin/flatpak-coredumpctl

/usr/bin/sg_stream_ctl

/usr/bin/sg_bg_ctl

/usr/bin/machinectl

시스템 주요 관리 명령어 1

대표적인 관리 명령어는 다음과 같다. 모든 명령어를 다 확인이 불가능 하지만, 대표적인 몇몇 명령어만 다루어 본다.

hostnamectl

호스트 이름을 설정하는 도구. 사용 파일은 `/etc/hostname`이지만, 수동으로 변경은 권장하지 않으며, 호스트 관련 정보는 이 명령어로 수정을 권장 한다. 추가적으로 CPE정보를 추가 제공한다.

<https://www.redhat.com/en/blog/common-platform-enumeration>

timedatectl

시간/날짜 및 타임존은 더 이상 `tzselect` 명령어가 아닌, 이 명령어로 수정한다. 가급적이면 수동으로 하지 않는다.

NTP 서버 및 클라이언트 정보 확인도 가능합니다. 다만, `chronyd` 아닌, `systemd-timesyncd`, `systemd-timedated`를 통해서 확인이 가능하다.

본딩 및 팀밍

teamdctl

리눅스 본딩을 대안으로 나온 팀밍 서비스 관리 명령어. 여전히 본딩을 지원하고 있으나, 가끔적이면 Teamd 기반으로 사용을 권장한다. 본딩 팀밍은 이전 **rh-ifcfg** 형식을 지원하지 않는다.

<https://github.com/jpirko/libteam>

최근 내용 업데이트

더 이상 리눅스 커뮤니티는 teamd로 전환을 하지 않기로 결정함. 2019년~2020년부터 분위기가 변경되기 시작함.

1. RHEL 8 (2019년 출시) 부터 Red Hat 계열에서는 bonding 추천으로 가이드 변경. RHEL 7까지는 teamd 권장
2. SUSE SLES 15 SP2 (2020년 출시) 무렵부터 team devic는 legac로 간주
3. Debian과 Ubuntu는 RHEL/SUSE보다는, 대략 2018~2019 "teamd를 메인으로 고려하지 않는다"고 선언
4. 결국 다시 bonding으로 다시 전환

teamd 네트워크 쇠락의 이유 1

1. 복잡성 (Complexity)

- teamd는 별도 데몬(daemon) 프로세스를 계속 동작
- d-bus 통신, JSON 구성, event handler 등 구조가 너무 복잡
- bonding은 커널 안에 깔끔하게 구현되어 있고, 별도 데몬 없이 바로 동작
→ 운영 안정성, 단순성 모두 bonding이 우위

teamd 네트워크 쇠락의 이유 2

2. 유지보수 문제 (Maintenance Overhead)

- teamd는 Red Hat 엔지니어 몇 명이 시작. 하지만, 프로젝트 유지가 어려움
- GitHub 저장소도 2020년 이후 업데이트 거의 없음
- 커널 레벨의 bonding은 계속 적극적으로 유지/보수 되고 있음
→ 장기적으로 보면 bonding이 생존 가능성이 높음.

teamd 네트워크 쇠락의 이유 3

3. 통합성 문제 (Integration Trouble)

- teamd는 특별한 기능(예: fast failover, link monitoring)을 제공
- systemd-networkd나 NetworkManager 같은 네트워크 관리 시스템과 통합이 안됨
- modern networking stack (Netplan, wicked, systemd-networkd) 쪽은 bonding이 적절

systemd 통합 코어

machinectl

systemd 기반으로 가상머신이나 혹은 컨테이너 서비스를 동작한다.

지원 파일 형식은 `raw`, `tar(Containerfile)`를 지원한다. `rootless`, `rootful` 형태로 구성이 가능.

udisksctl

기존의 `lsblk`, `mount`, `umount`으로 관리하는 부분을 `dbus`를 통해서 통합 및 관리. 앞으로 장치 연결 및 구성은 이 명령어로 관리한다.

`systemd-mountd`와 같이 상호작용한다. `.mount`유닛 통해서 구성된 자원을 관리한다.

sysctl

리눅스 커널 파라미터 조정 및 변경 명령어. 해당 명령어는 `systemd`과 통합이 되지 않았음. 이 과정에서는 이 부분에 대해서 다루지 않는다.

systemd 통합 코어

systemd-timer(.timer, crond)

기존 **crond** 서비스와 비슷하게, 특정 시간에 명령어 실행을 도와주는 서비스 유닛. 기존의 **crond** 데몬은 **.timer**로 대체가 될 예정이다.

이 부분은 뒤에서 더 다루도록 한다.

localectl

localectl으로 시스템 로케일을 변경한다. 이전에는 수동으로 로케일 설정을 변경하였으나, 현재는 **dbus** 및 **systemd** 통합으로 인하여 해당 명령어로 변경을 권장하고 있다.

이 명령어는 **시스템 전역(system profile)**으로 적용 시, 사용을 권장. **사용자 영역(user session)**은 기존과 동일하게 로케일 시스템 변수를 그대로 사용이 가능함.

systemd 통합 코어

journalctl

rsyslog(/var/log/)를 대신하여 앞으로 systemd-journald 서비스로 관리한다. 현재 대다수 리눅스 배포판은 journald 기반으로 넘어가고 있다. 이 부분도 뒤에서 더 다루도록 한다.

systemd에서 로그 기록을 남기는 곳은 다음과 같다.

- /run/log/journald
- /var/log/journald

현재 대다수 systemd 배포판은 systemd-journald로 전환을 하였고, 레드햇은 기존 시스템과 호환성을 위해서 여전히 rsyslogd를 지원하고 있다.

systemd 통합 코어

systemd dbus

시스템에서 사용하는 dbus채널 및 연결 상태를 확인한다. 이는 윈도우 서버에서 사용하는 com/dcom와 비슷한 개념이다. busctl 명령어를 통해서 확인이 가능하다.

logind

사용자 로그인 정책을 관리 시 사용하는 명령어. 이 명령어를 통해서 사용자 시작 프로그램 및 시스템에 연결이 되어있는 사용자 잠금 및 차단 기능을 사용할 수 있다. 로그인 관리 명령어는 loginctl를 통해서 관리한다.

bootctl

systemd에서 제공하는 부트 로더(boot loader). 일반적인 배포판은 grub2 기반으로 되어 있기 때문에 모든 기능을 사용할 수는 없지만, 기본적인 상태 확인 기능은 사용이 가능하다.

loginctl

loginctl

이전에는 로그인 세션을 프로세스 단위로 관리하였지만, 지금은 systemd에서 cgroup기반으로 세션 관리를 한다.

이전에 사용하였던 `last`, `lastlog`, `w`와 그리고 `pgrep`, `pkill`를 통해서 사용자 프로세스 종료가능하다. 역시, 이 부분도 systemd를 통해서 통합 관리가 되고 있다.

```
# loginctl list-users
  UID USER LINGER STATE
1000 tang no      active

1 users listed.
# loginctl list-sessions
SESSION  UID USER SEAT TTY   STATE  IDLE SINCE
      1 1000 tang      pts/0 active no

1 sessions listed.
```

loginctl

loginctl

세션 종료 이후에도 계속 프로그램이 동작이 되려면, 다음처럼 **loginctl** 명령어로 세션 설정을 변경해야 한다.

```
# loginctl enable-linger  
loginctl show-user test1 | grep Linger  
Linger=yes
```


networkctl

systemd에서는 네트워크 기능이 통합이 되었다. 현재 사용중인 ifcfg-script, NetworkManager는 앞으로 systemd-networkd로 변경이 된다.

networkctl

networkctl 명령어는 다음과 같이 사용이 가능하다. 이 부분에 대해서는 네트워크 세션에서 한번 더 다룬다.

```
# networkctl list
```

IDX	LINK	TYPE	OPERATIONAL	SETUP
1	lo	loopback	carrier	unmanaged
2	eth0	ether	routable	unmanaged
3	eth1	ether	routable	unmanaged

hostnamectl

systemd는 다음과 같은 영역 및 서비스들이 통합되고 있다. 아래는 영역별로 간단하게 정리한 내용이다.

hostnamectl

호스트 이름은 hostnamectl로 통합이 되고 있다. hostnamectl은 systemd의 자원의 일부분이며, "호스트 이름" 이외, 컴퓨터가 사용하는 "머신 아이디" 및 "폼-팩터(Form Factor)" 정보를 가지고 있다.

```
# hostnamectl set-hostname servera.example.com
# hostnamectl deployment lab
# hostnamectl icon-name vm
# hostnamectl chassis laptop
# hostnamectl location SEOUL
```

timedatectl

timedate

호스트의 시간 및 날짜 그리고, NTP를 담당하는 서비스.

앞으로 NTP 관련된 부분은 `systemd-timedated.service`으로 변경될 예정이다. 사용 방법은 아래와 같다.

```
# timedatectl ntp-servers
# timedatectl revert
# timedatectl timesync-status
# timedatectl show-timesync
# timedatectl set-ntp false
```

timedatectl

```
# timedatectl status
# timedatectl show
# timedatectl set-time 13:20:00
# timedatectl list-timezones
Asia/Tokyo
# timedatectl set-timezone Asia/Tokyo
# timedatectl set-local-rtc true
# timedatectl set-ntp true
```

machinectl

machinectl

systemd 기반으로 컨테이너 혹은 가상머신을 서비스 형태로 사용 시 사용한다. 아래 명령어로 간단하게 컨테이너를 구성한다. systemd-machined는 동작하는 구조는 다음과 같다.

```
systemd-nspawn
├─ systemd (PID 1 역할)
│   └─ Linux kernel
│       ├── namespaces
│       ├── cgroups v2
│       ├── seccomp
│       └─ capabilities
```

nspawn

systemd-nspawn

간단하게 컨테이너 기반으로 실행 시, 아래와 같이 명령어를 실행한다.

```
# dnf install -y systemd-container  
# mkdir -p /var/lib/machines/demo  
# systemd-nspawn -D /var/lib/machines/demo /bin/sh
```

nspawn

혹은 아래와 같이 구성이 가능하다.

```
# dnf install systemd-container
# dnf -y --releasever=9 --nogpg --installroot=/srv/test install systemd
passwd dnf rocky-release vim-minimal
# setenforce 0
# systemd-nspawn -D /srv/test
echo rocky | passwd --stdin root
dnf install procps-ng iproute
exit
setenforce 1
# systemd-nspawn -D /srv/test/ -b
```

-b는 부팅 옵션입니다.

machinectl

systemd-machinectl

systemd 기반으로 가상머신 생성 시 사용한다. 여기서는 간단하게 cirros 이미지 기반으로 VM를 생성 및 구성한다.

```
# dnf install -y systemd-container
# machinectl import-raw cirros-0.3.5.img cirros
# machinectl list-images
# systemd-nspawn -M cirros
```


machinectl

앞에 내용 계속 이어서...

```
# passwd  
# exit  
# machinectl start cirros  
# machinectl login cirros  
# machinectl list
```

localectl

localectl

이전에는 LC_ALL, LANG를 /etc/sysconfig/locale. 하지만, 지금은 systemd으로 통합이 되었기 때문에, localectl 명령어로 수정 및 변경해야 한다.

```
# localectl
System Locale: LANG=ko_KR.UTF-8
    VC Keymap: kr
    X11 Layout: kr
# localectl set-locale C
# localectl
System Locale: LANG=C
    VC Keymap: kr
    X11 Layout: kr
```

udiskctl

udiskctl

`mount`, `umount` 명령어로 관리하던 시스템 파일 시스템 마운트는 `udiskctl`로 통합이 될 예정이다. 지원하는 기능은 이전 명령과 비슷하게 블록장치 연결 및 상태 정보를 확인한다.

제일 큰 차이점은 `udiskctl`은 `mount`, `umount`와 다르게 `root` 권한이 없어도 사용이 가능하다. 이 명령어는 Polkit을 통해서 권한 관리가 가능하다.

```
# udiskctl status
```

```
# udiskctl mount -b /dev/sdb
```

```
Mounted /dev/sdb at /run/media/root/c7b98cdb-de0b-4b6f-a22e-36e4a2cb8669
```

유닛

시스템 블록 구성 파일들

런-레벨 변경

`emergency.target`, `rescue.target` 경우에는 `init(Sys-V)`에서 Single 혹은 S 모드와 비슷함. 단, 응급복구 모드인 `rd.break(ramdisk break)` 경우, 이전 `ramdisk(initramfs)` 기반 복구 기능과 조금 다르다.

- `multi-user` -> `init 3`
- `graphical` -> `init 5`

런-레벨 확인을 하기 위해서 아래와 같은 명령어로 조회가 가능하다.

```
# systemctl -t target list-units --all
# systemctl get-defaults
# systemctl list-dependencies <TARGET_NAME>
# systemctl isolate multi-user (# init 3)
```

TARGET

대상이름	런 레벨	링크 대상	설명
default.target	링크파일	graphical.target multi-user.target	기본적으로 multi-user.target이나 혹은 graphical.target으로 링크가 되어 있음.
graphical.target	5	runlevel5.target	multi-user에서 GUI로 전환
	4	runlevel4.target	사용하지 않음. 본래 런 레벨에서는 4번은 사용하지 않았음. 서비스를 등록하는 경우 multi-user.target를 수정하지 않는 선에서 사용이 가능하다.
multi-user.target	3	runlevel3.target	CLI(console)용 대상. 이전에는 런 레벨 3번이었다.

TARGET

대상이름	런 레벨	링크 대상	설명
없음	2	runlevel2.target	multi-user기반. 하지만, CLI기반으로 부팅이 되며, NFS 및 네트워크 서비스는 시작이 되지 않음.
rescue.target	1	runlevel1.target	읽기 전용 상태로 싱글모드(single mode)로 디스크의 루트 파일 시스템을 램 디스크에 마운트. "rd.break", "rd.rescue"
emergency.target	S		램 디스크에서 어떠한 서비스도 동작하지 않고, 어떠한 블록 장치도 디스크에 연결이 되어 있지 않는 상태. 오직 메인 콘솔만(tty1) 열려 있는 상태.
halt.target			시스템 종료. 하지만 powering단계는 무시.
reboot.target	6	runlevel6.target	재시작
poweroff.target	0	runlevel0.target	시스템 종료 및 powering도 같이 적용.

systemctl unit

systemd에서 관리하는 대다수 자원은 **systemctl** 명령어 기반으로 대다수 유닛을 제어 및 관리한다.

systemctl 명령어는 별도로 옵션을 지정하지 않으면 기본적으로 **.service** 유닛으로 명령어를 실행한다. 서비스 자원 이외에도 꽤 많은 자원이 있으며, systemd 버전에 따라서 자원이 되는 자원 형식이 추가가 될 수 있다.

systemd에서 지원하는 대표적인 시스템 자원 유닛은 아래와 같다.

```
".service", ".socket", ".device", ".mount", ".automount", ".swap",  
".target", ".path", ".timer", ".slice", ".scope"
```


systemctl

systemd에서 제공하는 기본 명령어는 다음과 같다. systemd버전 및 확장 기능 설치에 따라서 지원하는 명령어는 달라진다.

명령어	설명
<code>systemctl start <unit></code>	유닛 즉시 시작
<code>systemctl stop <unit></code>	유닛 즉시 정지
<code>systemctl restart <unit></code>	유닛 재시작
<code>systemctl reload <unit></code>	설정만 다시 불러오기 (서비스 종료 없이)
<code>systemctl enable <unit></code>	부팅 시 자동 시작 등록
<code>systemctl disable <unit></code>	부팅 시 자동 시작 해제

systemctl

앞에 내용 계속 이어서...

명령어	설명
<code>systemctl status <unit></code>	유닛 상태 확인
<code>systemctl is-active <unit></code>	유닛이 현재 실행 중인지 확인
<code>systemctl list-units</code>	현재 로드된 모든 유닛 보기
<code>systemctl daemon-reexec</code>	systemd 자체를 재실행 (PID 1 유지)
<code>systemctl daemon-reload</code>	유닛 파일 변경 후 다시 읽기

notify 스크립트 작업 완료

Type=notify는 서비스가 내부 초기화가 완료되었음을 systemd에 직접 알리는 방식이다. 일반 사용자가 직접 작성하는 경우는 드물며, 초기화 시점 제어가 필요한 서비스나 수동으로 구성한 데몬에서 사용된다.

최소 내용으로 간단하게 아래와 같이 테스트 한다.

```
# vi /usr/local/bin/notify-demo.sh
#!/bin/bash
sleep 3 ## 초기화 작업 흉내
systemd-notify --ready --status="Service is ready" ## systemd에 준비 완료 알림
sleep infinity ## 서비스 유지
# chmod +x /usr/local/bin/notify-demo.sh
```

notify 서비스 연결 및 구성

```
# vi /etc/systemd/system/notify-demo.service
[Unit]
Description=Minimal notify demo
[Service]
Type=notify
ExecStart=/usr/local/bin/notify-demo.sh
NotifyAccess=main
[Install]
WantedBy=multi-user.target
# systemctl daemon-reload
# systemctl start notify-demo
# systemctl status notify-demo
```

analyze

systemd에서 서비스 부트 업 시간을 확인하기 위해서 다음과 같이 명령어를 사용한다.

1. 제일 느린 서비스 순서대로 보여주는 명령어

```
# systemd-analyze blame
```

2. 느린 유닛들 위주로 트리 형태로 출력(target)

```
# systemd-analyze critical-chain
```

3. 전체 부팅 시간 확인은 다음과 같은 명령어

```
# systemd-analyze time
```

4. 이미지 형태로 확인

```
# systemd-analyze plot > boot.svg
```

cgls/cgtop

systemd에서 관리하는 유닛들의 CPU, Memory, Disk 상태를 추적한다.

systemd로 변경이 된 후, 모든 자원들은 cgroup를 통해서 추적 및 감시를 한다. 가상/컨테이너 및 모든 프로그램들은 전부 cgroup를 통해서 자원 제한 및 추적이 되기 때문에 중요한 자원 중 하나이다.

```
# systemd-cgls  
# systemd-cgtop
```

systemd에서는 cgroup를 관리하기 위해서 .slice 라는 자원을 생성한다.

```
# systemctl set-property httpd.service CPUWeight=200 MemoryMax=2G  
# systemctl set-property httpd.service IPAddressDeny=10.10.10.250
```

systemd subsystem

systemd에서 제공하는 하위 시스템

systemd-timesyncd

NTP 클라이언트 서비스

chronyd

레드햇 계열 및 데비안 계열에서 아직까지 표준으로 사용하는 NTP 표준 서버 및 클라이언트 도구이다. 현재 레드햇 계열은 chronyd 기반으로 구성되어 있다. 설정 방법은 간단하다. 아래 설정 내용은 기본적으로 설정된 내용이다.

```
# grep -Ev '^#|^$' /etc/chrony.conf
pool 2.rocky.pool.ntp.org iburst
offline
auto_offline
driftfile /var/lib/chrony/drift
makestep 1.0 3
offset -0.00005
rtcsync
```

NTP서버 정보를 변경 한다.

- pool
- server

iburst/burst의 차이점은 burst는 4번 iburst는 4~8번의 요청을 서비스 시작 시, 바로 요청한다. 다만, burst는 각 요청에 대해서 응답을 기다리기 때문에 동기화가 느리다.

특정한 문제로 시간 값을 잃어버린 경우, 평균 값을 가지고 지속적으로 시간을 조정한다.

chronyd

현재 대다수 리눅스 배포판은 `ntpd`에서 `chronyd`로 변경. 설정 내용은 크게 차이는 없다. 기존 `chronyd`는 다음과 같은 이점이 있다.

1. 빠른 동기화 속도. 최소로 동기화 하면서 동기화를 한다.
 - 보통 데스크탑 같이, 24시간 실시간 동기화가 필요하지 않는 시스템
2. CPU 클럭이 불안정한 시스템에 적합.
 - 예를 들어서 가상머신이나 불안한 클럭 상태의 CPU.
3. 동기화 드리프트 기능 지원.
 - 예를 들어서 데이터베이스 같은 미들웨어.
4. 트래픽이 포화일때 비대칭 처리를 안정적으로 가능.
5. 대다수 배포판은 현재 `ntpd`를 사용하지 않음.

systemd-timesyncd, ntpd 설정

systemd로 넘어 오면서 systemd-timesyncd을 통해서 동기화가 가능하다.

파일 위치는 다음 중 둘 중 하나를 사용해도 된다. 레드햇 포함, 다른 배포판들은 아직 기본으로 설치가 안되어 있기 때문에 사용을 원하는 경우, 설치를 해야 한다.

systemd-timesyncd를 실행하는 경우 자동으로 chronyd.service는 중지가 된다.

- /etc/systemd/timesyncd.conf
- /etc/systemd/timesyncd.conf.d/local.conf

```
# dnf install systemd-timsyncd
# systemctl enable --now systemd-timsyncd
# systemctl disable --now chronyd
```

systemd-timesyncd, ntpd

```
# vi /etc/systemd/timesyncd.conf  
[Time]  
NTP=kr.pool.ntp.org  
FallbackNTP=0.pool.ntp.org 1.pool.ntp.org 0.fr.pool.ntp.org  
ServerName=kr.pool.ntp.org  
# systemctl enable --now systemd-timesyncd  
# systemctl disable --now chronyd
```

systemd-timesyncd, ntpd

```
# timedatectl timesync-status
```

```
    Server: 121.174.142.82 (kr.pool.ntp.org)
```

```
Poll interval: 1min 4s (min: 32s; max 34min 8s)
```

```
    Leap: normal
```

```
    Version: 4
```

```
    Stratum: 3
```

NTP서버(chronyd)

내부적으로 NTP 서버를 구축하기 위해서 chronyd를 사용한다.

```
# vi /etc/chrony.conf
server ntp.lab.int iburst
allow 192.168.0.0/24
driftfile /var/lib/chrony/drift
makestep 1.0 3
rtcsync
keyfile /etc/chrony.keys
leapsectz right/UTC
logdir /var/log/chrony
# firewall-cmd --add-service=ntp
# firewall-cmd --runtime-to-permanent
```

systemd-timers

crond 대체 시스템

기존 스케줄러

현재 시스템은 20년 넘게 `at`, `crontab`, `anacron` 기반으로 예약 작업이 관리가 되었다. 기존 레거시 기반에서는 문제가 없지만, `systemd`에서는 통합이 어려웠다. 이러한 이유로 `at`, `crontab`은 레거시 소프트웨어로 설정이 되었다.

레드햇 및 다른 리눅스 계열 배포판에서 여전히 사용이 가능하다. 다만, 많은 기능이 `systemd`의 `.timer` 자원으로 변환이 되었다.

생성된 작업 리소스는 `/var/spool/cron/`, `/var/spool/at`에 사용자 이름으로 작업이 생성이 된다. 이 생성된 내용을 `crond`, `at` 프로그램이 모니터링 하여, 정해진 시간에 작업을 수행한다.

```
# crontab -l
# crontab -e -u
# atq batch
```


TIMER

레드햇 리눅스 기준, 앞으로 RHEL 8,9부터는 `crond`대신 `systemd-timer`기반으로 자원 구성을 권장한다. 현재, 이전에 사용하였던 예약된 시스템 작업들은 대다수가 `systemd`의 `.timer`기반으로 이전이 되었다.

```
# systemctl list-timers | awk '{ print $14 }'  
dnf-makecache.service  
logrotate.timer  
systemd-tmpfiles-clean.service
```

.timer example

이전 `crond` 처럼 `.timer` 기반으로 설정 및 사용이 가능하다.

```
# vi /etc/systemd/system/test.timer
```

```
[Unit]
```

```
Description=test timer as crond
```

```
Requires=test.service
```

```
[Timer]
```

```
Unit=test.service
```

```
OnCalendar=*-*-* *: *:00
```

```
[Install]
```

```
WantedBy=timers.target
```

.timer example

타이머 기반으로 예약 작업을 구성하면 아래와 같이 설정 파일을 작성한다.

```
# vi /etc/systemd/system/test.service
```

```
[Unit]
```

```
Description=test service
```

```
[Service]
```

```
Type=oneshot
```

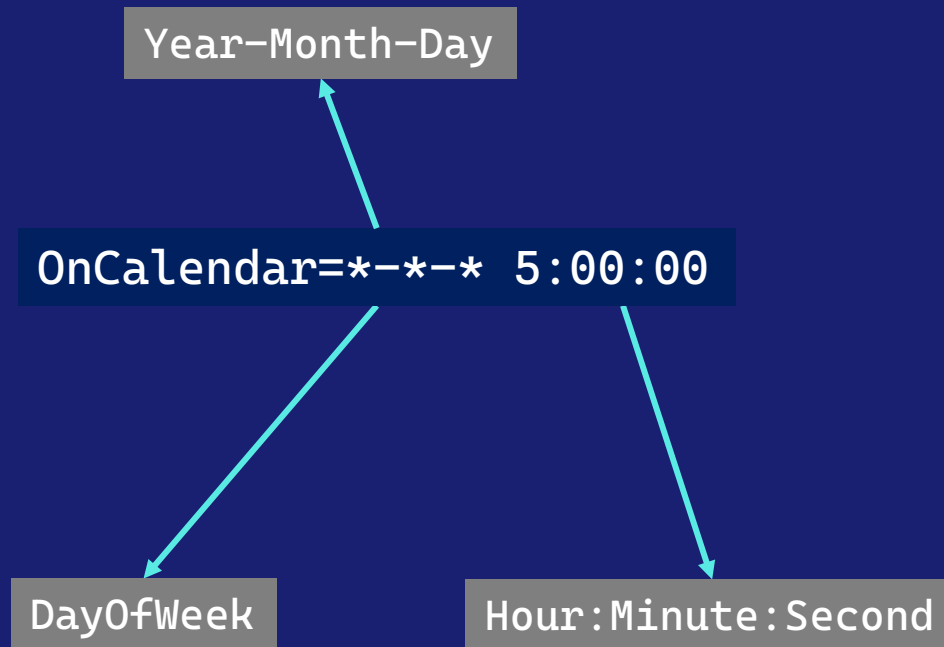
```
ExecStart=/usr/bin/free
```

```
[Install]
```

```
WantedBy=multi-user.target
```

systemd-timers 사양

상세하게 작업일정을 설정하기 위해서 OnCalendar 옵션 사용이 가능하다. OnCalendar에 사용이 가능한 옵션은 아래와 같다.



확장된 기능

최근, systemd에 기능이 추가가 되면서 `systemd-generator-cron2timer` 라는 서비스가 추가가 되었다. 기존에 사용하던 `crontab`에 작업을 등록하면 systemd는 자동적으로 `.timer` 유닛 자원으로 변환한다.

참고로 레드햇 계열은 의도적으로 변환 기능을 제공하지 않는다.

항목	설명
이름	systemd-generator-cron2timer
기능	cron 설정 파일을 systemd 타이머로 변환
동작 시점	systemd 부팅 또는 daemon-reexec 시
관련 경로	/run/systemd/generator/에 임시 유닛 생성
실행 방식	*.timer와 *.service 유닛 쌍으로 구성

전환 확인

레드햇 계열은 다음과 같이 수동으로 작성한다.

```
# vi /etc/systemd/system/demo.timer  
[Timer]  
OnCalendar=*:0/5  
Persistent=true
```

전환 확인

.timer 라는 서비스가 구성이 되면, .service로 동일한 이름으로 구성한다.

```
# vi /etc/systemd/system/demo.service
[Service]
Type=oneshot
ExecStart=/usr/bin/echo "systemd timer" >> /tmp/demo.log
```

정리

1. `generator`는 `cron daemon`을 완전히 대체하지는 않음.
2. `systemd-cron` 패키지를 사용하는 배포판에서 제공되는 경우도 있으며, SUSE나 일부 Debian 계열에서 기본적으로 포함되기도 함. 레드햇 계열은 아직 제공하지 않음.
3. 일반 사용자 `crontab -e`로 설정한 항목은 변환되지 않으며, `root` 혹은 시스템 전역 설정만 해당됨.

systemd-runs

임시 작업 예약

at

기존에 사용하던 at 명령어는 아래처럼 사용한다.

```
# dnf install at
# rpm -ql at
# systemctl status atd && systemctl start atd
# date
# at 14:10
warning: commands will be executed using /bin/sh
at> echo "it's at!" > /tmp/at.txt
job 2 at Fri Nov 11 15:27:00 2022
# atq
# at -l
# ls -l /var/spool/at
a0000101b39196  spool
```

systemd-run

특정 명령어 및 특정 서비스를 임시적으로 스케줄링에 등록하기 위해서 아래와 같이 사용이 가능하다. 뒤에 단위가 붙어있지 않으면 보통은 초(sec)단위로 동작한다.

앞으로 systemd에서는 이전에 사용한 job(at)를 .timer로 대체한다.

```
# systemd-run --on-active=30 /bin/touch /tmp/ihatesystemd
# systemd-run --on-active="30m" --unit vsftpd.service
# systemd-run -p IOWeight=10 updatedb
# systemd-run --on-boot=1800 --on-unit-active=1800 /usr/bin/free
# systemd-run --on-boot=1800 --on-unit-active=1800 -s vsftpd.service
# systemctl list-times
```

systemd-tmpfiles

임시 파일 시스템 관리

systemd-tmpfiles

이전에 crond, tmpfiles를 통해서 관리 하였던 임시 파일 관리자 기능이 systemd로 통합이 되었다. 임시 파일 관리자는 서비스대몬 형태로 동작한다.

```
# systemctl status systemd-tmpfiles-clean.service
# systemd-tmpfiles --cat-config
# /usr/lib/tmpfiles.d/criu.conf
d /run/criu          0755 root root -
# /usr/lib/tmpfiles.d/cryptsetup.conf
d /run/cryptsetup 0700 root root -
```

systemd-tmpfiles

임시 파일을 설정 관리 및 구성하기 위해서 다음과 같이 파일을 생성 및 관리한다. 모든 내용은 다 다루기에 어렵기 때문에 몇가지 대표적인 내용만 다룬다. 전체 내용은 아래 링크에서 확인한다.

- d는 해당 디렉터리가 존재하지 않으면 생성 및 구성한다.
- D는 해당 디렉터리가 존재하면, 내부 파일을 전부 삭제한다.

```
# vi /etc/tmpfiles.d/test.conf
d /run/test 1755 root root 30d
# systemd-tmpfiles --create
# systemd-tmpfiles --clean
# cat /etc/tmpfiles.d/abrt.conf
D /run/test 1755 root root -
# systemd-tmpfiles --create
# systemd-tmpfiles --clean/--remove
```

systemd-tmpfiles 조건식

기호	의미
!	이 라인은 부팅 시에만 안전하게 실행됨. 실행 중 시스템에서 수행하면 시스템을 깨뜨릴 수 있음.
-	이 라인이 create 단계에서 실패하더라도 systemd-tmpfiles 전체 실행이 실패로 간주되지 않음.
+	이 라인은 특별히 위험한 작업(예: 재귀 삭제) 이 포함될 수 있음을 표시. 보호 확인을 우회하고 강제로 실행함.
=	지정된 경로에 이미 객체가 있으면, 파일 타입이 맞는지 검사 후 불일치 시 제거.
~	이 라인은 부팅 시 최초 한 번만 실행됨. 이후 실행에서는 무시됨.
^	기존 객체의 내용은 유지, 하지만 소유자/권한/속성만 재적용.

systemd-mount

고수준 블록 장치 연결 및 구성

기존 마운트 및 새로운 MOUNT

기존에 사용하던 `mount` 명령어는 여전히 사용이 가능하다. 또한, 마운트 시 참조하는 `/etc/fstab`도 여전히 사용이 가능하다.

`systemd`에서는 내부적으로 더 이상 `/etc/fstab` 직접적으로 사용하지 않고, `.mount` 유닛으로 전환 후 사용한다. 앞으로 모든 시스템은들은 가급적이면 `.mount` 유닛 기반으로 구성 및 연결을 권장한다. 정리하면 다음과 같다.

1. `.mount` 확장자를 가지고 있음. 이 유닛은 일반적으로 `/etc/systemd/system`에 위치한다.
2. 디렉터리 구분자는 `/`가 아닌 `-`으로 구분
3. 기존의 `/etc/fstab` 파일은 `systemd-fstab-generator`으로 자동 변환
4. `/etc/fstab`의 내용은 `/run/systemd/generator` 밑에서 생성 및 만들어 짐
5. `/etc/fstab` 내용이 변경이 되면 반드시 `systemctl daemon-reload`으로 갱신 권장

마운트 명령어

기존에 사용한 명령어 `mount`는 여전히 사용이 가능하다. 다만, 시스템 블록으로 통합이 되면서 `systemd-mount`라는 명령어가 하나 추가되었다. 사용 방법은 기존 `mount` 명령어와 거의 동일하다.

```
# systemd-mount /dev/sdb1 /mnt/mydisk
# systemd-mount --options=ro /dev/sdb1 /mnt/mydisk
# systemd-mount --what=UUID=xxxx-xxxx --where=/mnt/mydisk
# systemd-mount --type=ext4 /dev/sdb1 /mnt/mydisk
# systemd-umount /mnt/mydisk
```

systemd 마운트 구성

`systemd-mount`는 영구적으로 마운트 기능을 제공하지 않는다. 재부팅을 하면 해당 내용들은 다시 제거가 되고, `initramfs`에서 부팅 시 다시 재구성한다.

이러한 이유로 새로운 디스크를 `.mount`유닛으로 관리하고 싶으면 다음처럼 작성한다.

```
[Unit]
Description=Mount mydisk
[Mount]
What=/dev/sdb1
Where=/mnt/mydisk
Type=ext4
Options=defaults
[Install]
WantedBy=multi-user.target
```

systemd 자동 마운트 일반 구성

이전에 사용하였던 **autofs**는 복잡하고 사용하기가 어려웠다. **systemd**로 변경이 되면서, **autofs**는 **automount**라는 자원으로 통합 및 변경이 되었다. **automount**는 **mount**유닛과 의존성이 있다.

사용 방법은 앞에서 사용하였던 **mount**와 거의 동일하다. 파일 이름에 꼭 **대시(-)**로 디렉터리 구별을 해야 한다.

```
# vi home-testuser.automount
[Unit]
Description=automount for testuser
ConditionPathExists=/home/testuser

[Automount]
Where=/home/testuser
TimeoutIdleSec=10

[Install]
WantedBy=multi-user.target
```

systemd 자동 마운트 원격 구성

.automount가 사용할 일반 .mount자원도 생성한다.

```
# vi home-testuser.mount
[Unit]
Description=mount for testuser

[Mount]
What=$YOUR_SERVER:/$YOUR_SHARE
Where=/home/testuser
Type=nfs
Options=_netdev,auto

[Install]
WantedBy=multi-user.target
```

이전 자동 마운트(참고용)

systemd의 automount와 비교하기 위해서 autofs 서버를 설치 및 구성한다.

필요하지 않는 경우, 굳이 진행하지 않아도 된다. 아래와 같은 방식 더 이상 사용을 권장하지 않는다.

```
# dnf search autofs
# dnf install autofs
# systemctl enable --now autofs.service
# vi /etc/auto.master.d/direct.conf
/-          /etc/auto.direct
# vi /etc/auto.direct
/direct     -fstype=auto,rw,async node1.example.com:/direct
# vi /etc/auto.master.d/indirect.conf
/home/ /etc/auto.indirect
# vi /etc/auto.indirect
*          -rw,soft,async          node1.example.com:/indirect/&
```

자동마운트 확인

연결이 완료가 되면, 아래와 같이 확인이 가능하다.

```
# showmount -e node1.example.com  
# ssh testuser@node1.example.com  
# pwd  
# df -h
```

정리

위의 기능을 정리하면 다음과 같다.

구분	autofs	systemd-automount
기본 구조	별도 autofs 데몬이 동작하며, 마운트 트리거를 감시	systemd의 .automount 유닛이 마운트 트리거를 담당
서비스 이름	autofs.service	-.mount, .automount 유닛 시스템
설정 파일	/etc/auto.master, /etc/auto.misc, /etc/auto.direct 등	.mount, .automount 유닛 파일
설정 복잡도	복잡한 트리, LDAP, NIS 연동 가능 (대규모 지원)	단순한 경로별 자동 마운트에 최적화
초기 설치	별도 패키지 설치 필요 (예: yum install autofs)	systemd 기본 탑재
관리 방법	autofs 서비스 리스타트 필요	systemctl로 유닛 reload/restart
대상 환경	대규모 네트워크 환경 (NFS, LDAP 대량 마운트)	소규모 서버, 단일 경로 자동 마운트

정리

앞에 내용 계속 이어서...

구분	autofs	systemd-automount
트리거 방식	커널 VFS 핸들링 후 autofs 데몬이 mount 요청	systemd가 파일시스템 접근 시 직접 mount 요청
의존성 관리	autofs 자체에서 mount 시점을 조절	systemd dependency로 부트/서비스 순서 제어 가능
퍼포먼스	대규모 환경에서 안정적 성능	간단한 환경에 빠르고 가벼움
유지보수	별도 데몬 관리 필요 (autofs 업그레이드 등)	systemd 업데이트에 통합됨

systemd-resolved

도메인 서비스 구성

설명

기존 리눅스 시스템은 유닉스 계열과 동일하게 다음과 같은 위치에서 도메인 정보를 관리 및 참조를 한다.

1. `/etc/hosts`
2. `/etc/resolv.conf`
3. `/etc/hosts.conf`

이 파일들은 관리자 혹은 특정 프로그램에서 관리하였지만, `systemd-resolved`를 설치하면, `systemd`에서 네트워크에 사용하는 호스트 정보들을 관리한다. 또한, 통합이 되면서 관리 명령어가 별도로 제공이 된다.

설명

`systemd-resolved`를 좀 더 자세히 설명하면 다음과 같다.

1. `systemd-resolved`는 `systemd` 네임 서비스로, 시스템의 DNS 요청을 처리하는 중앙 서비스
2. 이 서비스는 `/etc/resolv.conf`를 `symlink` 형태로 관리하며, `/run/systemd/resolve/stub-resolv.conf` 또는 `/run/systemd/resolve/resolv.conf` 설정 및 관리
3. DNS 서버, 도메인, 검색 목록 등은 `NetworkManager`, `systemd-networkd`, 그리고 수동 설정 등을 통해 수집
4. `/etc/nsswitch.conf` 파일에서 `hosts`: 항목에 `resolve` 또는 `dns`가 포함되어 있을 때만 `systemd-resolved` 사용

도메인 클라이언트 서비스 설치

설치는 다음과 같이 한다.

```
# dnf install systemd-resolved  
# zypper install systemd-resolved  
# systemctl enable --now systemd-resolved
```

도메인 클라이언트 서비스 구성

DNSSEC 사용이 필요한 경우, 아래와 같이 설정한다.

```
# ls -l /etc/resolv.conf
/etc/resolv.conf → /run/systemd/resolve/stub-resolv.conf
/etc/resolv.conf → /run/systemd/resolve/resolv.conf
# ls -l /etc/resolv.conf
# ln -sf /run/systemd/resolve/stub-resolv.conf /etc/resolv.conf
# vi /etc/systemd/resolved.conf
[Resolve]
DNSSEC=yes
DNSOverTLS=yes
# systemctl restart systemd-resolved
```

설치 후 확인

상태를 확인하기 위해서 아래와 같이 명령어를 실행한다. 각 인터페이스별 설정도 가능하다.

```
# resolvectl status -4 eth0
Link 2 (eth0)
Current Scopes: DNS LLNMR/IPv4 LLNMR/IPv6
...
DNS Servers: 8.8.8.8
```

구성 확인

특정 도메인을 조회하기 위해서 아래와 같이 실행한다.

```
# resolvectl query example.com
```


구성 확인

인터페이스 별 도메인 서버 질의는 아래와 같이 가능하다.

```
# resolvectl status eth0
```

systemd-container

컨테이너 서비스

설명

`systemd-container`는 컨테이너 환경에서 `systemd`가 정상 작동하도록 필요한 설정과 파일 구조를 제공하는 컨테이너 이미지 또는 컨테이너 빌드 전략. 일반적으로 다음과 같은 패턴을 따른다.

- `systemd`가 PID 1로 실행됨
- `--privileged` 또는 적절한 capabilities가 필요함
- `/sys/fs/cgroup`, `/run`, `/tmp` 등이 `tmpfs` 혹은 볼륨으로 제공됨
- SELinux, AppArmor 정책이 조정됨
- `journald` 기반으로 시스템 로깅이 필요한 경우

이 기능은 Podman과 비슷하게 보이지만, `systemd` 기반으로 관리되는 서비스로 동작하기 때문에 조금은 다른 운영 및 관리 방법이다.

일반적인 컨테이너 서비스

만약, systemd 기반이 아닌, Podman로 구성하는 경우, 보통 다음과 같이 권한 조정이 필요하다. 이 경우에는 init를 사용하지만, 컨테이너 내부에서 사용하는 init기반으로 동작한다.

```
# podman run --rm -it \  
  --name systemd-container \  
  --privileged \  
  --volume /sys/fs/cgroup:/sys/fs/cgroup:ro \  
  registry.fedoraproject.org/fedora:latest /sbin/init
```

systemd 기반 컨테이너 구성(레드햇 버전)

레드햇 계열 경우에는 다음과 같이 구성이 가능하다.

```
# dnf --installroot=/var/lib/machines/rocky-install --releasever=9 install
-y @core systemd passwd iproute iputils vim
echo "rockybox" | sudo tee /var/lib/machines/rocky-install/etc/hostname
# systemd-firstboot --root=/var/lib/machines/rocky-install \
  --locale=en_US.UTF-8 --timezone=Asia/Seoul --hostname=rockybox \
  --setup-machine-id
# touch /var/lib/machines/rocky-install/.nspawn
# chcon -R system_u:object_r:svirt_sandbox_file_t:s0
/var/lib/machines/rocky-install
# systemd-nspawn -D /var/lib/machines/rocky-install --machine rockybox --
boot --network-bridge=br0
```

systemd 기반 컨테이너 구성(수세 버전)

이러한 부분을 `machinectl`와 같은 명령어로 구성 및 관리가 가능하다. 수세 리눅스를 `nspawn`으로 구성하는 경우, 아래와 같이 실행한다.

```
# zypper --root /var/lib/machines/suse-install ar  
https://download.opensuse.org/tumbleweed/repo/oss/ repo-oss  
# zypper --root /var/lib/machines/suse-install --gpg-auto-import-keys  
refresh  
# zypper --root /var/lib/machines/suse-install install -y systemd vim  
iproute2 iputils  
# systemd-nspawn -D /var/lib/machines/suse-install --machine susebox --boot  
--network-bridge=br0
```

컨테이너 서버 기반 ssh 서버 구성

구성 및 실행이 완료가 되면 컨테이너에 접근 후, 다음과 같이 비밀번호 설정을 한다.

```
# passwd  
# dnf install -y openssh-server  
# systemctl enable sshd  
# systemctl start sshd
```

컨테이너 내부에서 테스트

내부에서 다음과 같이 명령어를 실행한다. 컨테이너가 마치 가상머신처럼 동작하는 걸 확인이 가능하다.

```
# machinectl shell rockybox  
systemctl status  
systemctl start sshd  
journalctl -xe
```


컨테이너 프로세스 관리

생성된 컨테이너를 관리 및 구성하기 위해서 아래와 같이 명령어 사용이 가능하다.

```
# machinectl  
# machinectl start rockybox  
# machinectl stop rockybox  
# machinectl shell rockybox  
# machinectl poweroff rockybox
```

컨테이너 서비스 유닛으로 구성

서비스 형태로 구성하기 위해서 아래와 같이 유닛 파일을 생성한다.

```
# /etc/systemd/nspawn/rockybox.nspawn
[Exec]
Boot=yes

[Network]
Bridge=br0

[Files]
Bind=/etc/resolv.conf
```

가상머신 구성(systemd 기반)

`machinectl`를 통해서 가상머신 이미지도 가볍게 실행이 가능하다. 다만, 조건은 qemu/kvm 사용이 가능한 환경이어야 한다.

1. QEMU/KVM
2. QCOW2 이미지
3. libvirt는 필요하지 않음

가상머신 구성(systemd 기반)

기존에 사용하였던 `systemd-nspawn`과 차이점은 아래와 같다.

항목	설명
<code>machinectl</code>	systemd 기반 머신(VM 또는 컨테이너) 관리 도구
<code>systemd-nspawn</code>	chroot 기반 system container 실행기
<code>systemd-machined</code>	machinectl과 backend에서 통신, QEMU/nspawn 등 연결
QEMU 통합	QEMU VM을 systemd의 machine으로 등록하면 제어 가능

가상머신 이미지 준비 및 등록

systemd 기반으로 구성하기 위해서 아래와 같이 명령어를 수행한다.

```
# wget https://cloud.centos.org/centos/9-stream/x86_64/images/CentOS-  
Stream-GenericCloud-9-latest.qcow2  
# machinectl import-raw CentOS-Stream-GenericCloud-9-latest.qcow2 centos9  
# systemd-nspawn -M centos9 --boot
```

nspawn 서비스 구성

혹은 서비스로 사용하기 위해서 아래와 같이 구성이 가능하다.

```
# vi /etc/systemd/nspawn/centos9.nspawn
[Exec]
Boot=yes

[Files]
Image=/var/lib/machines/centos9.raw
# machinectl start centos9
# machinectl shell centos9
```

컨테이너 및 가상머신 정리

실행 구조는 다음과 같이 된다.

항목	설명
PID 1	systemd (init 시스템)
프로세스 트리	systemd → dbus, sshd, nginx, journald, 등
서비스 관리	systemctl 명령이 완벽히 작동함
로그	journalctl 가능 (내부 journald 작동)

systemd-homed

새로운 홈 디렉터리 및 계정 관리

소개

systemd 기반 시스템으로 전환되면서, 기존의 사용자/그룹 관리 방식과 홈 디렉터리 구성 방식도 점차 변화하고 있다.

그 중 하나가 **systemd-homed** 기능으로, 이 기능은 사용자 계정과 홈 디렉터를 루프백 디스크 이미지 형태로 분리하여, 컨테이너 환경이나 USB 같은 이동식 미디어를 통해 구성하고 관리할 수 있도록 설계되었다.

이러한 방식은 사용자의 홈 디렉터를 암호화하거나, 여러 시스템 간에 손쉽게 공유·이동할 수 있는 장점을 제공한다.

그러나 현재로서는 기존의 **/etc/passwd** 기반 사용자 계정과는 달리, **SSSD**, **LDAP**, **Active Directory** 등 외부 인증 시스템과는 직접 연동되지 않는다.

따라서 기업 환경처럼 중앙 인증이 필요한 경우에는 기존의 전통적인 계정 관리 방식을 사용하는 것이 더 적합하다. 반면, 독립적이고 보안이 중요한 데스크탑 환경이나 컨테이너 기반 시스템에서는 **systemd-homed**를 고려해볼 수 있다.

소개

긴 이야기이지만 결론은 다음과 같다.

1. 더 이상 `passwd`, `adduser`와 같은 명령어를 사용하지 않는다
2. 각 사용자의 비밀번호는 홈 디렉터리에 구성 및 저장한다
3. 사용자 관련 파일을 하나로 통합한다

```
/home/choigookhyun.home/  
├─ home.json      ← UID, shell, groups, 정책  
├─ home.key       ← 암호화 키 (옵션)  
└─ data/          ← 실제 홈 디렉터리  
  
/home/choigookhyun.home  
(= LUKS 암호화된 단일 이미지 파일)
```

사용자 홈 디렉터리 구성

설치는 `systemd` v245 버전 이상에서 가능하다. 아래와 같이 실행한다. 레드햇은 9 버전 이후부터 사용이 가능하다. 레드햇은 EPEL 저장소가 구성되어 있어야 한다.

```
# dnf install systemd-homed
# zypper install systemd-home
# systemctl enable --now systemd-homed.service
# homectl create gookhyun --real-name="Gook Hyun Choi" --storage=directory
--disk-size=2G --member-of=wheel
```

사용자 홈 디렉터리 구성

암호화가 필요한 경우, 아래와 같이 실행한다.

```
# homectl create secureuser --real-name="Secure User" --storage=luks \  
--disk-size=5G --fs-type=ext4 --member-of=wheel
```

사용자 홈 디렉터리 목록 및 검사

생성된 사용자는 아래 명령어로 확인이 가능하다.

```
# homectl list  
# homectl inspect gookhyun
```

사용자 홈 디렉터리 기반 PAM 적용

로그인 시, homed 사용자를 적용하기 위해서 아래와 같이 PAM에 아래와 같이 수정한다.

```
# grep pam_systemd_home.so /etc/pam.d/*
# vi /etc/pam.d/system-auth
auth      required      pam_systemd_home.so
account   required      pam_systemd_home.so
password  required      pam_systemd_home.so
session   required      pam_systemd_home.so
```

구성

사용자 제거는 다음과 같이 가능하다.

```
# homectl remove gookhyun
```

systemd-mail

systemd 기반 모니터링 메일 서비스

소개

`systemd-status-mail`은 `systemd` 상태 정보를 요약하여 메일로 발송하는 자동화된 상태 보고 서비스다. 주로 서버나 시스템 상태를 주기적으로 모니터링하고, 해당 정보를 관리자가 받을 수 있도록 하기 위해 사용된다.

이 서비스는 `systemctl list-units`, `systemctl --failed`, `journalctl` 등과 같은 명령어를 활용하여 현재 `systemd` 유닛들의 상태를 수집하고, 이를 이메일 형식으로 정리하여 송신한다.

명령어 메일 클라이언트 설치

다음과 같이 의존성 도구를 설치한다. 각 배포판에 맞게 구성 및 설치한다.

```
# apt install mailutils  
# dnf install mailx  
# zypper install mailx
```

알림 서비스 구성

알림을 위한 메일 발송 스크립트를 작성한다.

```
# vi /usr/local/bin/systemd-status-mail.sh
#!/bin/bash
HOSTNAME=$(hostname)
MAILTO="admin@example.com"
{
    echo "Subject: [Systemd Status] $HOSTNAME systemd 상태 보고"
    echo ""
    echo "══ 실행 중인 서비스 목록 ══"
    systemctl list-units --type=service --state=running
    echo ""
}
```

알림 서비스 구성

위의 내용 계속 이어서...

```
echo "══ 실패한 서비스 목록 ══"
systemctl --failed
echo ""
echo "══ 최근 100줄 로그 ══"
journalctl -xe --no-pager -n 100
} | /usr/sbin/sendmail "$MAILTO"
# chmod +x /usr/local/bin/systemd-status-mail.sh
```

알림 서비스 구성

자동으로 실행하기 위해서 아래와 같이 `.timer` 서비스를 구성한다.

```
# vi /etc/systemd/system/systemd-status-mail.service
[Unit]
Description=Send systemd status report via email
[Service]
Type=oneshot
ExecStart=/usr/local/bin/systemd-status-mail.sh
```

알림 서비스 구성

위의 내용 계속 이어서...

```
# vi /etc/systemd/system/systemd-status-mail.timer
[Unit]
Description=Run systemd-status-mail daily
[Timer]
OnCalendar=daily
Persistent=true
[Install]
WantedBy=timers.target

# systemctl daemon-reexec
# systemctl daemon-reload
# systemctl enable --now systemd-status-mail.timer
```

정리

서비스가 올바르게 구성이 되면, 다음과 같이 관리자는 메일을 받게 된다.

Subject: **[Systemd Status] myserver.local systemd 상태 보고**

══ 실행 중인 서비스 목록 ══

- `systemd-journald.service` loaded active running Journal Service
- `sshd.service` loaded active running OpenSSH Daemon

══ 실패한 서비스 목록 ══

- `fail2ban.service` loaded failed failed Fail2Ban Service

══ 최근 100줄 로그 ══

Jun 17 06:30:10 myserver systemd[1]: Starting Daily apt upgrade and clean activities ...

Jun 17 06:30:11 myserver systemd[1]: Started Daily apt upgrade and clean activities.
...

systemd-oomd

시스템 메모리 보호

소개

OOM, Out Of Memory의 약자. OOM은 시스템 메모리가 부족하기 시작하면 OOM Killer가 동작하면, 최근에 생성된 프로세스 위주로 중지한다.

이전에는 커널에서 관리 하였지만, 현재는 systemd-oofd가 도입이 되어서 systemd에서 시스템 메모리 관리 및 OOM를 관리한다.

OOM은 기본적으로 리눅스 커널의 C-GROUP를 통해서 관리 및 추적이 된다. 추적 시, systemd에서는 .slice라는 자원을 생성하여 프로세스의 자원 상태를 확인한다.

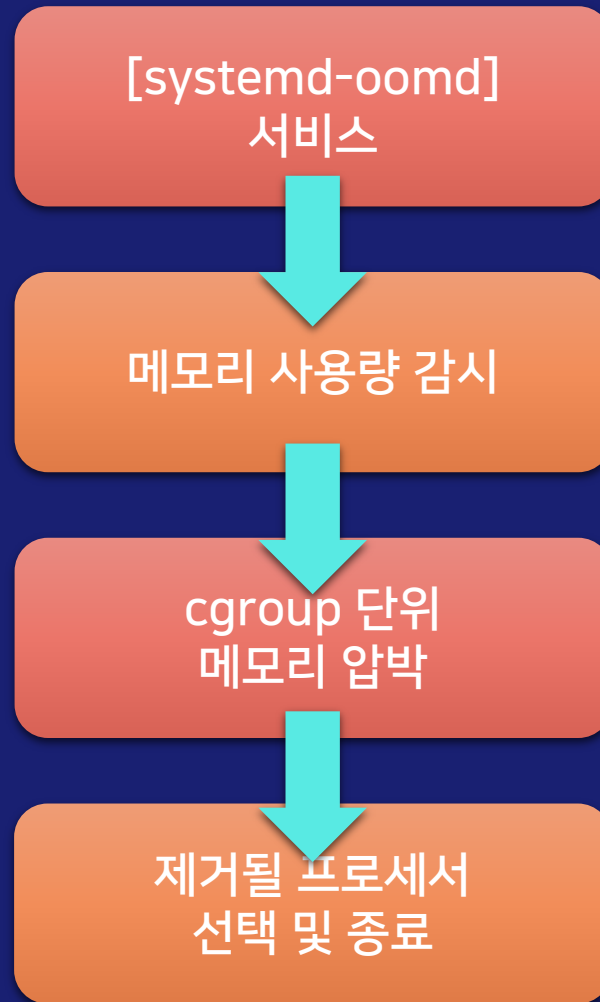
항목	설명
기능 이름	systemd-oofd (systemd 248버전부터 도입)
모니터 대상	서비스, 스코프, 슬라이스 (slice 단위)
동작 방식	메모리, 스왑 사용량을 모니터링해서 임계치 넘으면 프로세스를 정리(kill)
정책	MemoryPressure, MemoryMin, MemoryMax, Swap usage 기반 판단

소개

기존 kernel-oom과 systemd-oomd과 아래와 같은 차이점이 있다.

항목	커널 OOM Killer	systemd-oomd
개입 시점	메모리 진짜 완전히 고갈될 때	메모리 압박 단계에서 조기 대응
대상 선정	OOM Score 기준	압박+정책 기반 더 세밀하게
제어 가능성	낮음	높음 (서비스별 fine control)
목적	커널 보호	시스템 안정성 확보

OOM



명령어 및 설정 파일

올바르게 설치가 되어 있으면, 다음 명령어로 확인이 가능하다. 다만, 버전별로 지원이 안되는 경우 저장소에서 별도 설치 혹은 systemd를 업데이트 한다.

설정 파일 위치는 다음과 같다.

1. `/etc/systemd/oomd.conf`
2. `/etc/systemd/system.conf`
3. `/etc/systemd/user.conf`

```
# systemctl status systemd-oomd
# systemctl enable systemd-oomd
# systemctl start systemd-oomd
# systemctl is-active systemd-oomd
```

설정

설정 파일의 주요 지시자는 다음과 같다.

설정 항목	의미
DefaultMemoryPressureThresholdSec	메모리 압박 지속 시간 설정
DefaultMemoryPressureWatchSec	감시 주기 설정
ManagedOOMMemoryPressure	MemoryPressure 기반 OOM 관리
ManagedOOMSwap	Swap 기반 OOM 관리 활성화

설정

만약, 특정 유닛에게만 적용하는 경우 **드랍 디렉터리(drop directory)**를 만들어서 아래와 같이 지시자를 추가한다.

```
# mkdir -p /etc/systemd/system/httpd.service.d
# vi /etc/systemd/system/httpd.service.d/override.conf
[Service]
ManagedOOMSwap=kill
ManagedOOMMemoryPressure=kill
```

oom 적용 방법

다음과 같이 디렉토리를 구성 후 `systemd-oomd` 기반으로 구성한다.

위의 조건은 다음과 같다.

1. `web.service`, 이 서비스는 무조건 유지가 되어야 한다.
2. `background.service`, 이 서비스는 상태에 따라서 종료가 된다.

```
/etc/systemd/system/  
├─ web.service.d/  
│   └─ oom.conf  
├─ background.service.d/  
│   └─ oom.conf
```

oom 적용 방법

`web.service.d`에 다음과 같이 적용한다.

```
# mkdir -p /etc/systemd/system/httpd.service.d
# vi /etc/systemd/system/httpd.service.d/oom.conf
[Service]
ManagedOOMSwap=inhibit
ManagedOOMMemoryPressure=inhibit
```


oom 적용 방법

background.service.d에 다음과 같이 적용한다.

```
# cat << 'EOF' > /etc/systemd/system/stress.service
[Unit]
Description=Stress Memory Load
After=network.target
[Service]
ExecStart=/usr/bin/stress --vm 1 --vm-bytes 70% --vm-hang 0
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
EOF
```

oom 적용 방법

위의 내용 계속 이어서...

```
# mkdir -p /etc/systemd/system/stress.service.d
# cat << 'EOF' > /etc/systemd/system/stress.service.d/oom.conf
[Service]
ManagedOOMSwap=kill
ManagedOOMMemoryPressure=kill
OOMPolicy=kill
EOF
```

oom 확인

최종적으로 아래와 같이 적용한다.

```
# systemctl daemon-reload
# systemctl enable --now httpd.service
# systemctl enable --now stress.service
# journalctl -u systemd-oomd
# systemctl status -u httpd.slice
# systemctl status -u stress.slice
```

userspace/kexec reboot

시스템 재시작

재시작

systemd 254버전부터 다음과 같은 기능이 추가가 되었다.

1. `soft reboot`
2. `kexec reboot`

soft reboot은 소프트웨어 업데이트 및 사용자 영역(userspace)부분만 재시작이 필요한 경우 사용한다. 이 명령어는 커널 프로세스는 제외하고 사용자 프로세스만 중지 및 시작한다.

kexec reboot는 펌웨어(바이오스)/부트로더 영역을 제외하고 바로 새로운 커널 버전으로 시작한다. 이를 통해서 빠른 재시작이 가능하다. 다만, 중간에 hot-swap형태로 추가된 하드웨어가 있는 경우, 올바르게 인식이 안될 수 있다.

SOFT REBOOT

소프트 리부팅은 다음과 같이 사용이 가능하다. 혹은, 다음 부팅용 루트 준비 후(예: 스냅샷 마운트) 유저스페이스만 전환하여 부팅이 가능하다.

```
# systemctl soft-reboot  
# mount /dev/mapper/newroot /run/nextroot  
# systemctl soft-reboot
```

KEXEC REBOOT

커널 재시작은 다음과 같이 사용이 가능하다.

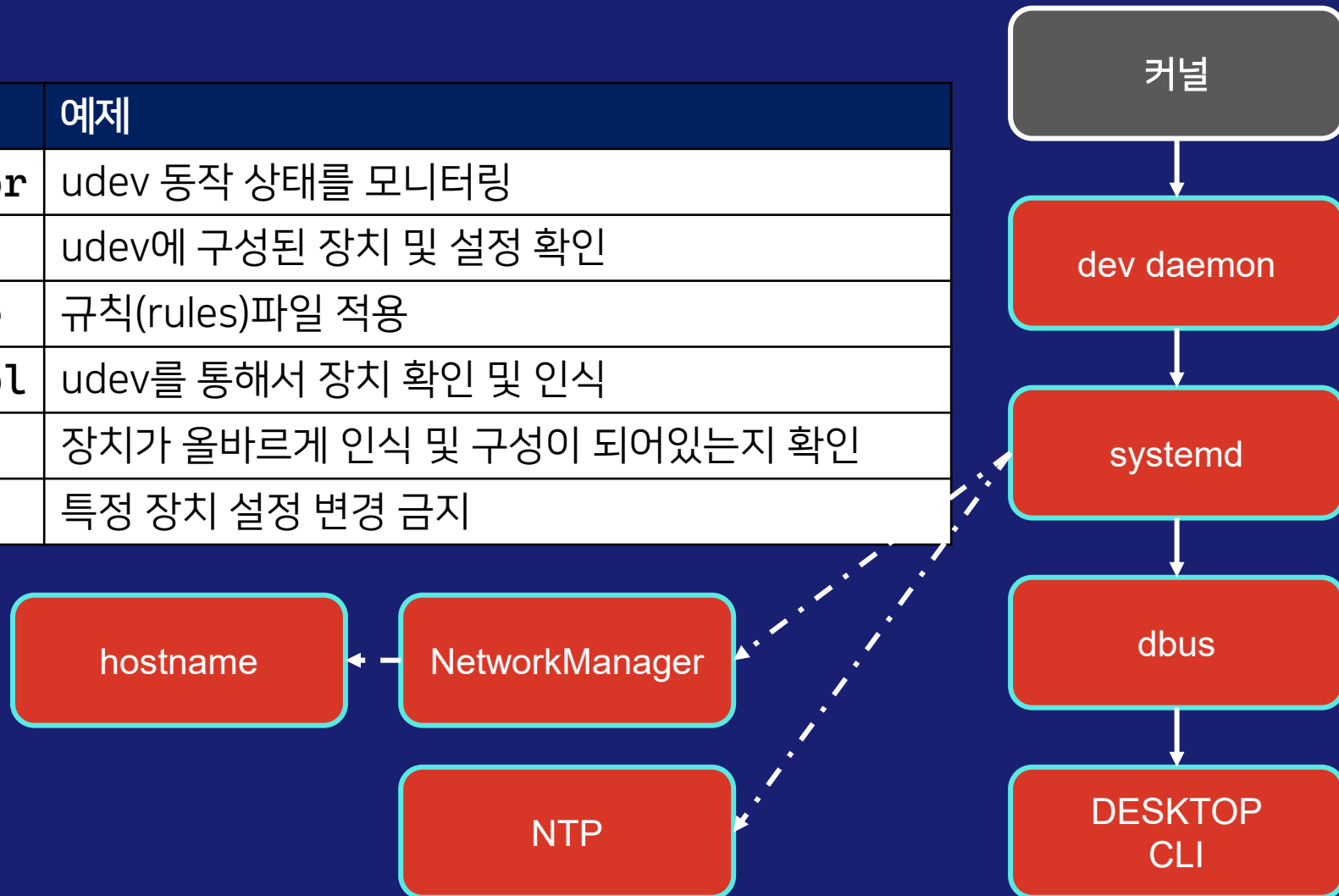
```
# dnf install -y kexec-tools
# grep -i kexec /boot/config-$(uname -r)
(선택) 원하는 커널/initrd를 미리 적재
# kexec -l /boot/vmlinuz-<version> --initrd=/boot/initramfs-<version>.img -
--command-line="$(cat /proc/cmdline)"
즉시 kexec 재부팅
# systemctl kexec
```

system-udev

systemd

설명

명령어	예제
udevadm monitor	udev 동작 상태를 모니터링
udevadm info	udev에 구성된 장치 및 설정 확인
udevadm settle	규칙(rules)파일 적용
udevadm control	udev를 통해서 장치 확인 및 인식
udevadm test	장치가 올바르게 인식 및 구성이 되어있는지 확인
udevadm lock	특정 장치 설정 변경 금지



systemd-udev

systemd-udev는 기존에 사용하던 udev를 시스템 블록으로 통합한 기능이다. udev는 **Userspace Device**의 약자이다. udev는 말 그대로 사용자가 접근 및 사용이 가능한 장치를 구성한다.

기존 방식은 udev 데몬을 통해서 관리를 하였으며, 이를 통해 `/dev`에 장치를 구성 및 생성한다. 이보다 더 이전 방식은 **"MAKEDEV"**를 사용하여 미리 장치를 생성해 놓고, 사용자가 사용하는 방식이었다.

현재 리눅스 커널은 사용자 프로그램과 정보 교환을 위해서 다음과 같은 구성으로 통신한다.

`kernel` → `udev` → `NetworkManager` → `D-Bus` → `Desktop Environment`

이 기능은 커널 2.5 버전(약 2002년경)에 본격적으로 릴리즈 되었으며, 현재 모든 리눅스 배포판에서 표준적으로 사용되고 있다.

udev를 사용하기 위해서는 다음과 같은 명령어로 조회 및 사용이 가능하다.

udev 명령어

명령어는 다음과 같이 제공한다.

명령어	설명	사용 예제
<code>udevadm monitor</code>	udev에서 인식된 장치를 모니터링 한다	<code>udevadm monitor --property</code>
<code>udevadm info</code>	udev에서 연결된 장치 정보를 확인한다	<code>udevadm info --name=/dev/sda</code>
<code>udevadm settle</code>	영구적으로 udev에 정책을 적용한다	<code>udevadm settle --timeout=10</code>

udev 명령어

앞에 내용 계속 이어서...

명령어	설명	사용 예제
<code>udevadm control</code>	udev에서 인식된 장치를 확인	<code>udevadm control --reload</code>
<code>udevadm test</code>	장치가 올바르게 인식되었는지 udev db를 확인	<code>udevadm test /sys/block/sda</code>
<code>udevadm lock</code>	특정 장치가 변경되지 않도록 락을 설정	<code>udevadm lock /dev/sda</code>

systemd udev명령어

systemd에서 udev를 사용하는 방법은 두 가지가 있다.

- 디스크 관리
- 디바이스 관리

디스크는 기존 방식으로 여전히 사용이 가능하다.

systemd로 통합이 되면서 **udisk** 명령어로 **udev**에서 인식 및 관리가 되고 있는 디스크 연결 및 확인이 가능하다. 이 부분은 **udisk**명령어 따로 다룬다.

```
# udiskctl status
```

MODEL	REVISION	SERIAL	DEVICE

Msft Virtual Disk	1.0	600224804580c5d15bbf0d4fd7e372f3	sda
Msft Virtual DVD-ROM	1.0	4d534654202020207305e...7ced624a7d	sr0

systemd udev 정책

systemd에서는 기존에 사용하던 udevd 데몬을 **systemd-udev**으로 통합하였다.

```
# systemctl status systemd-udev
```

이를 통해서 udev 정책 파일을 관리한다. 정책 파일은 다음과 같은 위치에 명시되어 있다.

```
# ls -l /usr/lib/systemd/systemd-udev
```

```
/usr/lib/systemd/systemd-udev -> ../../bin/udevadm
```

```
# ls /lib/udev/rules.d
```

```
60-net.rules
```

```
60-persistent-storage.rules
```

```
# cat 60-net.rules
```

```
ACTION=="add", SUBSYSTEM=="net", DRIVERS=="?*", ATTR{type}=="1",  
PROGRAM="/lib/udev/rename_device", RESULT=="?*", NAME="$result"
```

```
# file /lib/udev/rename_device
```

system-repart

systemd

설명

부팅 시, 자동으로 파티션 구성이 필요한 경우 사용하는 기능이다. 보통 가상머신에서 많이 사용한다. 이 기능은 `cloud-init`와 비슷하게 동작한다.

항목	내용
이름	systemd-repart
역할	디스크 파티션 자동 생성 및 관리 도구
기능	부팅 시점에 지정된 규칙에 따라 디스크 파티션을 동적으로 조정
특징	- systemd에 통합됨 - 기존 파티션 유지/확장 가능 - 파티션 테이블 자동 생성 지원
사용 예시	클라우드 VM, 베어메탈 장비 자동 디스크 초기화 등
도입 시기	systemd v247 (2020년 말쯤)

파티션 재구성

파티션 자동 적용 및 구성은 다음처럼 사용이 가능하다.

1. 부팅 시, 최소 10기가 최대 50기가로 디스크 크기 조정
2. 파일 시스템도 자동으로 크기 조정

```
# vi /etc/repart.d/00-root.conf
[Partition]
Type=linux-root-x86-64
SizeMinBytes=10G
SizeMaxBytes=50G
GrowFileSystem=true
# systemctl enable systemd-repart.service
```

system-sysex

systemd

설명

기존 시스템에 읽기전용 즉, **immutable** O/S와 같은 구성을 하기 위해서 이 기능을 사용한다. 이는 **OverlayFS** 기반으로 동작하며, 확장 모듈을 손쉽게 사용이 가능하다.

항목	내용
이름	systemd-sysex
역할	호스트 시스템에 추가 파일시스템 계층을 일시적으로 오버레이하는 기능
기능	- 읽기 전용 시스템에서도 추가 소프트웨어(예: 개발도구)를 연결(mount)할 수 있음 - 시스템을 오염시키지 않고 확장 가능
특징	- OverlayFS 기반 - 확장 모듈을 /usr/lib/extensions/ 같은 경로에 설치
사용 예시	Immutable OS (예: Fedora Silverblue), 컨테이너 호스트
도입 시기	systemd v249 (2021년 중반쯤)

IMMUTABLE OS 정의

이 기반으로 기존 O/S를 변경하지 않고 immutable OS로 구성이 가능하다.

- 1단계: RootFS를 Read-only로 마운트 설정
- 2단계: systemd-repart로 디스크 자동화 관리 설정
- 3단계: systemd-sysext로 필요한 소프트웨어 확장만 오버레이
- 4단계: 커널 모듈 같은 initramfs 조정 방식으로 포함

간단한 구현

다음처럼 `immutable` 및 확장 모듈을 아래와 같이 적용이 가능하다.

- `/usr/bin/htop` 같은 도구를 읽기 전용으로 시스템에 붙임
- 실제 시스템 파일을 수정하지 않음

```
# mkdir -p /var/lib/extensions/my-tools/usr/bin/  
# cp /usr/bin/htop /var/lib/extensions/my-tools/usr/bin/  
# systemctl enable systemd-sysex  
# systemctl start systemd-sysex  
# systemd-sysex merge
```

간단한 구현

먼저 Root filesystem를 ReadOnly으로 변경한다.

```
# vi /etc/fstab
UUID=xxxx-xxxx / ext4 ro 0 1
# systemctl daemon-reload
# systemctl edit systemd-remount-fs.service
```

간단한 구현

먼저 Root filesystem를 ReadOnly으로 변경한다. 아래와 같이 파티션을 조정하고, 데이터 파티션을 별도로 생성 및 구성한다.

```
# vi /etc/repart.d/00-root.conf
[Partition]
Type=linux-root-x86-64
SizeMinBytes=10G
GrowFileSystem=true
# vi /etc/repart.d/01-data.conf
[Partition]
Type=linux-data
SizeMinBytes=20G
SizeMaxBytes=100G
# systemd-sysext merge
```

간단한 구현

부팅 시 올바르게 동작하기 위해서 **initramfs**를 갱신한다. 그리고 다음과 같은 단계로 올바르게 동작하도록 모듈 및 램 디스크를 갱신한다.

1. 필요한 커널 모듈 복사
2. `/etc/dracut.conf` 수정
 - `add_drivers+="mymodule"`
3. `dracut` 명령어로 새 `initramfs` 생성

```
# vi /etc/dracut.conf
add_drivers+=" mymodule "
# dracut -f --kver $(uname -r)
```


systemd-boot+rec

systemd

설명

boot는 기존 grub2보다 더 가볍게 사용하기 위해서 통합된 부트로더. 이전에는 **gummiboot**라는 이름으로 사용. 현재 **systemd**, **freedesktop.org**에서는 boot로 통합을 권고하고 있다.

항목	설명
UEFI 필수	BIOS 지원 없음 (UEFI only)
빠른 부팅	불필요한 기능 제거, 최소화된 코드
설정 파일 기반	/boot/loader/loader.conf 와 /boot/loader/entries/*.conf 로 부트 메뉴 관리
디스크 별 진입 가능	여러 OS나 커널 버전 부팅 메뉴 쉽게 구성 가능
자동 인식	커널 이미지와 initramfs를 자동 인식해서 메뉴 생성

설명

디렉터리 구조는 기존과 동일하게 호환이 된다. 다만, 몇몇 설정 파일이 다르다.

```
/boot/
├── EFI/
│   ├── BOOT/
│   │   └── BOOTX64.EFI (systemd-boot 자체)
│   └── systemd/
│       └── systemd-bootx64.efi
├── loader/
│   ├── loader.conf
│   └── entries/
│       ├── arch.conf
│       ├── fedora.conf
│       └── rescue.conf
```

부트로더 설정

loader.conf는 다음과 같은 내용이 있다.

```
# vi /boot/loader/loader.conf
default arch
timeout 3
editor no
```

부트로더 설정

entries/arch.conf는 다음과 같은 내용이 있다.

```
# vi /boot/ entries/arch.conf
title    Arch Linux
linux    /vmlinuz-linux
initrd   /initramfs-linux.img
options  root=UUID=xxxx-xxxx-xxxx ro quiet splash
```

부트로더 명령어

올바르게 부팅이 되면, 다음 명령어로 조회 및 설치가 가능하다.

```
# bootctl status
# bootctl install
# bootctl update
# bootctl list
# bootctl remove
# bootctl set-default arch
# bootctl set-oneshot arch
```

부트로더 설정

systemd-rec는 systemd-boot에서 문제가 발생 시 복구 커널로 부팅을 수행한다. 별도로 설치하는 필요 없으며, systemd-boot에 다음과 같이 설정을 추가한다.

```
# vi /boot/loader/entries/recovery.conf
title      Recovery Mode
linux      /vmlinuz-linux
initrd     /initramfs-linux.img
options    root=UUID=xxxx-xxxx ro single
# vi /etc/systemd/boot-recovery.conf
RecoveryBootOptions=recovery
RecoveryMaxAttempts=3
RecoveryDelaySec=30s
# systemctl enable systemd-boot-check-no-failures.service
```

정리

모든 내용을 정리하면 다음과 같다.

구분	bootctl	systemd-bootrec
역할	systemd-boot 설치, 업데이트, 관리	부팅 실패 감지 및 Recovery 부팅 자동 전환
명령어 제공 여부	있음 (bootctl)	없음 (설정 파일 기반)
주요 설정 위치	/boot/loader/, /boot/loader/entries/	/etc/systemd/boot-recovery.conf
직접 수행 기능	부트로더 설치, 제거, 목록 조회, 상태 확인	부팅 실패 감지 후 recovery 엔트리로 자동 전환
사용 명령어 예시	bootctl status, bootctl install, bootctl update	(명령어 직접 없음 → 설정파일과 systemd 서비스 활성화 필요)

정리

모든 내용을 정리하면 다음과 같다.

구분	bootctl	systemd-bootrec
동작 조건	UEFI 기반 시스템 필수	UEFI 기반 + 부팅 실패 감지 시스템 활성화 필요
연동 서비스	bootctl, systemd-boot	systemd-boot-check-no-failures.service
Recovery 지원 여부	간접 지원 (엔트리 등록)	직접 지원 (자동 Recovery 부팅 전환)

systemd-machined-id

systemd

설명

systemd로 넘어 오면서, CPE사양에 맞추어서 machine-id를 제공한다. machine-id는 또한, CPE(Common Platform Enumeration)를 제공한다. 이를 통해서 시스템이나 혹은 소프트웨어에 식별 번호 및 정보를 제공한다.

대표적으로 다음과 같은 정보를 가지고 있다.

1. 운영체제 정보
2. 플랫폼 정보 및 버전

항목	machine-id	CPE
식별 대상	시스템/장비(호스트) 단일 식별자	OS/소프트웨어 제품군 식별자
예시	6b2f1c3be4ef486e967b87e0d3b9d3c5	cpe:/o:canonical:ubuntu:20.04
사용 목적	유니크 시스템 추적	버전/벤더 별 소프트웨어 분류 및 관리

설명

리눅스 한 대(또는 컨테이너/VM)를 다른 모든 시스템과 구분하기 위해 systemd는 128비트 식별자를 `/etc/machine-id` 파일에 보관한다.

이 값은 한 번 정해지면 그대로 유지되며, 부팅 초기에 systemd가 가장 먼저 읽어온다. 이 정보를 적극적으로 많이 사용하는 핵심 systemd 서비스는 `journald`, `NetworkManager`, `systemd-resolved`, `dbus` 등 다양한 서비스의 내부 키로 쓰인다.

호스트 네임이나 IP가 바뀌어도 `machine-id`는 변하지 않는다.

검사 및 검증

확인 및 사용하는 방법은 다음과 같다.

현재 ID 보기

```
# cat /etc/machine-id
```

systemd 도구로 보기(포맷 검증)

```
# systemd-machine-id-setup --print
```

검사 및 검증

다음과 같은 방법으로 재설정이 가능하다.

```
# systemctl stop systemd-logind systemd-networkd NetworkManager 2>/dev/null  
|| true  
# systemctl stop dbus 2>/dev/null || true
```

새 ID 발급

```
# rm -f /etc/machine-id  
# systemd-machine-id-setup
```

dbus 호환 링크 복구

```
# ln -sf /etc/machine-id /var/lib/dbus/machine-id  
# reboot
```

시스템 관리

서비스 관리(systemctl)

SELinux/Audit

디스크 관리(swap, vdo, lvm2)

로그관리

서비스 관리

systemd service

서비스 관리

`systemd`는 서비스 관리를 위해서 `systemctl` 명령어를 통해 관리한다.

이전 `service` 명령어는 여전히 지원하지만, 호환성 목적으로 존재하며, `service` 명령어를 실행하면 내부적으로 `systemctl` 명령어로 재실행된다.

모든 시스템 블록 자원은 `systemd` 로 통합되고 있기 때문에, `systemctl` 명령어에 대한 이해도가 낮으면, 운영 및 장애 처리에 많은 문제가 발생할 수 있다.

또한, `systemctl` 명령어 이외에도 대표적인 `journalctl` 명령어에 대해서도 학습이 필요하다.

서비스 기본 관리 명령어

systemd에서 서비스 관리 시 제일 많이 사용하는 기본 명령어이다.

명령어	설명	예제
status	systemd유닛 상태 확인	<code>systemctl status sshd</code>
restart	유닛 재시작	<code>systemctl restart httpd</code>
start	유닛 시작	<code>systemctl start httpd</code>
stop	유닛 중지	<code>systemctl stop httpd</code>
reload	유닛 메모리 갱신	<code>systemctl reload httpd</code>
is-active	유닛 동작 상태 확인	<code>systemctl is-active httpd</code>
is-enabled	유닛 부트-업 확인	<code>systemctl is-enabled httpd</code>

서비스 관리 확장 명령어

유닛 상태 확인 및 자원 상태에 대해서 확인하는 명령어이다. 밑에 두개 명령어는 예외처리가 들어간 명령어이다.

명령어	설명	예제
<code>list-units</code>	서비스 유닛 목록을 확인	<code>systemctl list-units -t mount</code>
<code>list-automounts</code>	자동 마운트 장치 목록을 확인	<code>systemctl list-automounts</code>
<code>list-unit-files</code>	시스템에 등록된 유닛 파일 목록출력	<code>systemctl list-unit-files</code>
<code>list-sockets</code>	서비스가 사용하는 소켓 목록을 출력	<code>systemctl list-sockets</code>
<code>list-timers</code>	타이머 서비스 목록을 출력	<code>systemctl list-timers</code>

서비스 관리 확장 명령어

유닛 상태 확인 및 자원 상태에 대해서 확인하는 명령어이다. 밑에 두개 명령어는 예외처리가 들어간 명령어이다.

명령어	설명	예제
<code>try-restart</code>	재시작이 가능한지 시도한다.	<code>systemctl try-restart httpd</code>
<code>reload-or-restart</code>	재시작 혹은 메모리 갱신 시도를 한다.	<code>systemctl reload-or-restart vsftpd</code>
<code>try-reload-or-restart</code>	메모리 갱신 혹은 재시작 한다.	<code>systemctl try-reload-or-restart vsftpd</code>

서비스 관리 확장 명령어

아래 명령어는 프로세서 및 네임스페이스 관리 명령어.

명령어	설명	예제
clean	시스템에 기록된 상태 정보를 초기화	<code>systemctl clean sshd</code>
freeze	서비스를 임시로 중지	<code>systemctl freeze httpd</code>
set-property	프로세스의 cgroup에 자원 우선 순위를 선언	<code>systemctl set-property foobar.service CPUWeight=200</code>
bind	특정 파일 및 디렉터리를 네임스페이스 장치할당	
mount-image	컨테이너나 혹은 가상머신 이미지를 네임스페이스 연결	<code>systemctl mount-image -- mkdir bar.service /tmp/img.raw /var/lib/baz/img</code>

서비스 관리 확장 명령어

로깅 및 프로세서가 어떠한 유닛에서 관리되는지 확인하는 명령어. 이 명령어는 systemd 버전에 따라서 동작이 안될 수도 있다.

명령어	설명	예제
<code>service-log-level</code>	특정 서비스에 대해서 로깅 수준을 설정한다.	<code>systemctl service-log-level httpd 2</code>
<code>service-log-target</code>	특정 타겟에 대해서 로깅 수준을 설정한다.	<code>systemctl service-log-target multi-user 2</code>
<code>reset-failed</code>	유닛에 기록된 "실패"기록을 초기화 한다.	<code>systemctl reset-failed sshd.socket</code>
<code>whoami</code>	systemd 254버전부터 지원한다. 어떤 프로세스가 어떤 유닛으로 구성되었는지 확인한다.	<code>systemctl whoami \$(pgrep sshd head -1)</code>

서비스 관리 확장 명령어

명령어	설명	예제
reenable	특정 유닛 혹은 유닛들에 대해서 'disable', 'enable'를 동시에 수행한다.	<code>systemctl reenable sshd</code>
preset	유닛의 기본값을 확인한다.	<code>systemctl preset httpd</code>
preset-all	모든 유닛에 대해서 기본값을 확인한다.	<code>systemctl preset-all</code>
mask	유닛이 동작되지 않도록 마스크 한다.	<code>systemctl mask httpd</code>
unmask	마스크된 유닛을 활성화한다.	<code>systemctl unmask httpd</code>
revert	벤더 기본값으로 유닛 설정을 변경한다.	<code>systemctl revert httpd</code>

서비스 관리 확장 명령어

명령어	설명	예제
<code>edit</code>	유닛 파일을 수정(nano)	<code>systemctl edit httpd</code>
<code>list-machines</code>	시스템에 구성된 컨테이너 머신확인	<code>systemctl list-machines</code>
<code>is-system-running</code>	호스트 컴퓨터의 상태를 확인	<code>systemctl is-system-running</code>

SELINUX/AUDIT

시스템

SELINUX

SELinux는 미국 NSA에서 제작 후, 오픈소스 커뮤니티에 기여. 정확히는 레드햇이 해당 소스코드를 받았으며, 이 코드 기반으로 커널 기반의 MAC보안 시스템을 구성하였음. 기존 리눅스 시스템은 DAC만 지원 및 구성하였기 때문에, 미국 NIST기준에 맞지 않았다.

- **DAC:** Discretionary Access Control
- **MAC:** Mandatory Access Control

대다수 리눅스 시스템은 MAC 둘 중 하나를 사용하고 있다.

SELINUX

AppArmor

상대적으로 SELinux보다 사용하기 쉬우며, 대다수 GNU/Linux 배포판은 이를 채택하고 있다. 레드햇 계열 배포판은 AppArmor 사용이 어렵다.

SELinux

레드햇 계열 및 컨테이너 시스템에서 많이 채용하고 있다. 커널 빌트인 기반으로 동작하기 때문에 사용이 복잡하다.

SELINUX

SELinux 사용 상태 확인.

1: selinux 일시적으로 사용

0: selinux 일시적으로 중지

일시적으로 SELinux 사용 상태를 중지 혹은 사용으로 변경.

```
# getenforce
```

```
Enforcing
```

```
# setenforce 1
```

SELINUX

부팅 시, SELinux적용 상태를 변경하기 위해서는 아래를 수정 혹은 명령어를 실행한다.

```
# vi /etc/selinux/config
SELINUX=enforcing
SELINUXTYPE=targeted
# grubby --update-kernel ALL --args selinux=1
# grubby --update-kernel ALL --remove-args selinux
```

SELINUX

모드/정책	설명	사용 예제
enforcing	강제로 SELinux 정책을 적용한다.	setenforce 1
permissive	감사만 수행하며, 정책은 실제로 적용하지 않는다.	setenforce 0
targeted	프로세스 중심으로 정책을 적용한다.	/etc/selinux/config에서 SELINU XTYPE=targeted 설정
mls	다중 계층 보안(Multi-Level Security)으로, 등급별로 접근을 분리한다.	/etc/selinux/config에서 SELINU XTYPE=mls 설정
minimum	특정 프로세스만 검사하며, 일반적으로 컨테이너 시스 템에 권장한다.	/etc/selinux/config에서 SELINU XTYPE=minimum 설정

SEMANAGE

SELinux에서 사용하는 모든 컨텍스트에 대해서 관리 및 수정이 가능.

- 일반적으로 보통 -l 옵션은 list이다. -C 옵션은 Customized 옵션
- 사용자가 수정하거나 혹은 변경한 부분만 출력한다. -a는 add

```
# semanage fcontext -l | grep httpd
# semanage fcontext -a -t httpd_sys_content_t '/srv/htdocs(/.*)?'
# semanage port -l
# semanage port -a -t http_port_t -p tcp 81
# semanage boolean -m --on httpd_can_sendmail
# semanage boolean -l -C
```

BOOLEAN

프로그램에서 사용하는 기능을 허용 및 제한하는 기능이다. 프로그램에 기능이 활성화 되어도, Boolean 으로 차단이 되면, 올바르게 사용이 불가능 하다.

`getsebool`: 프로그램에서 사용하는 특정 기능(콜) 목록을 확인.

```
# getsebool xdm_write_home  
xdm_write_home → off
```


BOOLEAN

특정 기능을 사용 혹은 미사용 할지 수정 명령어. 이 명령어는 프로그램의 시스템 콜을 제한 및 제어한다.

```
# setsebool -P xdm_write_home=1  
# setsebool xdm_write_home=1
```

변경된 내용에 대해서 확인하기 위해서는 다음과 같이 실행한다.

```
# semanage boolean -lC
```

-l: list

-C: Customized

컨테이너+SELINUX



컨테이너+SELINUX

대표 예시

OpenShift (Red Hat의 Kubernetes 배포판)에서는 컨테이너 별로 minimum policy를 적용해서, 컨테이너 내부는 강제 검사, 호스트와 다른 프로세스는 영향을 받지 않게 설정.

특히, 레드햇 오픈 시프트는 4.x 부터는 다음과 같이 한다.

컴퓨터 노드의 컨테이너 런타임(crio, containerd)은 minimum policy를 적용해서, 컨테이너 내부 행위만 SELinux로 감시하고, host 시스템 자체는 targeted 정책으로 운영한다.

컨테이너+SELINUX

구성 요소	적용 SELinux 정책	역할
Host 시스템	targeted	시스템 전반 보호
Podman 데몬	targeted 정책을 따름	컨테이너 생성 시 라벨 부여
컨테이너 내부	minimum	컨테이너 내부 행위만 검사

컨테이너+SELINUX

[Host System]

|

└─ SELinux Policy: targeted

|

▼

[Podman 실행]

|

└─ 컨테이너 생성 시 SELinux Label 부여

|

(예: system_u:system_r:container_t:s0)

▼

컨테이너+SELINUX

[컨테이너 내부]

|
└─ SELinux Policy: minimum
|
▼

[컨테이너 프로세스 접근 제어]

|
└─ 파일/디바이스 접근 시 SELinux 확인
└─ 위반 시 AVC Deny 기록 (journalctl, audit.log)

포드만 적용 예제

포드만에 적용하면 다음과 같이 적용이 된다.

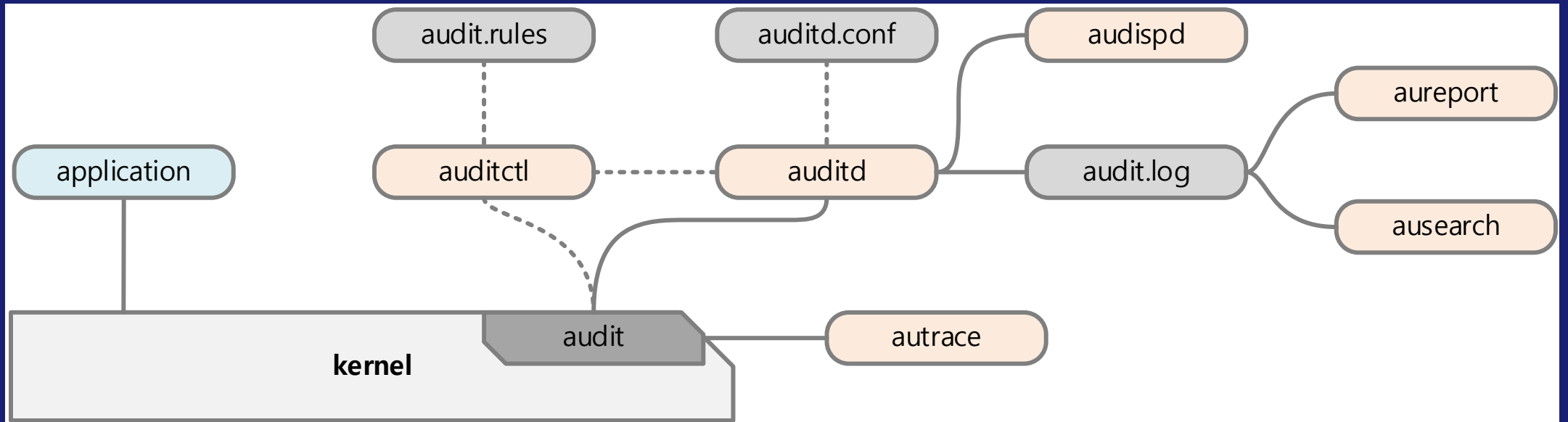
`container_t` 타입으로 컨테이너가 라벨링 된다. Host의 SELinux Policy(targeted)에 따라, 컨테이너 내부는 minimum 정책으로 감시된다.

SELinux 옵션을 추가해서 Podman으로 컨테이너 실행

```
# podman run --security-opt label=type:container_t -d nginx
```

AUDIT

auditd는 시스템에서 발생하는 시스템 콜, 사용자, 파일 및 디렉터리 같은 자원에 대해서 감사한다.



AUDIT

특정 프로그램에서 실행하는 모든 콜에 대해서 등록 후 확인한다.

```
# auditctl -a exit,always -S all -F pid=1001
```

특정사용자가 파일 접근(open)에 대해서 등록 후 확인한다.

```
# auditctl -a exit,always -S open -F auid=1001
```

성공적으로 파일을 접근한 콜에 대해서 기록한다.

```
# auditctl -a exit,always -S open -F success=0
```

AUDIT

옵션	설명
-a	콜 액션, exit, always 프로그램 종료, 사용 중일때 콜 기록을 남긴다.
-S	콜 이름. "open" 경우에는 파일에 읽기 접근 시 기록을 남긴다. 모든 콜에 대해서 기록이 필요한 경우 "all"로 한다.
-F	조건 필터. 명시한 조건에 따라서 기록을 남긴다.
exit, always	종료 및 모든 콜 이벤트 기록.

AUDIT

특정 파일에 변경사항 확인하기.

```
# auditctl -w /etc/shadow -p wa  
# auditctl -a exit,always -F path=/etc/shadow -F perm=wa
```

디렉터리 퍼미션 확인 및 감사.

```
# auditctl -w /etc/ -p wa  
# auditctl -a exit,always -F dir=/etc/ -F perm=wa
```

AUDIT

옵션	설명
-w	감사할 대상자원을 명시한다. 보통 파일이나 디렉터리.
-p	퍼미션. "rwx"로 구별해서 적는다. "a"는 "attribute"이다.
-F	조건 필터. 명시한 조건에 따라서 기록을 남긴다.
wa	쓰기 및 속성 기록.

AUDIT(search/report)

발생한 이벤트에 대해서 검색 및 확인하기 위해서 다음과 같이 조회가 가능하다. 또한, 파일 접근에 대한 보고서 생성도 아래 명령어로 가능하다.

```
# ausearch --pid $(pgrep sshd | head -1)
# ausearch -ua 1000 -i
# ausearch --start yesterday --end now -m SYSCALL -sv no -i
# aureport --start 03/20/2024 00:00:00 --end 03/21/2013 00:00:00
# aureport -x
# aureport -x --summary
# aureport -u --failed --summary -i
# aureport --login --summary -i
# ausearch --start today --loginuid 1000 --raw | aureport -f --summary
# aureport -t
```

디스크 관리

일반정보

LVM2/STRATIS/VDO

디스크 관리 명령어

lsblk

블록 장치 목록을 확인하는 명령어. -o 옵션을 통해서 장치의 모델명 및 마운트 정보 확인이 가능하다.

```
# lsblk -o PATH,SIZE,RO,TYPE,MOUNTPOINT,UUID,MODEL
```

blkid

슈퍼 블록에서 생성된 UUID 정보를 확인한다.

```
# blkid -i /dev/sda
```

fdisk

MBR 형태로 디스크에 파티션을 생성한다. 프라이머리는 최대 4개 까지 생성, 확장 파티션을 운영체제 별로 다르기는 하지만 보통 32개까지 가능하다. x86에서는 더 이상 MBR 형태로 파티션 구성은 권장하지 않는다.

blkid

슈퍼 블록에 기록이 되어 있는 파티션 혹은 파티션 정보를 읽어와서 UUID정보를 출력한다.

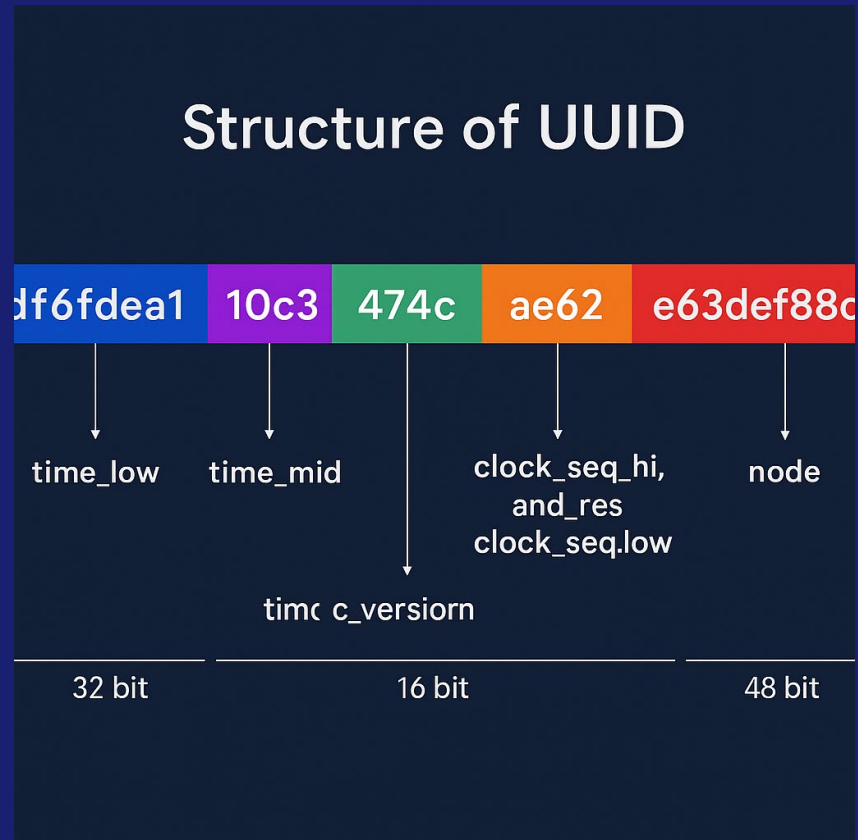
여기서 말하는 UUID는 Universal Unique Identifier의 약자이며, 128비트로 구성이 되어 있는 정보이다. 참고로 UUID는 GUID(Global Unique Identifier)에서 유래가 되었다.

Version 5 UUID는 이름 기반(Namespace+SHA1) UUID로, 주로 "특정 값"을 기반으로 생성할 때 사용.

슈퍼 블록에는 파일시스템 UUID가 기록되어 있으며, UUID는 128비트로 구성된 고유 식별자이다. UUID는 GUID에서 유래되었고, 리눅스에서는 주로 version 1 또는 version 4 UUID가 사용된다.

<https://github.com/torvalds/linux/blob/master/lib/uuid.c>

UUID 구조



파티션 및 디스크 정보

PARTUUID

part-uuid는 시스템에서 사용하고 있는 디스크 혹은 드라이브의 UUID. 이 정보는 DM(Device Mapper)로 생성이 되며, 보통 /dev/disk에서 소프트 링크로 구성이 되어 있다.

DISKUUID

disk-uuid는 일반적으로 /etc/fstab에 등록된 내용. 보통 특정 파티션에 링크가 되어 있다. 위의 심볼릭 링크는 /dev/disk에서 확인이 가능하다.

참고로 위의 partuuid, diskuuid와 같은 정보들은 /dev/disk/by-*에 형식별 장치 이름이 구성되어 있다. 이 정보들은 또한 명령어로 확인이 가능하다.

```
# udevadm info /dev/sdb
# dmsetup ls
```

디스크 관리 명령어

gdisk

EFI(GPT)형태로 디스크에 파티션을 생성한다. 최대 128개까지 생성이 가능하다. 현재 대다수 리눅스는 GPT기반으로 구성한다. 'gdisk'명령어는 CLI모드를 지원하지 않는다.

partprobe

추가된 디스크를 커널의 비트맵(bitmap)에 갱신한다. 다만, 사용 중에 갱신하는 경우, 잠깐 I/O가 중지가 될 수 있다.

```
# partprobe -d -s /dev/sdb  
# partprobe /dev/sdb
```

파티션 관리(로컬 디스크)

partx

partx는 **kpartx**와 비슷하지만, **partx**는 일반 블록장치에 사용한다. **partprobe**는 커널에 모든 블록장치 영역을 조회하지만, **partx**는 특정 블록 장치 및 파티션만 커널 메모리에서 수정한다. 또한, **partx**는 이미지 디스크 관리도 지원한다.

```
# partx --show - /dev/sdb1
# partx --add --nr 3:5 /dev/sdd      ## 파티션 3에서 5번까지 메모리에서 추가
# partx --delete --nr :-1 /dev/sdd  ## 맨 마지막 파티션만 메모리에서 삭제
# partx --add /dev/sdd               ## 모든 디스크의 파티션 정보를 메모리에 추가
```

파티션 관리(디바이스 매퍼)

kpartx

단일 디스크 혹은 파티션을 추가 및 삭제하는 명령어.

보통 멀티패스로 구성된 디스크를 디바이스 매퍼(devicemapper)에 추가 시 사용하며, 일반 디스크 추가에는 사용하지 않는다. 다만, 인식에 문제가 있으면 `partprobe` 명령어를 같이 사용한다.

```
# kpartx -av /dev/mapper/mpathb
```

멀티패스로 올바르게 인식이 되지 않으면, 보통 다음과 같이 명령어를 수행한다.

```
# kpartx -pp /dev/mapper/mpathb  
# partprobe /dev/mapper/mapthb  
# kpartx -av /dev/dm-7
```

파티션 관리

parted

CLI기반으로 디스크의 파티션 관리를 한다. 기존의 `fdisk`, `gdisk`는 대화형이기 때문에 자동화하기에는 적절하지 않는 도구이다.

```
# parted /dev/mapper/mpathb print
# parted /dev/mapper/mpathb mklabel gpt
# parted /dev/mapper/mpathb1 --align opt mkpart data 1 100%
```

`xf`s, `ext4`로 50:50으로 구성하기 위해서는 다음과 같이 명령어를 수행한다.

```
# parted /dev/sdb --align opt mkpart xfs 1 50%
# parted /dev/sdb --align opt mkpart ext4 51% 100%
```

파티션 관리

parted

파티션에 이름을 설정 혹은 출력하기 위해서 다음처럼 명령어를 실행한다.

```
# parted /dev/sdb name 1 data-part  
# parted /dev/sdb print
```

파티션 관리

sfdisk

여러 디스크 정보 확인이나 혹은 조작하기 위해서 사용하는 명령어. 보통은 백업이나 정보 확인 시, 사용한다.

```
# sfdisk -Z /dev/sdb
# sfdisk -n 0:0:+200M -t 0:ef02 -c 0:"bios_boot" /dev/sdb
# sfdisk -n 0:0:+1G -t 0:8300 -c 0:"linux_boot" /dev/sdb
# sfdisk -n 0:0:+1G -t 0:8200 -c 0:"linux_swap" /dev/sdb
# sfdisk -p /dev/sdb
```


파티션 정보 백업

sfdisk

혹은 파티션 편집하기 전에 복구를 위해서 다음처럼 실행하여 백업이 가능하다.

```
# sfdisk -d /dev/sdb > sdb.backup
# sfdisk -f /dev/sdb < sdb.backup
# sgdisk --backup=/tmp/gpt.backup /dev/sdb
# sgdisk --load-backup=/tmp/gtp.backup
```

파티션 및 디스크 정보

cgdisk

TUI기반으로 사용이 가능한 텍스트 에디터. 이 도구는 "EFI", "MBR" 둘 다 지원한다.

hwinfo

하드웨어 정보를 확인하는 명령어. 블록 장치 정보 확인을 원하는 경우 다음과 같다.

```
# hwinfo --block --short
```

부팅 디스크(aka ramdisk) 구성

리눅스 배포판은 아직까지 `/etc/fstab` 혹은 커널모듈 변경, 관리 및 연결할 블록 장치, 파티션을 관리한다.

다만, `systemd`로 변경이 되면서, **`fstab`** 정보는 램-디스크(ramdisk)에 저장되기 때문에, 꼭 램 디스크를 갱신해야 한다.

가급적이면, 램 디스크는 전부 갱신하는 게 제일 안전하다.

```
# dracut --list
# dracut -m systemd-initrd --force
# lsinitrd -m
# lsinitrd
# dracut -f
# systemctl reload
```

systemd 블록 장치 관리

systemd에서 마운트 정보는 다음처럼 확인이 가능하다.

```
# systemd-mount --list  
# systemd-mount -u
```

앞으로 systemd에서는 `/etc/fstab`를 사용하지 않고, `systemd-mount`를 통해서 시스템 블록 장치를 연결 및 구성을 한다. 이 자원은 이미 사용하고 있으며, 추후에는 더 이상 `/etc/fstab`를 사용하지 않는다.

```
# systemctl list-unit-files --type mount
```

기존에 사용하고 있는 `/etc/fstab`의 정보는 `systemd-fstab-generator`를 통해서 유닛 파일을 생성 및 관리한다.

디스크 이름 명명

리눅스는 다음과 같은 디스크 네이밍 규칙을 가지고 있다.

디스크 종류	디바이스 이름 규칙
가상 디스크 (Virtual Disk)	/dev/vd?
USB / SCSI / 외장 디스크	/dev/sd?
IDE 디스크	/dev/hd?
CD-ROM 드라이브	/dev/sr0

디스크 이름 명명

파티션은 보통 다음과 같은 네이밍 규칙을 따른다.

`/dev/sd[a-z][1-15]`

1. 리눅스의 SCSI 디스크는 하나의 디스크에 대해 최대 16개의 파티션 번호(minor number)를 가질 수 있다.
2. 리눅스는 디스크와 장치를 /dev 아래에서 구분하여 네이밍하며, 디스크당 최대 16개 파티션을 지원한다.

메이저(major) 번호도 디바이스 클래스에 따라 할당되며, SCSI 디스크 장치(예: /dev/sd*)의 경우 메이저 넘버는 보통 8번이 고정되어 있다.

일반적으로 디스크 장치 번호는 0부터 시작하며, 파티션 번호는 1부터 시작하는 것이 일반적이다.

디스크 이름 생성

리눅스 배포판마다 블록 장치 생성 및 관리 개수는 조금씩 다르지만, 일반적으로 약 128개 정도의 장치를 미리 생성하는 경우가 많다.

SUSE(SLES, openSUSE)나 다른 몇몇 배포판은 초기 설정 시 16개 또는 30개 정도만 장치를 생성하기도 한다. 필요에 따라, `mknod`, `MAKEDEV`(또는 `makedev`) 같은 명령어를 통해 추가적인 장치를 수동으로 생성할 수 있다.

커널이 인식한 블록 장치 정보는 다음 경로에서 확인 가능하다.

- `/proc/partitions`: 커널이 관리하는 디스크와 파티션 리스트
- `/dev/disk/`: UUID, 파티션 LABEL, 경로별로 디바이스를 정리해서 제공

디스크 관련 커널 파라미터는 다음 디렉터리 아래에서 조정할 수 있다:

- `/sys/class/scsi_host/`: SCSI 호스트 어댑터 설정
- `/sys/block/`: 블록 장치의 속성 관리 (큐 깊이, 스케줄러 등)

udev

사용자 영역(userspace)에서 동작하는 소프트웨어이다. /dev/에 등록되어 있는 장치는 사용자가 사용하기 위해서는 장치를 확인 후, 블록 장치를 파일 시스템에 마운트 한다.

하지만, 점점 많은 장치가 시스템에 연결 및 구성이 되면서 시스템 사용자가 사용에 불편함을 느꼈으며, 이를 해결하기 위해서 udev를 구성하였다.

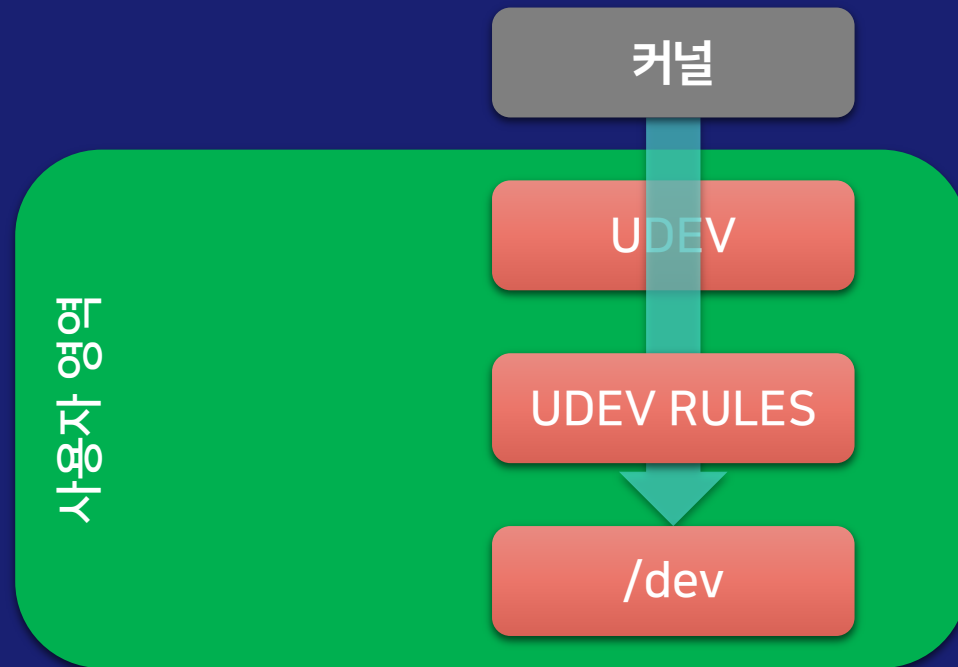
현재 udev는 systemd 블록에 통합이 되어서 모든 대다수 리눅스 배포판은 udev기반으로 구성한다. udev의 최대 장점은 사용자가 규칙(rule)기반으로 손쉽게 작성 및 구성이 가능하다.

```
# udevadm info /sys/class/net/eth0
```

이 기능은 커널 2.5에 도입이 되었으며, 완성형은 2.6버전이 넘어가면서 완전히 커널 및 시스템과 통합이 이루어 졌다. udev는 하드웨어 정보가 필요하기 때문에 hwdb 파일 기반으로 하드웨어 정보 및 장치 이름을 결정한다.

```
# /usr/bin/systemd-hwdb query mouse:*:name:*Trackball*:*
```


udev



커널 블록

커널 블록장치는 다음과 같이 지원 및 제공한다.

그룹	예시 파일 시스템	설명
Block-based FS	ext2, ext4, xfs, btrfs, ntfs3, squashfs, iso9660	하드 디스크/SSD 같은 블록 장치 기반 파일 시스템
Network FS	nfs, smbfs, ceph, coda 등	네트워크를 통해 접근하는 파일 시스템
Pseudo FS	proc, sysfs, pipefs, futexfs 등	물리적 저장소가 없는 시스템 가상 파일 시스템 (정보/인터페이스 제공)
Special Purpose FS	tmpfs, ramfs, devtmpfs 등	메모리 기반 파일 시스템, 디바이스 관리
Raw Flash FS	ubifs, jffs2 등	NAND/NOR Flash 메모리용 파일 시스템
Stackable FS	ecryptfs, overlayfs 등	기존 파일 시스템 위에 덧붙여 동작하는 계층형 파일 시스템

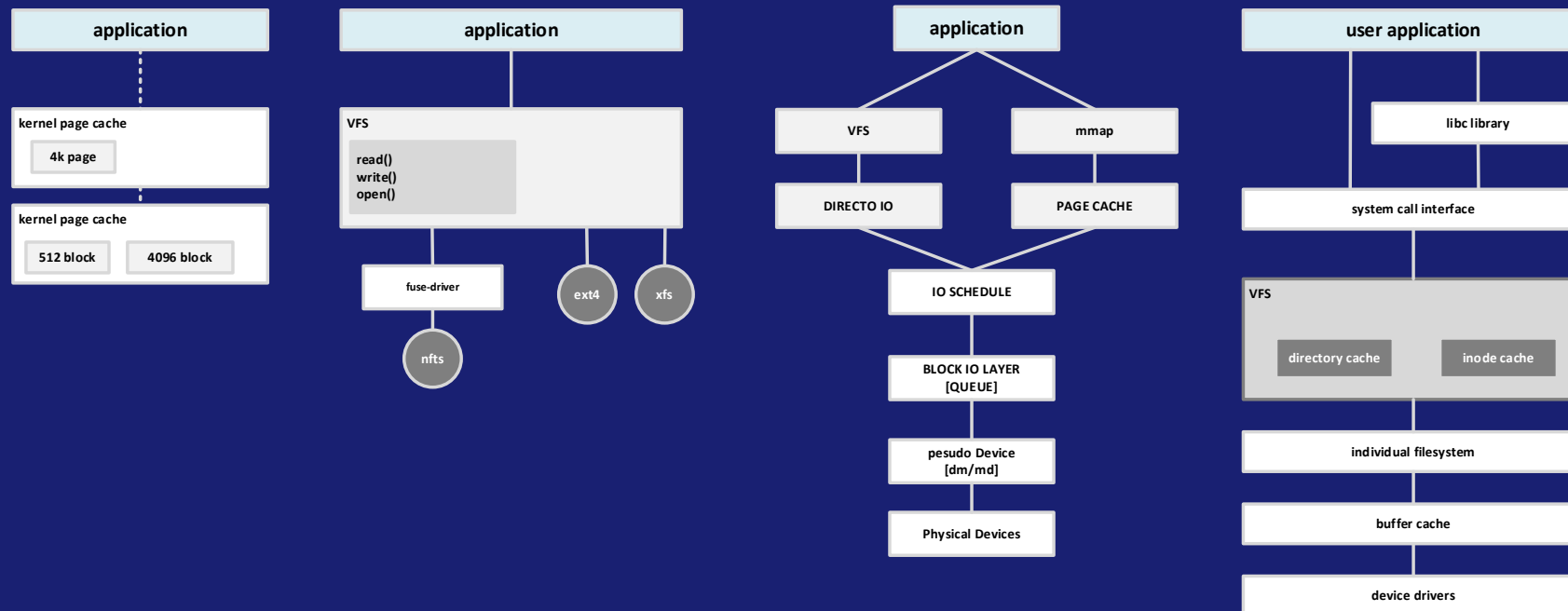
커널 블록

부과기능은 다음과 같다.

항목	역할
fscache	네트워크 파일 시스템에 대한 캐싱을 담당
fuse	Filesystem in Userspace, 사용자 공간에서 파일 시스템을 구현 가능
Page Cache	파일 시스템 I/O를 빠르게 처리하기 위해 메모리에 캐시하는 구조
Direct I/O (O_DIRECT)	캐시를 우회하고 디스크에 직접 접근하는 방식

VFS

VFS는 **Virtual File System**의 약자이다. 가상 파일 시스템이 아니라, 여러가지의 파일 시스템을 통일된 레이어에서 제공하기 때문에 사용자가 파일 시스템 명시가 필요 없이 접근이 가능하다. 또한, 메모리 버퍼 기능을 제공하기 때문에 이를 통해서 성능 향상도 가능하다.



디바이스 매퍼

커널에서 인식된 장치를 직접적으로 사용하는 경우, 장치 이름을 손쉽게 확인이 어렵지 않다. 또한, 많은 장치를 추가하는 경우 관리하기가 어려운 부분이 있다.

그래서 **고수준 레벨(Higher Level Block Device Manager)**로 장치를 관리할 수 있다. 보통 Devicemapper는 DM이라고 부르기도 하며, LVM2/VD0/Stratis에서 블록장치 백-엔드 관리자로 사용한다.

디바이스 매퍼는 소프트웨어 계층과 연결이 가능하기 때문에, 디스크 암호가 필요한 경우, DM를 통해서 디스크 전체나 혹은 파티션을 암호화하여 사용이 가능하다.

VFS(Virtual File System)는 Device Mapper레이어 하위에서 통합적으로 장치 접근을 처리한다.

디바이스 매퍼



devicemapper function

기능	설명
cache	캐시 디스크(SSD)와 데이터 디스크(HDD)를 조합하여 하이브리드 디스크를 구성한다.
clone	디스크 복제 또는 복원 용도로 사용하며, 주로 스냅샷 기반 디스크 복제에 활용된다.
crypt	리눅스 커널의 Crypto API를 사용하여 블록 장치를 암호화한다.
delay	장치별 읽기/쓰기 지연을 인위적으로 설정한다. (테스트용)
error	일부러 블록 장치에 I/O 오류를 발생시킨다. (테스트용)
linear	DM을 통해 여러 블록 장치를 선형으로 연결하여 하나의 논리 디바이스를 구성한다.
mirror	맵핑을 통해 블록 장치를 미러링하여 데이터 이중화를 구성한다.

devicemapper function

기능	설명
multipath	다중 경로(multipath) 장치를 그룹화하고 맵핑하여고가용성 및 로드밸런싱을 지원한다.
raid	리눅스 소프트웨어 RAID 구성을 위한 장치 맵핑(dm-raid)을 제공한다.
snapshot	LVM2 기반으로 COW 방식 스냅샷을 생성하고 관리한다.
striped	청크 크기를 기준으로 데이터를 여러 장치에 분산하여 저장한다.
thin	실제 쓰기 발생 시에만 블록을 할당하는 Thin Provisioning 기능을 제공한다.
zero	/dev/zero와 유사하게 빈 데이터(0)를 쓰거나, 쓰기를 생략한다.

devicemapper application

기능	설명
cryptsetup	디스크 및 파티션 암호화가 필요한 경우, dm-crypt를 통해 암호화한다.
dm-crypt/LUKS	암호화된 장치를 LUKS 포맷을 통해 관리하고 구성한다.
dm-cache	하이브리드 볼륨 구성 시, SSD+HDD 조합을 통해 캐시 기능을 제공한다.
dm-integrity	LUKS 또는 RAID 장치 무결성 검증을 위해 사용한다.
dm-log-writes	Device Mapper 장치의 쓰기 기록을 저장하고 분석할 수 있도록 한다.
dm-verity	읽기 전용 블록 장치의 무결성을 검증할 때 사용한다.
dmraid(8)	BIOS/펌웨어 기반 Fake RAID 구성을 위해 Device Mapper를 사용한다.
DM Multipath	다중 경로 I/O를 통해 로드 밸런싱 및 장애 복구를 지원한다.

devicemapper application

기능	설명
Docker/Podman	COW 기반으로 컨테이너 이미지 및 데이터를 관리한다. (일반적으로 overlayfs 사용)
DRBD	노드 간 블록 레벨 복제를 지원하는 분산 블록 장치 솔루션이다. (Ceph과는 별도)
EVMS(deprecated)	현재는 더 이상 사용되지 않는다. (LVM2가 대체)
kpartx(8)	Device Mapper 장치에 파티션 테이블을 읽어 추가하거나 제거한다.
LVM2	논리 볼륨(LV)을 Device Mapper를 통해 관리한다.
VDO	Virtual Data Optimizer를 통해 중복 제거, 압축, 씬 프로비저닝 기능을 제공한다. (LVM2 기반 확장 가능)

스왑

GENERAL SWAP

ZRAM SWAP

다중 스왑 및 파일 스왑

리눅스에서 사용하는 스왑(SWAP)은 보통 2가지 형태로 사용한다.

- 파티션 형태의 raw block
- 파일 형태의 file block

대다수 리눅스 배포판은 raw block 형태를 선호한다. 하지만, 특정 상황에서 file block 형태를 사용해야 하는 경우도 있다. 최근, 레드햇 계열 배포판은 기존에서 사용하던 일반 스왑(block swap)에서, 메모리 스왑(zswap)으로 변경하고 있다.

일반 블록 스왑 구성은 대다수 엔지니어가 알고 있기 때문에, 해당 부분은 랩에서 다루지 않는다.

파일기반 스왑 구성

file block형태를 사용하기 위해서는 다음과 같이 사용이 가능하다.

```
# dd if=/dev/zero of=/tmp/temp_swap.img bs=1G count=1
# mkswap /tmp/temp_swap.img
# swapon /tmp/temp_swap.img
# swapon -s
```

파일기반 다중 스왑

여러 개의 블록 스왑을 사용하는 경우, 아래와 같이 "/etc/fstab"에 설정 및 구성한다.

```
# dd if=/dev/zero of=/var/spool/temp_swap.dat bs=1G count=1
# mkswap /var/spool/temp_swap.dat
# swapon /var/spool/temp_swap.dat
# vi /etc/fstab
/dev/sdd2 swap swap defaults,pri=10 0 0
/root/temp_swap.dat none swap defaults,pri=20 0 0
```

ZSWAP(일반)

메모리 형태의 스왑을 사용하고 싶은 경우, RHEL 및 CentOS 8에서는 **zswap** 사용이 가능하다.

다음과 같은 명령어로 사용이 가능하다. **systemd** 기반에서 다음과 같이 **zram** 생성 및 구성한다.

<https://www.kernel.org/doc/html/v5.9/admin-guide/blockdev/zram.html>

zram를 사용하기 위해서는 반드시 사용하는 디스크는 메모리 형태 디스크이어야 한다.

```
# dnf install zram-generator
# cp /usr/share/doc/zram-generator/zram-generator.conf.example
/etc/systemd/zram-generator.conf
# systemctl enable --now systemd-zram-setup@zram0.service
```

ZSWAP(특정위치)

혹은 특정 위치에 zram 구성을 원하는 경우, 아래와 같이 생성 및 구성이 가능하다.

```
# vi /etc/systemd/zram-generator.conf
[zram1]
zram-size=ram/2
mount-point=/var/compressed
options=X-mount.mode=1777
```


ZSWAP(확인)

zswap는 기존에 사용한 스왑과 같이 사용이 가능하다. 다만, 권장은 동시 사용보다 하나의 스왑 형식을 사용하는 걸 권장한다. 이유는 zswap은 메모리를 사용하기 때문에 오버헤드(overhead)가 낮으며, 일반 스왑은 블록장치를 사용하기 때문에 오버헤드가 높다.

```
# swapon -s
```

Filename	Type	Size	Used	Priority
/dev/dm-1	partition	4141052	0	-2

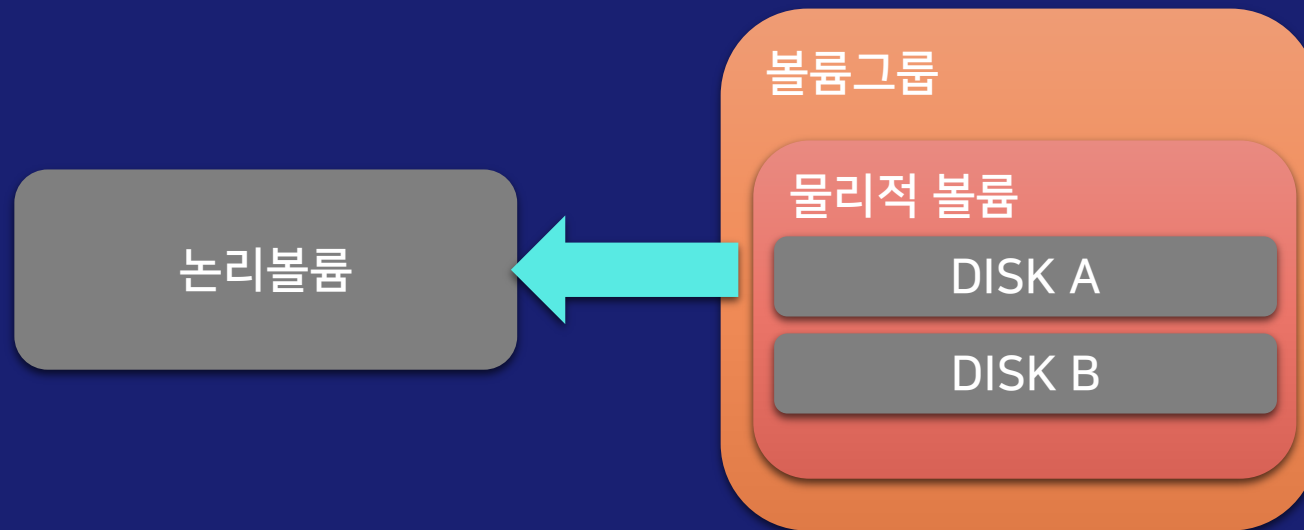
```
# zramctl /dev/zram0
```

NAME	ALGORITHM	DISKSIZE	DATA	COMPR	TOTAL	STREAMS	MOUNTPPOINT
/dev/zram0	lzo-rle	1.8G	652K	11.2K	84K	4	/var/compressed

LVM2

VOLUME MANAGER

LVM2



LVM2 명령어

이전에는 LVM2에서 많이 사용하던 명령어는 다음과 같다. 아래 명령어는 독립적인 명령어가 아니라, 배시 셸에서 함수로 구현한 기능이며, 이들은 보통 심볼릭 링크로 구성되어 있다.

```
pvcreate(/usr/sbin/pvcreate: symbolic link to lvm)
vgcreate(/usr/sbin/vgcreate: symbolic link to lvm)
lvcreate(/usr/sbin/lvcreate: symbolic link to lvm)
vgchange(/usr/sbin/vgcreate: symbolic link to lvm)
```

하지만, 위의 명령어로 LVM2 디스크를 완벽하게 관리할 수가 없기 때문에 다음과 같은 명령어로 관리 방법도 고려해야 한다. 만약, `lvmdevices` 명령어를 사용한다면, `lvm` 명령어로 다음처럼 관리가 가능하다.

```
# lvm lvmdevices
# lvm pvcreate
```

LVM2 볼륨 및 논리 장치 생성

기본적인 LVM 그룹 및 논리적 장치 생성. 아래 명령어로 생성이 가능하다. 시작 전, **hexedit**를 설치한다.

```
# fdisk /dev/sdb  
# pvcreate /dev/sdb1
```

VG에 1기가 전부 할당.

```
# vgcreate /dev/sdc test-vg
```

LVM2 볼륨 및 논리 장치 생성

논리적 디스크 생성.

```
# lvcreate -n test-lv -l 100%Free test-vg
# mkfs.xfs /dev/test-vg/test-lv
# mkdir -p /mnt/test-lv
# mount /dev/test-vg/test-lv /mnt/test-lv
```

LVM2 볼륨그룹 확장

1. 기존에 만든 "test-vg"공간을 확장한다. 먼저 "500MiB" 크기의 파티션을 추가한다.
2. 추가로 만든 파티션 혹은 디스크를 test-vg에 추가한다.

```
# fdisk /dev/sdb2  
# vgextend test-vg /dev/sdb2
```

LVM2 볼륨그룹 확장

확장된 볼륨 크기만큼 논리 디스크를 확장한다.

- **-r**: Resize to FS는 자동으로 수행한다. 예를 들어서 ext4는 `resize2fs`, xfs는 `xfs_growfs` 명령어를 사용한다.
- **-l/-L**: 두 개의 크기 옵션을 동시에 사용할 수 없다. 둘 중 하나만 사용해야 한다.
 - `-(extent size)`, `-L(Unit size)`.
- 확장 시, 반드시 앞에 기호 플러스를 붙여야 한다. 마이너스 기호도 사용이 가능하지만, 몇몇 파일 시스템 제외하고 축소는 지원하지 않는다.

```
# vgextend test-vg /dev/sdb2
```

```
# lvextend -r -l [PE_NUMBER] -L [UNIT_SIZE] /dev/test-vg/test-lv
```


LVM2 BACKUP

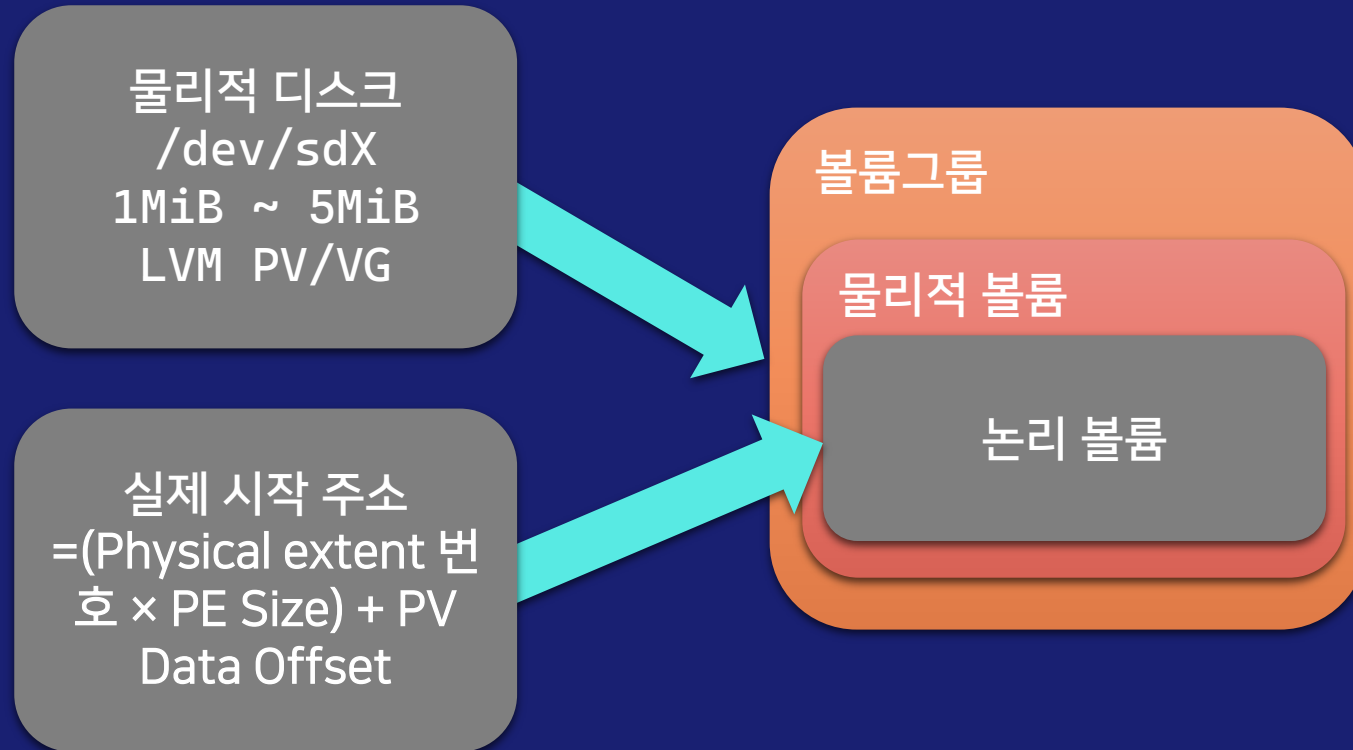
LVM2 백업은 lvm명령어를 실행하면 자동적으로 생성이 된다. 다만, 복구 부분은 사용자가 직접 해야 한다. 복구를 하기 위해서 다음과 같이 명령어로 조회 및 복구를 하면 된다.

단, 복구 명령어를 실행하면, /etc/lvm/backup에 저장된 내용을 블록 장치의 블록 영역에 다시 덮어쓰우기를 진행한다.

백업되는 영역은 Volume Group만 백업이 된다. Physical Volume영역은 x86에서는 그렇게 중요하지 않다.

```
# vgcfgbackup
# vgcfgrestore
# vgcfgrestore testvg -l
  File:                /etc/lvm/archive/testvg_00000-1010607222.vg/testvg_00000-
1010607222.vg
  VG name:              testvg
```

LVM2 METADATA 다이어그램



LVM2 METADATA 설명

LVM2 메타 정보는 물리적 디스크(블록 디바이스)에 저장된다. 본래 LVM 시스템은 IBM AIX 시스템에서 넘어온 기술이다. 초기 AIX 환경에서는 하드웨어 펌웨어나 전용 파티션 영역에 LVM 메타데이터를 저장하는 방식을 사용했다.

그러나, x86 기반 리눅스 시스템에서는 이런 펌웨어 기반 저장 방식이 불가능했기 때문에, LVM 메타 정보는 블록 디바이스의 특정 영역에 저장되도록 설계되었다.

구체적으로 다음과 같다.

1. Physical Volume (PV) 생성 시
2. 디스크 상에 LVM 헤더와 메타데이터 블록이 기록된다.
3. 이 메타 정보에는 Volume Group(VG), Logical Volume(LV) 등의 정보가 포함된다.

LVM2 METADATA 확인

확인은 다음과 같이 가능하다.

```
# pvcreate /dev/sdb
# hexedit -r /dev/sdb
000001F8  00 00 00 00 00 00 00 00 4C 41 42 45 4C 4F 4E 45 01 00 00 00
00 00 00 00 .....LABELONE.....
00000210  73 21 33 53 20 00 00 00 4C 56 4D 32 20 30 30 31 67 58 70 45
55 31 37 45 s!3S ... LVM2 001gXpEU17E
00000228  34 67 5A 67 6C 39 7A 6D 72 74 61 4F 58 43 45 69 6C 75 54 73
61 33 6C 47 4gZgl9zmrtaxXCEiluTsa3lG
```

LVM2 METADATA 확인

Volume Group을 생성하면 해당 정보를 디스크에 다음과 같이 저장한다. 해당 부분은 VG 클러스터 영역 정보이다.

```
00000FF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  BA A4 FA B9
20 4C 56 4D  ..... LVM
00001008  32 20 78 5B 35 41 25 72 30 4E 2A 3E 01 00 00 00 00 10 00 00
00 00 00 00  2 x[5A%r0N*>.....
00001020  00 F0 0F 00 00 00 00 00 00 02 00 00 00 00 00 00 00 59 03 00 00
00 00 00 00  .....Y.....
00001038  05 83 47 4C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00  ..GL.....
```

LVM2 METADATA 확인

VG 및 LV 구성이 되면, 메타정보가 생성이 된다. 이 내용은 O/S에 저장이 된다.

위치는 /etc/lvm이며, 명령어 실행 전/후 변경 내용을 가지고 있다. 실제 런타임 정보는 전부 블록 장치에 저장이 된다. LVM2로 구성한 디스크는 블록장치가 나가면 LVM2의 설정 정보가 사라지기 때문에, /etc/lvm/backup을 통해서 복구를 해야 한다.

```
00001200    74 65 73 74  76 67 20 7B  0A 69 64 20  3D 20 22 65  43 79 4C 64
52 2D 41 53  testvg {.id = "eCyLdR-AS
00001218    31 75 2D 33  56 33 31 2D  30 55 64 41  2D 6F 68 73  65 2D 4C 73
74 4D 2D 59  1u-3V31-0UdA-ohse-LstM-Y
00001230    4F 68 55 6C  5A 22 0A 73  65 71 6E 6F  20 3D 20 31  0A 66 6F 72
6D 61 74 20  0hUŁZ".seqno = 1.format
00001248    3D 20 22 6C  76 6D 32 22  0A 73 74 61  74 75 73 20  3D 20 5B 22
52 45 53 49  = "lvm2".status = ["RESI
```

LVM2의 미래

LVM2는 여전히 모든 리눅스 배포판에서 기본 사용되지만, 블록 장치 관리 복잡성 때문에 Red Hat은 Stratis 프로젝트를 통해 XFS 기반 스토리지 관리의 단순화를 추구하고 있다.

Stratis는 RHEL 7부터 시험적으로 도입되었으며, RHEL 8부터 공식 기능으로 자리잡았다.

향후 Root Filesystem 레벨까지 Stratis 통합이 진행될 가능성이 높지만, 현재(2025년 기준)까지는 ext4/xfs를 기본으로 하고 Stratis는 선택적 사용이 일반적이다.

- 스토리지 및 파일 시스템 관리는 Stratis로 통합
- BTRFS는 아직 오라클이 주도적으로 진행하고 있기 때문에 레드햇은 여전히 이 부분에 불안
- LVM2 and Device Mapper Architecture는 커널에 이미 표준 영역으로 통합
- LVM2는 미래도 계속 사용하지만, 기업용 수준에서 운영에는 적절하지 않는 부분이 여전히 남아 있다

VDO

COMPRESSED STORAGE

진행 전 준비사항

시작하기 전에 디스크 하나를 추가한다. 랩에서 필요한 디스크는 총 3개의 디스크가 필요하다.

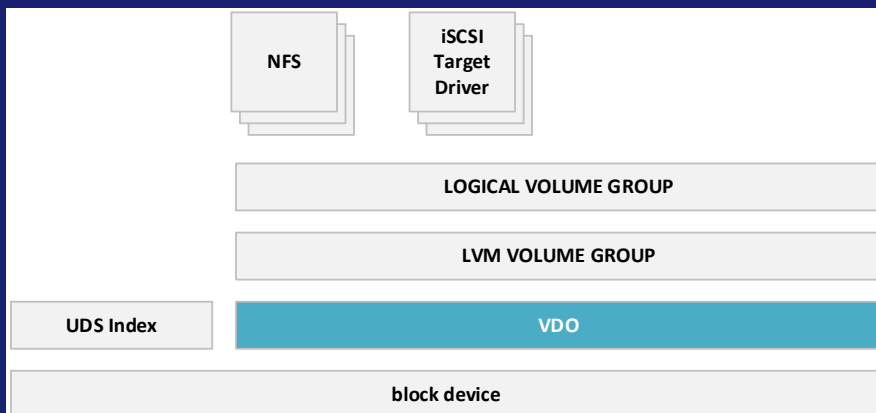
1. /dev/vdb, LVM2
2. /dev/vdc, vdo
3. /dev/vdd, Stratis

VDO 설명

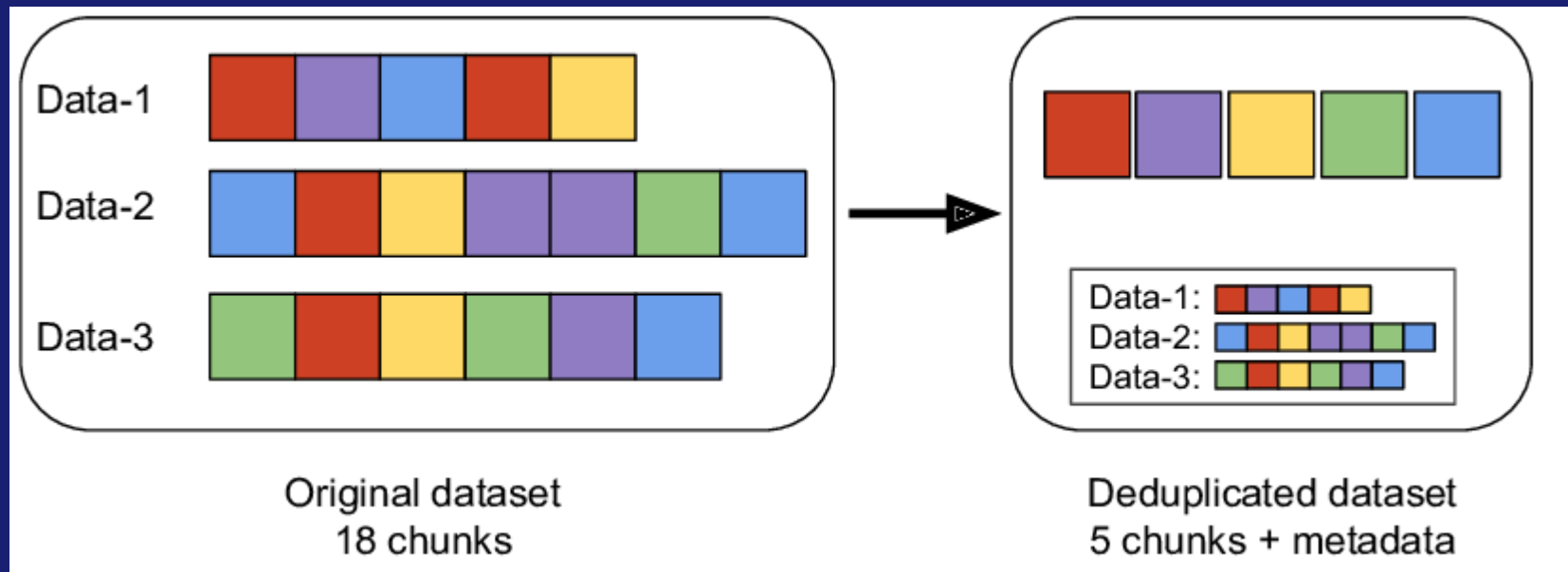
VDO는 가상머신 기반에서 사용을 많이 한다.

실제로 가상머신이 아니어도 사용이 가능하며, 같은 블록 정보가 발생이 되는 경우, VDO 기반으로 블록 데이터 저장을 권장한다. 가상머신의 이미지 디스크는 동일한 블록 데이터를 사용한다. 이는 `vdo.ko` 모듈이 블록 장치에서 확인하여 중복된 부분을 압축한다.

현재는 레드햇 기준 RHEL 8버전 이후부터는 LVM2로 통합이 되었기 때문에, 레드햇 계열 7/8버전과 9과는 사용방법이 다르다.



VDO 구조



VDO와 LVM2 관계



DUP/RAID/SNAPSHOT

설명	XFS	BTRFS	ZFS
압축	VDO기반에서 제공	inline / offline	inline
중복제거(Deduplication)	offline	inline / offline	inline
외부 메타데이터	네	아니요	네, 외부장치 명시 가능
읽기/쓰기 캐쉬	LVO/B-Cache 레이어	LVO/B-Cache 레이어	L2arc / slog
메모리 비트 로테이션 보호	아니요	네	네
레이드 지원 형식	아니요	Raid 1+10	Raid 1+raidz(5)+raidz2(6)+ raidz3(7)+10+50+60...

DUP/RAID/SNAPSHOT

설명	XFS	BTRFS	ZFS
레이드 재배치 지원	-	네	아니요
중복제거 기능 끄기(조건에 따라)	네	네	아니요
가상머신/컨테이너 스냅샷	가상머신에서만 가능	가상머신/컨테이너	가상머신/컨테이너

VDO(DE-DUPLICATE OPTIMIZATION)

LVM2 기반의 VDO(Virtual Data Optimizer) 서비스는 중복된 블록을 감지하여 하나로 압축하거나 묶어주는 블록 관리 시스템이다.

대다수 엔터프라이즈 파일 시스템(ZFS, Btrfs 등)은 내장된 중복제거(deduplication) 기능을 제공하는데, 레드햇 계열 배포판은 처음에는 btrfs로 전환하려 했지만, 성능 문제 및 안정성 이슈로 인해 기존의 XFS 파일 시스템을 계속 사용하기로 결정했다.

그러나, XFS 파일 시스템 자체는

- 중복 제거
- 압축
- 씬 프로버저닝

같은 기능을 기본적으로 제공하지 않는다. 이러한 한계를 보완하기 위해 Red Hat은 XFS 위에 VDO를 추가하여 디스크 볼륨 및 블록 최적화 기능을 제공하고 있다.

VDO 지원 배포판

VDO(Virtual Data Optimizer)는 이전에 독립적인 서비스로 사용. 지금은 LVM2기반으로 vdo 스토리지가 구성 및 관리가 된다. 레드햇 기준으로 다음과 같이 요구사항이 필요하다. 데비안 및 다른 배포판 경우에는 xfs를 지원하지만, 아직 대다수는 VDO를 지원하지 않는다.

아래 배포판은 vdo.service가 필요하다.

- RHEL 7
- RHEL 8(초기 버전)

아래 배포판은 vdo.service가 필요하지 않는다.

- RHEL 8 후기 버전/9/10

VDO 지원 배포판

Debian/Ubuntu 등 다른 배포판은 VDO 기능 일부를 지원하지만, 대부분은 아직 VDO가 공식 통합되지 않음.

배포판	동작 방식	설명
RHEL 7	vdo.service 필요	독립적인 데몬으로 VDO 관리
RHEL 8 (초기)	vdo.service 필요	LVM 통합 전까지 서비스 기반
RHEL 8.2 이후/9	vdo.service 불필요	LVM2 통합 (vdo-pool target 사용)

VDO 지원하는 디바이스 매니저

이전에는 독립적인 형태로 지원하였으나, 지금은 LVM2 기반으로만 사용이 가능하다.

기능	물리적 장치	구성되는 장치
VDO on LVM	VDO pool LV	VDO LV
LVM thin provisioning	Thin pool	Thin volume

VDO 사용시 주의사항

커널 모듈 서명이 올바르지 않아서 커널에 올라가지 못함.

Windows Hypervisor Hyper-V를 사용하는 경우, SecureBoot을 비활성화 후, 수행하면 잘 됨. 최소 크기가 3기가 이상이면 동작.

vdo 패키지는 버전에 따라서 없을 수 있음. 올바르게 `dm-vdo.ko` 모듈을 찾지 못하는 경우 아래처럼, 링크를 생성. 로키리눅스에서는 패키징 버그가 있음.

```
# dnf install vdo kmod-kvdo -y
# ln -snf ../extra/kmod-kvdo/vdo/kvdo.ko dm-vdo.ko
# depmod -a
```

VDO 구성 방법

VDO를 생성하기 위해서는 레드햇 계열은 8버전 이후부터 LVM2 기반으로 생성 및 구성해야 한다.

```
# pvcreate /dev/sdb
# vgcreate vg-vdo /dev/sdb
# lvcreate --type vdo --name lv-vdo -l 100%Free vg-vdo
# vod vdostats
```

STRATIS

XFS POOL

Stratis

레드햇에서 사용하는 XFS는 POOL기능이 없다. 이러한 부분은 LVM2로 해결을 하였지만, 운영이 복잡하고 "Pool"기능 보다 는 기존 레이드 기술과 가까운 부분이 있다. 이러한 이유로, 대규모 파일 시스템에서 제공하는 "Pool"기능을 별도의 추상적인 블록영역으로 구현하였다.

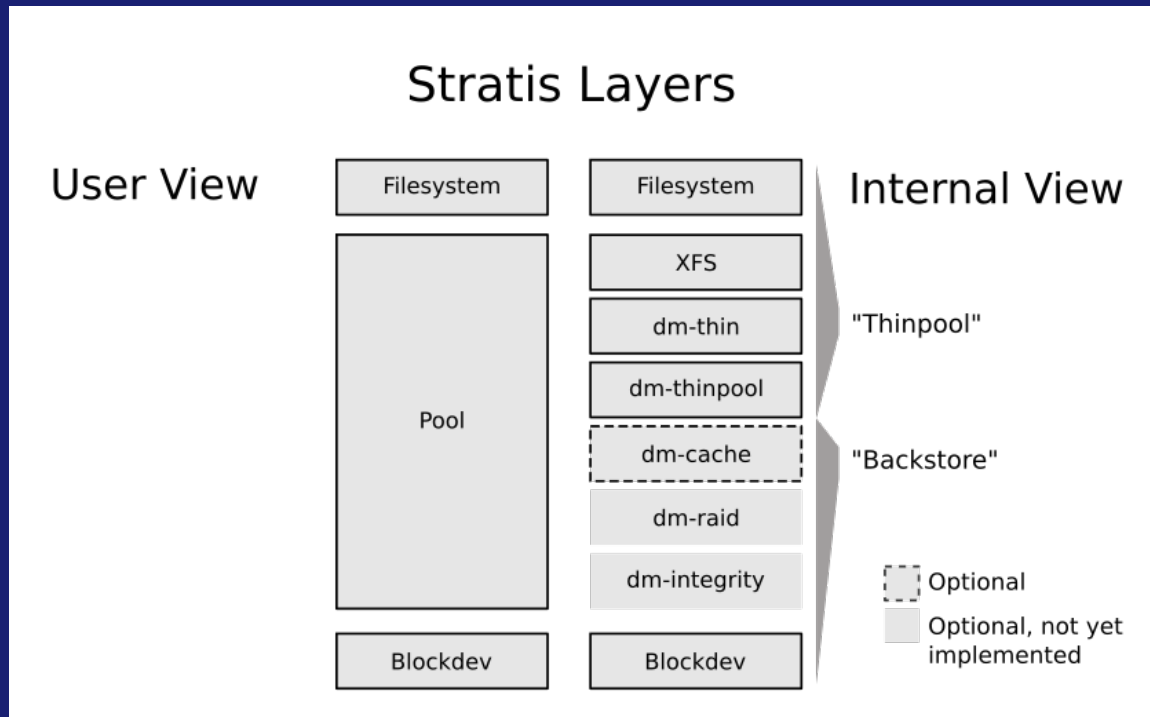
레드햇 계열 배포판에서는 **Stratis**라는 이름으로 제공하고 있으며, 이를 사용하기 위해서는 레드햇 계열 기준 7버전 이상을 권장한다. 또한, 파일 시스템 및 "Startisd"데몬 버전에 따라서 기능이 다르기 때문에, 가능한 최신 버전 사용을 권장하며, 8버전 이후로 상용 시스템에 사용하기가 적합하다.

현재 Startis는 ROOT FILESYSTEM영역도 적용이 가능하기 때문에, 앞으로 LVM2기반의 ROOT FILESYSTEM은 Stratis로 교체가 될 예정이다.

[XFS 기능 링크](#)

Stratis

Stratis는 기존에 사용하던 LVM2과 비슷하게 "DeviceMapper", "Udev"를 활용하여 백-엔드 구성이 되어있다. LVM2와 미디어 레이어를 DM를 사용하지만, 사용자가 복잡하게 관리 및 구현하는 부분이 없다.



Stratis 설치

스토리지 풀 도구를 설치한다. Stratis는 Stratisd데몬으로 관리가 되기 때문에, 반드시 데몬 서비스가 동작이 되어야 한다.

```
# dnf search stratis
# dnf install stratisd stratisd-tools stratis-cli -y
# systemctl status stratisd
# systemctl enable --now stratisd
# stratis blockdev list
# stratis pool list
# stratis pool create
# stratis pool create firstpool /dev/sdd
```


Stratis 구성

Stratis는 Pool 생성 시, 기본으로 xfs 기반으로 구성이 된다. 다른 파일 시스템으로 선택은 어려운 부분이다. 아래 명령어로 디스크를 구성하면 자연스럽게 파일시스템이 xfs으로 생성이 된다.

```
# stratis filesystem create --size 1GiB firstpool first-xfs
# stratis filesystem list
> /dev/stratis/firstpool/first-xfs
# dmsetup ls
# hexedit /dev/stratis/firstpool/first-xfs
# hexedit /dev/sdd
```

Stratis 구성

```
# stratis pool add-data firstpool /dev/sde  
# stratis pool list  
firstpool    20 GiB / 610.50 MiB / 19.40 GiB    ~Ca,~Cr, Op    63b843af-0277-4751-  
af5e-f7f0057efc56  
# stratis filesystem create --size 2GiB firstpool second-xfs
```

Stratis 구성

```
# stratis fs snapshot firstpool first-xfs snap-first-xfs
# stratis fs list
# lsblk --output=UUID /dev/stratis/test-pool/testfs
# nano /etc/fstab

UUID=<UUID> /mnt/stratis xfs defaults,x-systemd.requires=stratisd.service 0 0
```

로그

systemd-journald

journald

기존에 사용하던 **syslog(rsyslog)**를 대신하는 로깅 데몬 시스템.

제일 큰 차이점은 **journald**는 바이너리 데이터베이스 기반으로 오류 수준별로 기록을 남긴다. 아직까지는 대다수 시스템은 **rsyslogd** 기반으로 구성되어 있지만, 곧 모든 시스템은 **systemd-journald** 기반으로 변경될 예정이다.

```
# systemctl status systemd-journald
# vi /etc/systemd/journal.conf
Storage=persistent
# cp -a /run/log/journal/ /var/log/
# journalctl -b
```

journalctl 명령어

systemd기반에서는 더 이상 syslogd를 사용하지 않는다. 다만, 대다수 시스템은 호환성을 위해서 syslogd을 여전히 지원하고 있다.

여전히 로그는 syslog에도 남기고 있지만, 앞으로 systemd 기반에서는 journald 서비스로 로그 기록을 바이너리 데이터베이스로 저장한다. 이를 사용하기 위해서는 journalctl 명령어로 데이터베이스를 조회하여 유닛 및 커널 관련된 메시지 확인이 가능하다.

제일 큰 장점은 기존에 어려웠던 메시지 우선순위를 손쉽게 조회가 가능하다. 자주 사용하는 옵션은 아래와 같다.

-b	부팅 시 발생한 로그를 확인한다.
-fl	기존에 'tail -f'명령어와 동일하다.
-p	메시지 우선 순위를 필터링 합니다. err, warning, info, notice, debug와 같은 옵션을 지원한다.
-t	확인할 유닛 형식을 선택한다. 일반적으로 .service, .timer와 같이 명시한다.
-u:	유닛 이름을 명시한다.
__SYSTEMD_*	systemD키워드 명령어를 통해서 자원을 조회한다. 직접 데이터베이스 필드를 선택한다.

저널 중앙서버

중앙서버 기능을 사용하기 위해서는 아래 패키지를 설치해야 한다. 구현하기 위해서 가상서버 node1에 구성한다. node1은 journald의 서버 역할을 한다.

```
node1]# dnf install systemd-journal-remote
node1]# mkdir -p /var/log/journal/remote
node1]# vi /etc/systemd/journal-remote.conf
SplitMode=host
node1]# vi /etc/systemd/journal-upload.conf
URL=10.10.10.1:19532
node1]# cp /lib/systemd/system/systemd-journal-remote.service
/etc/systemd/system/
```

저널 중앙서버

앞에 내용 계속 이어서...

```
node1]# vi systemd-journal-remote.service
ExecStart=/usr/lib/systemd/systemd-journal-remote --listen-http=-3 --
output=/var/log/journal/remote/
node1]# firewall-cmd --add-port=19532/tcp
node1]# systemctl daemon-reload
node1]# systemctl enable --now systemd-journal-upload.service systemd-
journal-remote.service systemd-journal-remote.socket
```


저널 클라이언트

클라이언트 서버 node2는 다음과 같이 패키지를 설치 및 구성한다.

```
node2]# dnf install systemd-journal-remote
node2]# vi /etc/systemd/journal-upload.conf
[Upload]
URL=http://10.10.10.1:19532
node2]# vi systemd-journal-remote.service
ExecStart=/usr/lib/systemd/systemd-journal-remote --listen-http=-3 --
output=/var/log/journal/remote/
node2]# systemctl daemon-reload
node2]# systemctl enable --now systemd-journal-remote
node2]# systemctl enable --now systemd-journal-upload
```

저널 로그확인

아래 명령어로 올바르게 동작하는지 확인한다. 이 명령어는 node1번에서 실행한다.

```
node1]# systemd-cat ls /ls
node1]# systemd-cat cat /etc/hostname
node2]# systemd-cat cat /etc/hostname
# journalctl --file /var/log/journal/remote/remote-10.10.10.1.journal
# journalctl --file /var/log/journal/remote/remote-10.10.10.2.journal
```

저널 서버 인증키

TLS로 전송을 원하는 경우 아래 명령어로 TLS키를 생성 후 node1/2에 배포한다. 이 교육에서는 해당 부분은 다루지 않으며, 명령어만 언급한다.

저널 서버 인증키 생성

```
# openssl req -newkey rsa:2048 -days 3650 -x509 -nodes -out ca.pem -keyout  
ca.key -subj '/CN=Certificate authority/'  
# cat <<EOF> ca.conf  
[ ca ]  
default_ca = this  
[ this ]  
new_certs_dir = .  
certificate = ca.pem  
database = ./index  
private_key = ca.key  
serial = ./serial
```

저널 서버 인증키 생성

```
default_days = 3650
default_md = default
policy = policy_anything
[ policy_anything ]
countryName          = optional
stateOrProvinceName  = optional
localityName         = optional
organizationName      = optional
organizationalUnitName = optional
commonName           = supplied
emailAddress          = optional
EOF
```

저널 서버 인증키 생성

아래 명령어를 순서대로 진행한다.

```
# echo 0001 > serial
# SERVER=node1.example.com
# CLIENT=node2.example.com
# openssl req -newkey rsa:2048 -nodes -out $SERVER.csr -keyout $SERVER.key
-subj "/CN=$SERVER/"
# openssl ca -batch -config ca.conf -notext -in $SERVER.csr -out
$SERVER.pem
# openssl req -newkey rsa:2048 -nodes -out $CLIENT.csr -keyout $CLIENT.key
-subj "/CN=$CLIENT/"
# openssl ca -batch -config ca.conf -notext -in $CLIENT.csr -out
$CLIENT.pem
```

journald 영구 로깅 구성

```
# cp -a /run/log/journald /var/log/  
# vi /etc/systemd/journal.conf  
[Journal]  
Storage=persistent  
# systemctl restart systemd-journald  
# systemctl is-active systemd-journald  
# journalctl -b -1  
# journalctl -b <BOOT_ID>
```

journalctl boot 로그 검색

```
# journalctl --list-boots
```

IDX	BOOT ID	FIRST ENTRY	LAST ENTRY
0	e19e1af774df49ea84f3e461c71681b1	Fri 2024-03-29 08:55:01 KST	Fri 2024-03-29 20:15:54 KST

journalctl boot 로그 검색

```
# journalctl --since "2023-04-17 12:00:00" --until "2023-04-18 12:00:00"  
# journalctl --since yesterday -p err -p crit  
-- No entries --  
# journalctl --since 09:00 --until "1 hour ago"
```

journalctl 일반 유닛

일반 유닛은 다음과 같이 검색이 가능하다.

```
# journalctl -u httpd -l -f
```

```
Mar 29 20:16:43 test-lab.example.com systemd[1]: Starting The Apache HTTP Server ...
```

```
Mar 29 20:16:44 test-lab.example.com systemd[1]: Started The Apache HTTP Server.
```

```
Mar 29 20:16:44 test-lab.example.com httpd[43547]: Server configured, listening on:  
port 80
```

journalctl 일반 유닛

```
# journalctl -u httpd.service -u nginx.service --since today
```

```
# journalctl _PID=8080
```

```
# id -u www-data
```

```
33
```

```
# journalctl _UID=33 --since today
```

podman 컨테이너 로깅

컨테이너 런타임에서 발행한 메시지를 syslog가 아닌 journald으로 로깅하기 위해서 다음과 같이 설정한다. 도커는 아직 journald를 지원하지 않으며, Podman만 백 로깅 기능을 지원한다.

```
# vi ~/.config/containers/containers.conf
[containers]
log_driver = "journald"

# podman info | grep logDriver
logDriver: journald

# podman logs <CONTAINER_NAME>
# journalctl -f
```

커널 메시지 확인

journald에서 커널에서 발생한 메시지 확인하기 위해서 아래와 같이 명령어를 사용한다.

```
# journalctl -k
# journalctl -k -b -5
# journalctl -k -b -p err -p warning
# journalctl --output cat
```

옵션	설명
-k	커널 메세지
-b	부팅 순서 마이너스 붙을수록 과거
-p	우선순위

journald priority

-p 옵션을 통해서 동시에 여러 오류 우선 순위를 검색할 수 있다. 아래는 우선순위 번호이다.

우선 순위	우선 순위 문자 분류
0	emerg
1	alert
2	crit
3	err
4	warning
5	notice
6	info
7	debug

journalctl

별다른 수정 없이 로그를 보고 싶은 경우, 다음과 같이 명령어 실행.

```
# journalctl --no-full  
# journalctl -a  
# journalctl /usr/sbin/sshd
```

출력 방법은 변경하고 싶으면 다음과 같이 실행한다.

```
# journalctl -b -u httpd -o json  
# journalctl -b -u nginx -o json-pretty  
# journalctl -b -u sshd -o cat
```

journalctl

최근 메시지를 출력하고 싶으면 다음과 같이 한다. 로그 메시지가 얼마나 디스크 용량을 사용하는지 확인하려면 다음과 같이 한다.

```
# journalctl -n  
# journalctl -n 20  
# journalctl -f  
# journalctl --disk-usage
```


journalctl

사이즈를 줄이기 위해서 다음과 같이 명령어를 실행한다.

```
# journalctl --vacuum-size=1G  
# journalctl --vacuum-time=1years
```

서비스 관리

콘솔 로깅

콘솔 화면 로깅 설치

시스템에 사용하는 콘솔에 대해서 보안 감사가 필요한 경우, tlog를 통해서 로깅을 한다. 이전과 많이 다른 부분은 실시간으로 사용자 화면을 녹화한다.

Cockpit 기반으로 사용하기 위해서 다음처럼 설정한다. 그냥 tlog만 사용하면, 아래와 같이 설치한다.

```
# dnf install -y tlog cockpit cockpit-session-recording cockpit-ws
# systemctl enable cockpit.socket --now
# systemctl enable cockpit.service --now
# systemctl status cockpit.socket
# dnf install tlog -y
```

콘솔 명령어 로깅 설정

테스트 할 사용자 및 보안 서비스인 sssd 설정을 변경한다.

```
# groupadd -g 10000 suspicious
# useradd hacker1
# groupmems -g suspicious -a hacker1
# vi sssd-session-recording.conf
[sssd]
services=nss, pam
domains=nssfiles
```

콘솔 명령어 로깅 설정

앞에 내용 계속 이어서...

```
[domain/nssfiles]
id_provider=proxy
proxy_lib_name=files
proxy_pam_target=sssd-shadowutils
```

```
[session_recording]
scope=some
users=hacker1, tang
groups=adm
```

콘솔 로깅 재생

기록된 내용을 목록 및 재생하기 위해서 아래와 같이 작업을 수행한다.

```
# journalctl -t tlog-rec-session
# journalctl | grep tlog
# journalctl -t tlog-rec-session --output=json-pretty | less
# tlog-play -r journal -s <SESSION_ID>
# tlog-play -r journal --speed 2
```

옵션	설명
-r journal	journal에서 기록 읽기
-r file	파일 기반 기록 재생
-s <ID>	특정 세션 ID 지정
--speed <n>	재생 속도 조절 (예: 2 = 2배속)
--no-wait	입력 지연 없이 즉시 재생

네트워크

관리 명령어

네트워크

네트워크 네이밍 및 마이그레이션

네트워크 네이밍

네트워크 네이밍을 변경하기 위해서는 다음과 같이 설정들을 수정한다. 리눅스 커널이 3.x에 들어오면서 바이오스 혹은 펌웨어 기반으로 네트워크 장치 네이밍을 사용하였다. 기존에 리눅스에서 사용하던 장치 이름 방식은 보통 "eth1", "eth2:1" 이와 같이 사용하였다.

```
# vi /etc/default/grub
...
GRUB_CMDLINE_LINUX="crashkernel=auto resume=/dev/mapper/cs-swap
rd.lvm.lv=cs/root rd.lvm.lv=cs/swap biosdevname=0 net.ifnames=0"
...
# grub2-mkconfig -o /etc/grub2.cfg
```

네트워크 네이밍

혹은 아래 방법으로 전환 가능.

```
# ln -s /dev/null /etc/udev/rules.d/80-net-name-slot.rules
```

위와 같이 하는 경우, 더 이상 Dell Naming를 사용하지 않고 기존 방식으로 사용.

[dell bios naming](#)

하지만...

네트워크 네이밍

앞으로는 이 방식을 통해서 장치 이름을 변경해야 한다. systemd-networkd을 통해서 장치 이름 변경이 가능하다.

`biosdevname`, `net.ifnames` 옵션은 레드햇 기준으로 8.2 이후, 그리고 커널은 5 버전 후기 버전부터는 더 이상 이 옵션 사용이 불가능하다. 앞으로 모든 장치의 이름은 udev를 통해서 구성 및 관리가 강제적용 된다.

```
# udevadm info /sys/class/net/  
# udevadm info /sys/class/net/eth0
```

네트워크 네이밍

장치 이름은 systemd에서는 .link 유닛을 통해서 udev에서 구성해야 한다.

```
# vi /etc/systemd/network/eth1.link
[Match]
OriginalName=eth1
MACAddress=00:15:5d:44:6f:ac
[Link]
AlternativeNamesPolicy=
AlternativeName=new-eth1
```

네트워크 네이밍

커널/udev/systemd에 따라서 다르지만, 아래와 같은 기능을 제공하면 설정이 가능하다.

```
# ip link property add dev eth1 altname internal  
# ip link show eth1
```

네트워크 네이밍

배포판	기본 네이밍 정책	상세 설명
RHEL 7	systemd + biosdevname 혼용	기본 systemd 네이밍 활성화, biosdevname 지원. (biosdevname=0 net.ifnames=0 가능)
RHEL 8	systemd biosdevname=비활성 기본	완전히 systemd-udev 기반. biosdevname 기본 비활성화.
RHEL 9	systemd 100%	biosdevname 패키지 자체도 거의 배포 안 함. systemd-udev만 사용.
CentOS 7	RHEL 7과 동일	systemd + biosdevname 혼용. 기본 활성화.
CentOS 8	RHEL 8과 동일	systemd 기반. biosdevname 사용 안 함.
CentOS Stream 8/9	systemd 100%	RHEL8/9 흐름 동일하게 적용.
Fedora 25 이상	systemd 100%	biosdevname 제거. systemd-udev 기반으로 만 네이밍.

네트워크 네이밍

배포판	기본 네이밍 정책	상세 설명
Debian 9 (Stretch)	systemd 100%	기본적으로 systemd Predictable Naming 규칙 사용.
Debian 10/11/12	systemd 100%	완전히 systemd로 네트워크 이름 관리.
Ubuntu 16.04	systemd 초기 도입 biosdevname 일부 혼용	일부는 biosdevname, 일부는 systemd predictable 네이밍
Ubuntu 18.04/20.04/22.04	systemd 100%	biosdevname 거의 완전 제거, systemd만 사용.
SLES 12 SP2 이상	systemd 100%	biosdevname 제거, systemd-udev로만 네이밍.
openSUSE Leap 15.x	systemd 100%	biosdevname 지원 안 함, systemd-udev만

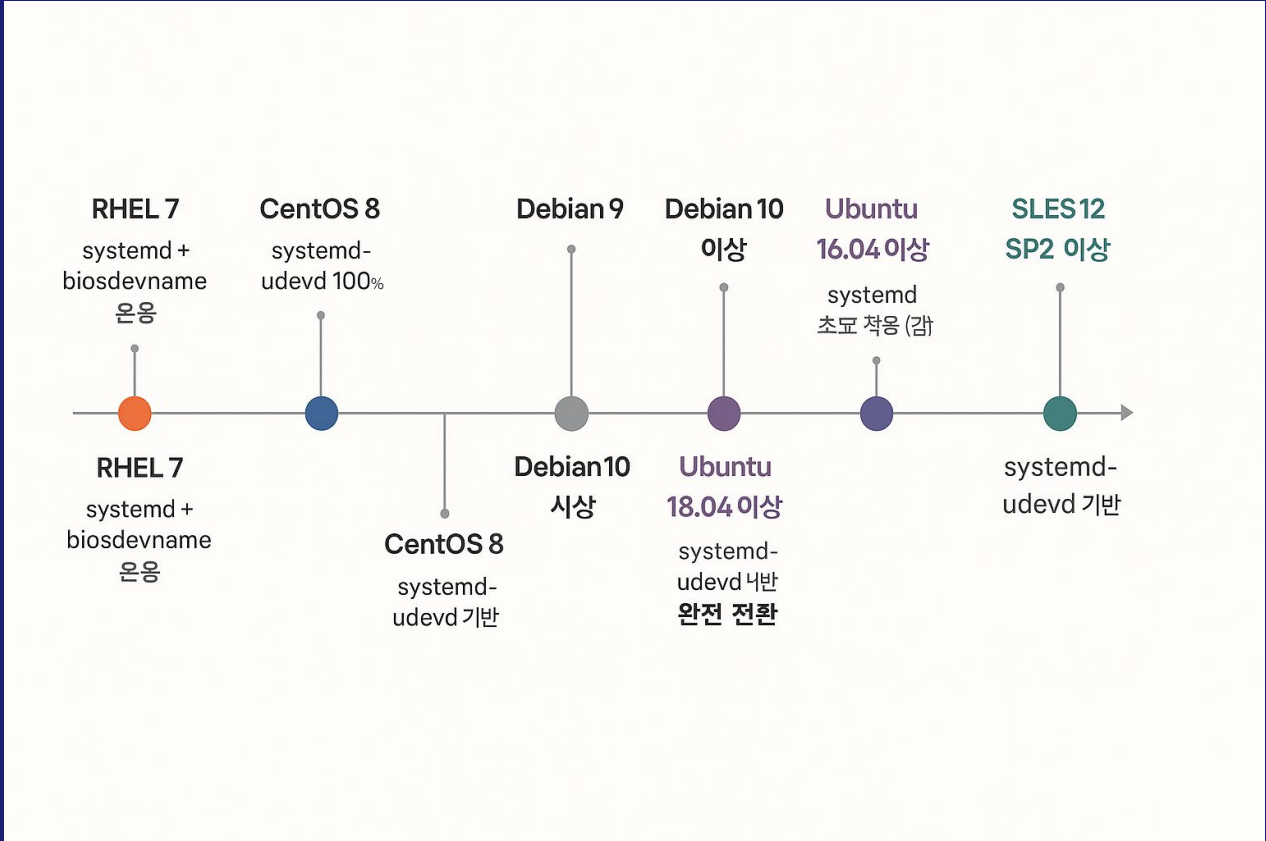
네트워크 네이밍

1. 2015~2017 전후로 등장한 최신 배포판은 전부 systemd-udev 기반 네이밍 사용.
2. biosdevname은 RHEL7, Ubuntu 16.04 이런 옛날 세대에서만 조금 남아 있음.
3. 지금은 net.ifnames=0 옵션 하나로 대부분 eth0, eth1 스타일 복구 가능.
4. biosdevname=0 옵션은 RHEL7/Ubuntu 16.04 때까지만 진짜 영향 있었음.
5. Fedora, Ubuntu, Debian, RHEL, SLES 전부 systemd-udev 완전 통합.

네트워크

systemd-networkd

systemd-network



분류자 설명

systemd에서는 분류자가 있다. 네트워크에서는 이 분류자에 따라서 자원 인식이 달라진다.

분류자	용도/설명
00-	시스템 기본 (대부분 배포판이 system-default 설정용으로 사용)
10-	기본 인터페이스 설정 (ex: eth0, wlan0 같은 단일 장치 설정)
20-	VLAN, 브릿지, 본딩 같은 가상/추가 인터페이스 설정 시작
30-	고급 네트워크 구조 (예: 본딩(bond), 브릿지(bridge) 멤버 등록)
40-	별도 라우팅 정책(Route, RoutingPolicyRule) 설정
50-	VPN (WireGuard, IPsec, Tinc 등 가상 네트워크 장치 설정)

분류자 설명

systemd에서는 분류자가 있다. 네트워크에서는 이 분류자에 따라서 자원 인식이 달라진다.

분류자	용도/설명
60-	QoS, Traffic Control (ex: Network QoS Tuning)
70-	특정 상황용 특수 인터페이스 (루프백 외 기타 가상 장치)
80-	테스트/임시 설정 (랩 환경용)
90-	사용자 오버라이드(User override) 또는 강제 적용 설정
99-	최후 적용, 강제 시스템 전체 기본(Default final) 설정

분류자 설명

앞에 숫자는 우선순위가 아니다.

포인트	설명
사전 규칙 강제 아님	systemd-networkd 숫자는 강제하지는 않음. 관리하기 쉽게 대부분 관례처럼 사용.
관리 편의성	설정이 많아질수록 숫자로 분류하면 디버깅이나 운영할 때 훨씬 편함
우선순위 아님	적용 순서는 이름 순서지만, 인터페이스 매칭은 독립적임

systemd-networkd

앞으로 모든 리눅스 배포판은 systemd기반으로 통합이 된다. 현재는 그 작업이 진행중이다. 시스템 운영에 주요 핵심 자원인 네트워크 영역은 systemd-networkd이다. 이를 통해서 네트워크 인터페이스 설정 파일 및 디바이스 관리를 지원한다.

기본 구성 파일은 INI 및 TOML형식을 지원한다.

네트워크 매니저 경우에는 nmcli가 관리 명령어처럼, systemd-networkd는 networkctl이 관리 명령어이다.

다만, networkd는 설정 파일을 수동으로 작성해야 한다. 아래는 간단하게 관리하는 명령어이다.

```
# networkctl list
# networkctl status eth0
# systemctl status systemd-networkd
# systemctl is-active systemd-networkd
```

정적 아이피 설정

정적 아이피 설정은 아래와 같이 한다.

```
# vi /etc/systemd/network/10-static.network
[Match]
Name=eth1
[Network]
Address=10.10.10.1/24
Gateway=10.10.10.254
DNS=10.10.10.254
# networkctl list
# networkctl reload
# networkctl up eth1
# networkctl status eth1
```

동적 아이피 설정

DHCP는 다음과 같이 한다.

```
# vi /etc/systemd/network/10-dhcp.network
[Match]
Name=eth0
[Network]
DHCP=yes
# networkctl list
# networkctl reload
# networkctl up eth1
# networkctl status eth1
```


VLAN 구성

먼저, 논리적 장치를 설정한다.

```
# vi /etc/systemd/network/10-vlan.netdev
[NetDev]
Name=vlan10
Kind=vlan
[VLAN]
Id=10
```

VLAN 구성

논리적 장치 구성이 완료가 되면, 아래와 같이 아이피 및 장치 이름 구성이 가능하다.

```
# vi /etc/systemd/network/10-vlan.network
[Match]
Name=vlan10

[Network]
Address=192.168.100.10/24
Gateway=192.168.100.1
```

브릿지 장치

브릿지 구성은 다음과 같이 한다.

```
# vi /etc/systemd/network/10-br0.netdev
[NetDev]
Name=br0
Kind=bridge
# vi /etc/systemd/network/10-br0.network
[Match]
Name=br0
[Network]
DHCP=yes
```

브릿지 네트워크 설정

브릿지 구성은 다음과 같이 한다.

```
# vi /etc/systemd/network/20-eth0.network  
[Match]  
Name=eth0  
  
[Network]  
Bridge=br0
```

본딩 장치 구성

본딩 구성은 다음과 같이 한다.

```
# vi /etc/systemd/network/10-bond0.netdev
[NetDev]
Name=bond0
Kind=bond

[Bond]
Mode=active-backup
MIIMonitorSec=1s
```

본딩 네트워크 설정

본딩 구성은 다음과 같이 한다.

```
# vi /etc/systemd/network/20-bond0.network  
[Match]  
Name=bond0  
  
[Network]  
DHCP=yes
```

루프백 네트워크 장치 설정

루프백 튜닝은 다음과 같이 한다.

```
# vi /etc/systemd/network/10-lo.network
[Match]
Name=lo

[Network]
Address=127.0.0.2/8
Address>::2/128
```

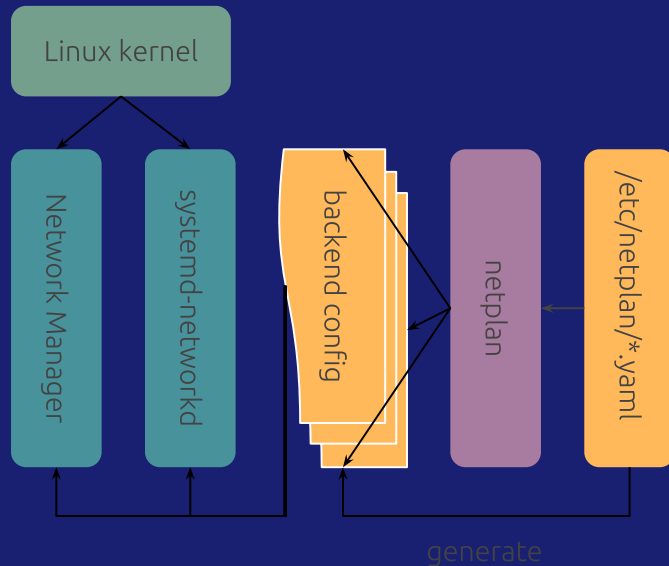
네트워크 관리자

Netplan

netplan

netplan은 `systemd-networkd` 및 `NetworkManager`를 백-엔드로 사용하는 컨트롤 플레인 도구다. 이를 통해서 앞서 이야기 하였던 두 개의 도구를 손쉽게 백-엔드 구성해서 사용한다.

다만, 레드햇 계열 배포판에서는 네트워크매니저가 기본이며, 데비안 계열에서는 `netplan` 기반으로 `NetworkManager` 및 `systemd-networkd`를 지원한다.



netplan

```
# dnf install netplan -y
# vi /etc/netplan/eth1.yaml

network:

  version: 2

  renderer: networkd

  ethernets:

    enp3s0:
```

netplan

addresses:

- 10.10.10.2/24

nameservers:

search:

- "mycompany.local"

addresses:

- 10.10.10.253
- 8.8.8.8

netplan

```
# netplan apply  
# netplan get  
# cat /etc/resolv.conf  
# ip a s eth0 | grep inet
```

네트워크 방화벽

firewalld

nftables

nftables 설명

기존에 사용하던 `iptables`는 대용량의 컨테이너 및 가상머신 운영에는 적절하지 않았다. 그래서 기존에 사용하던 `{ip, ip6, arp, eb}tables`명령어를 새로운 `in-kernel packet framework`로 변경하였다.

새로운 테이블(nftables) 프로그램은 기존에 사용하던 `Netfilter`의 정책도 호환이 가능하며, 인프라에 구성된 NAT와 그리고 사용자 영역의 큐잉(queueing)과 로그 기능을 하위 시스템으로 제공한다.

이전 `iptables`(text file)와 다른 부분은 `nftables`(JSON)기반으로 정책파일을 관리한다. 그래서 이전 테이블보다 빠르게 입출력 및 검색이 가능하다.

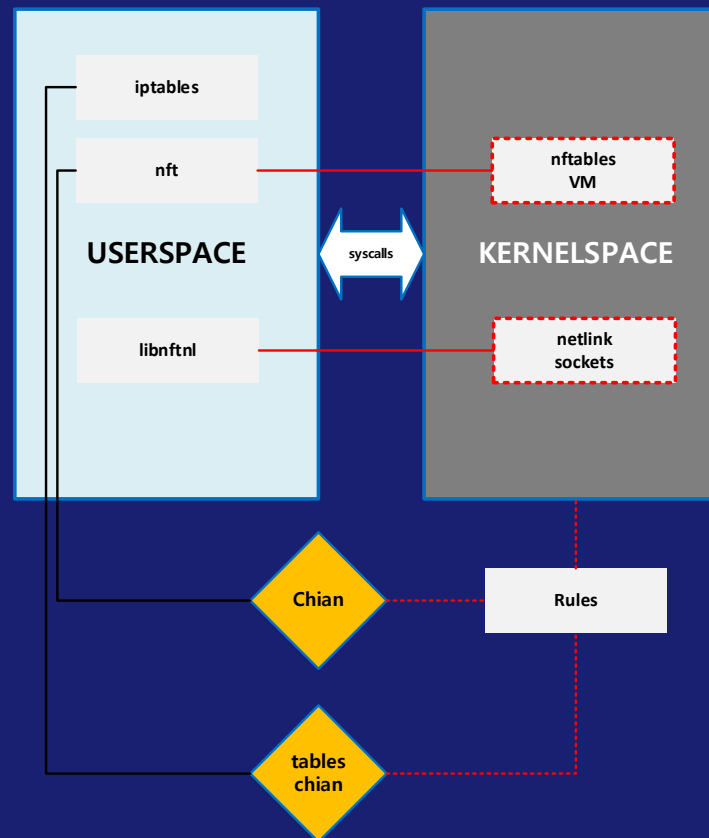
nftables 설명

nftables는 VM(Virtual Machine)를 가지고 있다. 자바의 JVM과 마찬가지로 nft-vm를 가지고 있다. 사용자가 선언한 내용은 바이트코드 형태로 컴파일되며, 이 기반으로 netlink를 구성 및 생성한다.

nft는 Netlink API를 사용하여, 이를 기반으로 커널에 작업을 수행한다.

결론적으로는, nft는 컴파일러 및 디 컴파일러가 있으며, 데이터는 JSON기반으로 되어 있다.

nftables 구조



nftables 명령어

여전히 호환성 모드로 iptables 사용은 가능하나 가급적이면 nftables 기반으로 작업 권장.

```
# dnf install -y nftables iptables-services iptables-legacy
# systemctl enable --now nftables.service
# systemctl start iptables
# systemctl is-active iptables nftables firewalld
```

nftables 명령어

정책 출력은 아래 명령어로 가능하다.

```
# nft list tables ip  
# nft list tables  
# nft list counters
```

특정 아이피 드롭.

```
# nft add rule ip filter OUTPUT ip daddr 1.2.3.4 drop
```

nftables 명령어

정책을 추가하는 명령어는 다음과 같이 사용한다. 예를 들어서 특정 아이피 드랍을 원하는 경우, 아래와 같이 사용한다.

```
# nft add rule ip filter OUTPUT ip daddr 1.2.3.4 drop
```

테이블 생성은 다음과 같이 한다. 같은 이름으로 테이블을 생성하지만, 사용하는 필터 위치가 다르기 때문에 중복이 되지 않는다.

```
# nft add table inet base_table  
# nft add table arp base_table
```

목록 출력은 아래와 같은 명령어로 가능하다.

```
# nft list tables  
# nft list tables inet
```

nftables 명령어

테이블 제거를 하기 위해서는 아래와 같이 명령어를 입력한다.

```
# nft delete table inet base_table
```

체인을 테이블에 추가하기 위해서 다음과 같이 명령어를 입력한다.

```
# nft add chain inet base_table input_filter "{type filter hook input  
priority 0;}"  
# nft -a list table inet base_table
```

80/TCP 포트를 막기 위해서 다음과 같이 명령어를 사용한다.

```
# nft add rule inet base_table input_filter tcp dport 80 drop
```

nftables 명령어

모든 rules내용을 확인하기 위해서는 다음과 같이 명령어를 사용한다.

```
# nft -a list ruleset
```

핸들러 번호로 제거하기 위해서는 다음과 같이 명령어를 사용한다.

```
# nft delete rule inet base_table input_filter handle 3
```

nftables 정책

규칙(rule)은 일괄적으로 적용하기 위해서는 JSON형태로 파일을 작성 후, nft 명령어로 밀어 넣으면 된다.
nft는 "#!"를 지원하기 때문에 아래와 같이 작성 및 사용이 가능하다.

nftables 정책

아래와 같이 수동으로 정책 파일 생성 및 관리가 가능하다.

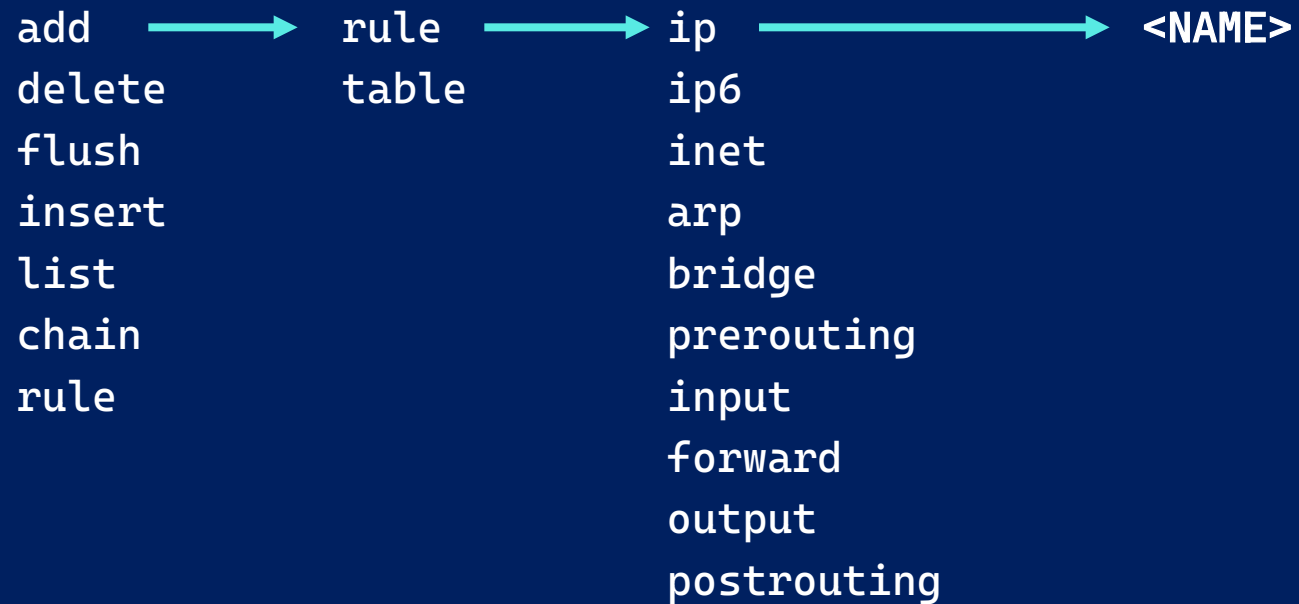
```
# vi httpd_service.sh
#! /sbin/nft -f
define http_ports = {80, 443}
flush ruleset
table inet local {
    chain input {
        type filter hook input priority 0; policy drop;
        tcp dport $http_ports counter accept comment "incoming http traffic";
    }
    chain output {
        type filter hook output priority 0; policy drop;
    }
}
# nft -a list ruleset
```



nftables

명령어 사용 방법은 다음과 같다.

nft



nftables 프로토콜

nftables의 family(protocol)은 다음과 같다. 자세한 설명은 <https://lwn.net/Articles/631372/> 참고.

ip	IPv4프로토콜 정책
arp	Address Resolution Protocol 제어 체인
ip6	IPv6프로토콜 정책
bridge	Linux Bridge 및 OVS Bridge
inet	일반적으로 많이 사용하는 애플리케이션 포트 정책
netdev	Container 및 VirtualMachine에서 사용하는 TAP(Test Access Point) 장치

nftables 명령어

대소문자 구별 합니다!



특정 아이피 드롭, 하지만 카운팅 모듈 사용.

```
# nft add rule ip filter OUTPUT ip daddr 1.2.3.4 counter drop
```

특정 아이피로 나가는 아이피 대역에 대한 카운팅.

```
# nft add rule ip filter OUTPUT ip daddr 192.168.1.0/24 counter
```

특정 포트번호에 대한 패킷 드랍.

```
# nft add rule ip filter INPUT tcp dport 80 drop
```

nftables 명령어

명령어 사용 예제. 아래 명령어가 제일 많이 사용하는 명령어 중 하나이다. 아래 명령어로 간단하게 시도한다.
ICMP 입력을 허용 합니다.

```
# nft add rule filter INPUT icmp type-echo-request accept
```

1.2.3.4로 나가는 트래픽에 대해서 거절 합니다.

```
# nft add rule ip filter OUTPUT ip protocol icmp ip daddr 1.2.3.4 counter drop
```

filter 리스트의 테이블을 확인 합니다.

```
# nft -a list table filter
```

nftables 명령어

특정 rule을 제거한다. 번호를 꼭 넣어 주어야 한다.

```
# nft delete rule filter OUTPUT handle <NUMBER>
```

모든 rule를 메모리에서 비웁니다.

```
# nft flush table filter
```

filter테이블에 80/TCP포트를 허용 합니다.

```
# nft insert rule filter INPUT tcp dport 80 counter accept
```

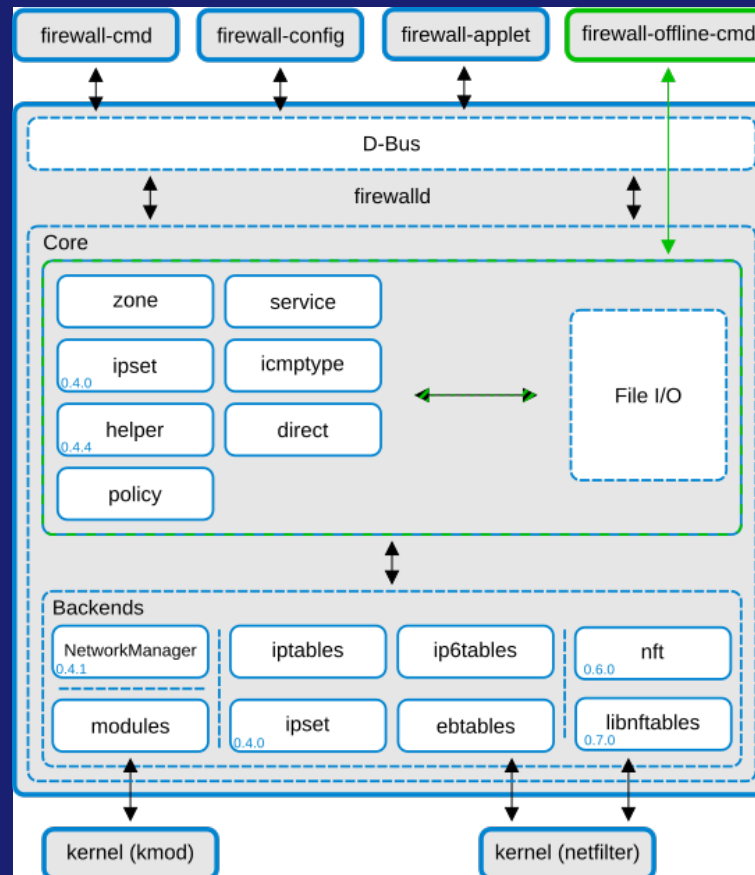
firewalld 설명

기존에 사용하는 **iptables**, **nftables** 위에 고급 계층을 올려서 사용자가 쉽게 사용할 수 있도록 한다. 현 레드햇 계열 배포판은 더 이상 **iptables**를 사용하지 않는다. 또한, **firewalld**도 **iptables**를 지원하지 않는다.

firewall-cmd 명령어 기반으로 영역(zone), 서비스(service), 포트(port)와 같은 구성 요소를 XML기반으로 선언이 가능하다. 사용자가 원하는 경우, 추가적으로 XML파일을 만들어서 추가가 가능하다.

레드햇 계열은 RHEL 9 이후부터 **nft/firewalld**만 사용이 가능하다.

firewalld 구성



firewalld 명령어

```
# systemctl start firewalld  
# ls -l /lib/firewalld  
# cd /lib/firewalld/services  
...  
http.xml  
...
```

firewalld 명령어

존(zone) 목록을 확인 및 **특정 존** 아이피/포트/서비스 및 기본 존 변경.

```
# firewall-cmd --get-zones
# firewall-cmd --list-all --zone=
# firewall-cmd --add-source=10.10.10.0/24 --zone=block --permanent
# firewall-cmd --add-service=https --zone=drop
# firewall-cmd --add-service=http                ## public zone
# firewall-cmd --add-port=8899/tcp
# firewall-cmd --set-default-zone=block
# firewall-cmd --get-default-zone
```


패키지 관리 방법

dnf/module

repository mirror

dnf

dnf는(은) 기존에 사용하던 yum 명령어를 대체하는 새로운 패키지 관리자. dnf에서 제일 큰 차이점은 바로 **modules**라는 기능이 새로 도입이 되었음.

이를 통해서 특정 프로그램 설치 시 의존성이 필요한 경우 dnf module를 통해서 패키지 제공이 된다.

또한 이 기능은 이전에 사용하였던 **SCL(Software Collection Library)**와 비슷하게 호스트 영향 없이 확장이 가능하다.

왜 module이 등장했을까?

기존 RPM의 한계

패키지 버전은 하나만 설치 가능

OS 수명(8~10년) 동안

- 애플리케이션 버전은 빠르게 변화

결과

- OS 업그레이드 없이는 최신 앱 사용 어려움
- 벤더/애플리케이션 요구사항 충돌

엔터프라이즈 환경에서 치명적

dnf module의 핵심 개념

1. Module

하나의 논리적 소프트웨어 묶음

- nodejs, python, php, postgresql

2. Stream

해당 모듈의 버전 계열

- nodejs:16
- nodejs:18

dnf module의 핵심 개념

3. Profile

설치 목적별 패키지 집합

- common
- development
- minimal

dnf module 구조

```
module: nodejs
├─ stream: 16
│   ├─ profile: common
│   └─ profile: development
└─ stream: 18
    ├─ profile: common
    └─ profile: minimal
```

확장 모듈 MODULE

모듈 목록을 아래에서 확인이 가능하다.

<https://gitlab.com/redhat/centos-stream/modules/>

모듈을 확장하기 위해서 다음과 같이 명령어를 실행한다.

```
# dnf install https://rpms.remirepo.net/enterprise/remi-release-9.rpm -y  
# dnf --enablerepo=remi-modular --disablerepo=appstream module list -y
```

module

```
# dnf module list
```

```
CentOS Stream 8 - AppStream
```

Name	Stream	Profiles Summary
container-tools	rhel8 [d][e]	common [Most recent (rolling) versions of ...

rollback

yum, dnf는 rollback기능을 제공한다.

```
dnf history      rollback
                   undo
                   redo
                   list
                   info
                   install
                   remove
                   update/upgrade
```

yum vs dnf-3

yum과 dnf의 제일 큰 차이점은 yum은 순수하게 파이선으로 작성이 되어 있다.

하지만, dnf는 libdnf3라는 C 언어로 재-구성이 되었다. 이를 통해서 기존에 yum에서 불편했던 느린 반응 및 파이썬 라이브러리 문제 발생을 방지할 수 있다.

현재 대다수 RPM 및 레드햇 기반의 배포판은 "YUM"에서 "DNF-3"로 변경이 된 상태이다.

RPM commands

RPM 명령어는 기존에는 rpm이라는 하나의 명령어만 사용하였다. RPM은 기능이 확장 되면서, 명령어 분리하기 시작. 아래 처럼 분리가 되었다.

- **rpm**: 기본 설치 명령어. 일반적으로 패키지 추가/삭제/제거 및 확인.
- **rpm2archive**: ".rpm"패키지를 ".tgz"로 변경한다.
- **rpm2cpio**: ".rpm"에서 cpio묶여 있는 파일을 푸는 명령어.
- **rpmdb**: RPM의 BerkelyDB 조회 시 사용하는 명령어.
- **rpmkeys**: RPM에서 사용하는 공개키 관리.
- **rpmquery**: RPM에서 사용하는 시그 키 관리.
- **rpmverify**: RPM에 등록된 패키지의 자원들에 대한 의존성 검사.

RPM commands

일반적으로 rpm 명령어를 통해서 패키지 관리 및 검사가 가능하다. 아래는 대표적으로 많이 사용하는 명령어이다.

```
rpm -ql <PACKAGE_NAME>
rpm -qa --last <PACKAGE_NAME>
rpm -q <PACKAGE_NAME>
rpm -qf <FILENAME>
rpm -ivh <PACKAGE_NAME>
rpm -ev <PACKAGE_NAME>
rpm -qi <PACKAGE_NAME>
rpm -Vp <PACKAGE_NAME>
rpm -Va <PACKAGE_NAME>
rpm -qa gpg-pubkey*
rpm -q --scripts systemd
rpm -q --triggers systemd
```

yum/dnf

앞서 이야기 하였지만, YUM 명령어는 DNF-3로 변경이 되었음. 저장소의 모든 RPM파일을 받고, 저장소를 구성하기 위해서는 다음과 같은 명령어로 구성이 가능함.

```
# dnf reposync
# dnf install createrepo_c
# createrepo_c .
```

위의 명령어로 구성이 완료가 되면, Apache, Nginx기반으로 저장소를 구성하면 됨. 예제는 아래와 같이 진행한다.

저장소 미러링

이전에 사용하던 미러링 기능도 dnf명령어로 통합이 되었다. 저장소 미러를 받기 위해서 다음과 같이 명령어를 수행한다.

```
# mkdir -p /mnt/reposdisk/rpms
# dnf reposync -p /mnt/repodisk/rpms
# createrepos_c /mnt/repodisk/rpms .
# dnf install httpd -y
# mount -obind /mnt/repodisk/rpms /var/www/html/
# vi /etc/yum.repos.d/mirros.repo
[poweer]
name=internal repo
baseurl=<IP>
gpgcheck=0
enable=1
```

관리도구

프로세스